

第四場

從提示詞工程了解生成式AI的本質

張哲維

(國立成功大學歷史學系助理教授)

國科會人文處歷史學門主管會議

歷史學的智慧人文轉向 - 運用生成式AI優化歷史研究流程工作坊

2026年9月19日 0900-1030

國立清華大學綜合二館8樓會議室

證據·脈絡·可驗證性
AI 賦能史學研究新視野

第二天重點

- 從AI原理探究AI本質
- AI Agent如何契合歷史研究流程
- 未來展望

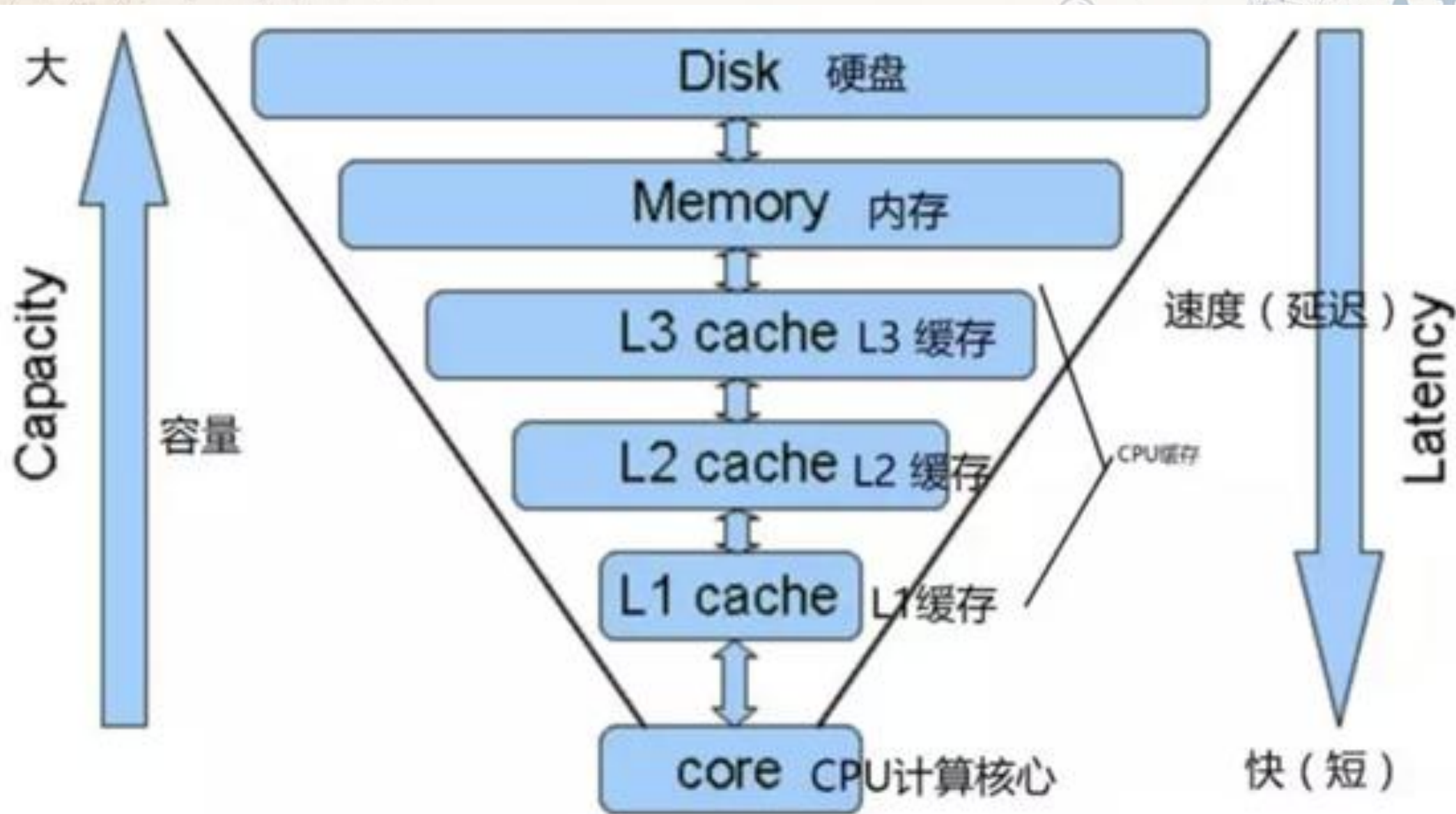
證據·脈絡·可驗證性
AI 賦能史學研究新視野

認識 程式語言

人是如何讓機器做事

證據·脈絡·可驗證性
AI 賦能史學研究新視野

給電腦指令，會發生什麼事？



指令

MIPS32 Add Immediate Instruction

001000	00001	00010	0000000010101110
OP Code	Addr 1	Addr 2	Immediate value

Equivalent mnemonic:

addi \$r1, \$r2, 350

證據·脈絡·可驗證性
AI 賦能史學研究新視野

機器碼 (machine code)

```
1 FDX 12:01a 23- 1
A 002000 C2 30 REP #$30
A 002002 18 CLC
A 002003 F8 SED
A 002004 A9 34 12 LDA #$1234
A 002007 69 21 43 ADC #$4321
A 00200A 8F 03 7F 01 STA $017F03
A 00200E D8 CLD
A 00200F E2 30 SEP #$30
A 002011 00 BRK
A 2012

r
PB PC NUmxDIZC .A .X .Y SP DP DB
; 00 E012 00110000 0000 0000 0002 CFFF 0000 00
g 2000

BREAK

PB PC NUmxDIZC .A .X .Y SP DP DB
; 00 2013 00110000 5555 0000 0002 CFFF 0000 00
m 7f03 7f03
>007F03 55 55 00 00 00 00 00 00 00 00 00 00 00 00 00:UU.....
```

AI

蒙·脈絡·可驗證性
賦能史學研究新視野

組合語言

```
FE30- 20 B4 FC 90 F7 60 B1 3C
*FDEDL
```

```
FE30- 20 B4 FC 90 F7 60 B1 3C      JMB      ( $0036 )
FE30- 20 B4 FC 90 F7 60 B1 3C      ##A0
FE30- 20 B4 FC 90 F7 60 B1 3C      $FDF6
FE30- 20 B4 FC 90 F7 60 B1 3C      $32
FE30- 20 B4 FC 90 F7 60 B1 3C      $35
FE30- 20 B4 FC 90 F7 60 B1 3C      $34
FE30- 20 B4 FC 90 F7 60 B1 3C      $FDA3
FE30- 20 B4 FC 90 F7 60 B1 3C      $FE10
FE30- 20 B4 FC 90 F7 60 B1 3C      ##BA
FE30- 20 B4 FC 90 F7 60 B1 3C      $FDC6
FE30- 20 B4 FC 90 F7 60 B1 3C      $31
FE30- 20 B4 FC 90 F7 60 B1 3C      $3E
FE30- 20 B4 FC 90 F7 60 B1 3C      ( $40 ), Y
FE30- 20 B4 FC 90 F7 60 B1 3C      $40
*
```

AI

脈絡·可驗證性
能史學研究新視野

IBM 370系統的組合語言指令書

IBM

System/370 Reference Summary

GX20-1850-2

Third Edition (February 1974)

This edition supersedes GX20-1850-1. It includes new machine instructions and control register functions associated with multi-processing, and the new instructions CS, CDS, CLRIO, IPK, and SPKA. To the extent that space allows, additional non-TP devices most often attached to System/370s have been added to the I/O command code tables.

The card is intended primarily for use by S/370 assembler language programmers. It contains basic machine information on Models 115 through 168 summarized from the *System/370 Principles of Operation* (GA22-7000), frequently used information from *OS/VS and DOS/VS Assembler Language* (GC33-4010), command codes for various I/O devices, and a multi-code translation table. The card will be updated from time to time. However, the above manuals and others cited on the card are the authoritative reference sources and will be first to reflect changes.

The names of instructions essentially new with S/370 are shown in italics. Some machine instructions are optional or not available for some models. For those that are available on a particular model, the user is referred to the appropriate functional characteristics manual. For a particular installation, one must ascertain which optional hardware features and programming system(s) have been installed. The floating-point and extended floating-point instructions, as well as the instructions listed below, are not standard on every model. Monitoring (the MC instruction) is not available on the Model 165.

Conditional swapping CDS, CS
CPU timer and clock comparator SCKC, SPT, STCKC, STPT
Direct control RDD, WRD
Dynamic address translation LRA, PTLB, RRB, STNSM, STOSM
Input/output CLRIO, SIOF
Multiprocessing SIGP, SPX, STAP, STPX
PSW key handling IPK, SPKA

Comments about this publication may be sent to the address below. All comments and suggestions become the property of IBM.

IBM Corporation, Technical Publications/Systems, Dept. 824,
1133 Westchester Avenue, White Plains, N. Y. 10604

MACHINE INSTRUCTIONS

NAME	MNEMONIC	OP CODE	FOR MAT	OPERANDS
Add (c)	AR	1A	RR	R1,R2
Add (c)	A	5A	RX	R1,D2(X2,B2)
Add Decimal (c)	AP	FA	SS	D1(L1,B1),D2(L2,B2)
Add Halfword (c)	AH	4A	RX	R1,D2(X2,B2)
Add Logical (c)	ALR	1E	RR	R1,R2
Add Logical (c)	AL	5E	RX	R1,D2(X2,B2)
AND (c)	NR	14	RR	R1,R2
AND (c)	N	54	RX	R1,D2(X2,B2)
AND (c)	NI	94	SI	D1(B1),I2
AND (c)	NC	D4	SS	D1(L,B1),D2(B2)
Branch and Link	BALR	05	RR	R1,R2
Branch and Link	BAL	45	RX	R1,D2(X2,B2)
Branch on Condition	BCR	07	RR	M1,R2
Branch on Condition	BC	47	RX	M1,D2(X2,B2)
Branch on Count	BCTR	06	RR	R1,R2
Branch on Count	BCT	46	RX	R1,D2(X2,B2)
Branch on Index High	BXH	86	RS	R1,R3,D2(B2)
Branch on Index Low or Equal	BXLE	87	RS	R1,R3,D2(B2)
Clear I/O (c,p)	CLRIO	9D01	S	D2(B2)
Compare (c)	CR	19	RR	R1,R2
Compare (c)	C	59	RX	R1,D2(X2,B2)
Compare and Swap (c)	CS	BA	RS	R1,R3,D2(B2)
Compare Decimal (c)	CP	F9	SS	D1(L1,B1),D2(L2,B2)
Compare Double and Swap (c)	CDS	BB	RS	R1,R3,D2(B2)
Compare Halfword (c)	CH	49	RX	R1,D2(X2,B2)
Compare Logical (c)	CLR	15	RR	R1,R2
Compare Logical (c)	CL	55	RX	R1,D2(X2,B2)
Compare Logical (c)	CLC	D5	SS	D1(L,B1),D2(B2)
Compare Logical (c)	CLI	95	SI	D1(B1),I2
Compare Logical Characters under Mask (c)	CLM	BD	HS	R1,M3,D2(B2)
Compare Logical Long (c)	CLCL	0F	RR	R1,R2
Convert to Binary	CVB	4F	RX	R1,D2(X2,B2)
Convert to Decimal	CVD	83		Model-dependent
Diagnose (p)				
Divide	DR	1D	RR	R1,R2
Divide	D	5D	RX	R1,D2(X2,B2)
Divide Decimal	DP	FD	SS	D1(L1,B1),D2(L2,B2)
Edit (c)	ED	DE	SS	D1(L,B1),D2(B2)
Edit and Mark (c)	EDMK	DF	SS	D1(L,B1),D2(B2)
Exclusive OR (c)	XR	17	RR	R1,R2
Exclusive OR (c)	X	57	RX	R1,D2(X2,B2)
Exclusive OR (c)	XI	97	SI	D1(B1),I2
Exclusive OR (c)	XC	D7	SS	D1(L,B1),D2(B2)
Execute	EX	44	RX	R1,D2(X2,B2)
Halt I/O (c,p)	HIO	9E00	S	D2(B2)
Halt Device (c,p)	HIOV	9E01	S	D2(B2)
Insert Character	IC	43	RX	R1,D2(X2,B2)
Insert Characters under Mask (c)	ICM	BF	RS	R1,M3,D2(B2)
Insert PSW Key (p)	IPK	B20B		
Insert Storage Key (p)	ISK	09	RR	R1,R2
Load	LR	18	RR	R1,R2
Load	L	58	RX	R1,D2(X2,B2)
Load Address	LA	41	RX	R1,D2(X2,B2)
Load and Test (c)	LTR	12	RR	R1,R2
Load Complement (c)	LCR	13	RR	R1,R2
Load Control (p)	LCTL	B7	RS	R1,R3,D2(B2)
Load Halfword	LH	48	RX	R1,D2(X2,B2)
Load Multiple	LM	98	RS	R1,R3,D2(B2)
Load Negative (c)	LNR	11	RR	R1,R2
Load Positive (c)	LPR	10	RR	R1,R2
Load PSW (n,p)	LPSW	82	S	D2(B2)
Load Real Address (c,p)	LRA	B1	RX	R1,D2(X2,B2)
Monitor Call	MC	AF	SI	D1(B1),I2
Move	MVI	92	SI	D1(B1),I2
Move	MVC	D2	SS	D1(L,B1),D2(B2)
Move	MVCL	0E	RR	R1,R2
Move Long (c)	MVNL	D1	SS	D1(L,B1),D2(B2)
Move Numerics	MVO	F1	SS	D1(L1,B1),D2(L2,B2)
Move with Offset	MVZ	D3	SS	D1(L,B1),D2(B2)
Move Zones	MR	1C	RR	R1,R2
Multiply	M	5C	RX	R1,D2(X2,B2)
Multiply	MP	FC	SS	D1(L1,B1),D2(L2,B2)
Multiply Decimal	MH	4C	RX	R1,D2(X2,B2)
Multiply Halfword	OR	16	RR	R1,R2

MACHINE INSTRUCTIONS (Contd)

NAME	MNEMONIC	OP CODE	FOR MAT	OPERANDS
OR (c)	O	56	RX	R1,D2(X2,B2)
OR (c)	OI	96	SI	D1(B1),I2
OR (c)	OC	D6	SS	D1(L,B1),D2(B2)
Pack	PACK	F2	SS	D1(L1,B1),D2(L2,B2)
Purge TLB (p)	PTLB	B20D		
Read Direct (p)	RDD	85	SI	D1(B1),I2
Reset Reference Bit (c,p)	RRB	B213	S	D2(B2)
Set Clock (c,p)	SCK	B204	S	D2(B2)
Set Clock Comparator (p)	SCKC	B206	S	D2(B2)
Set CPU Timer (p)	SPT	B208	S	D2(B2)
Set Prefix (p)	SPX	B210	S	D2(B2)
Set Program Mask (n)	SPM	04	RR	R1
Set PSW Key from Address (p)	SPKA	B20A	S	D2(B2)
Set Storage Key (p)	SSK	08	RR	R1,R2
Set System Mask (p)	SSM	80	S	D2(B2)
Shift and Round Decimal (c)	SRP	F0	SS	D1(L1,B1),D2(B2),I3
Shift Left Double (c)	S LDA	8F	RS	R1,D2(B2)
Shift Left Double Logical	S LDA	8D	RS	R1,D2(B2)
Shift Left Single (c)	S LA	8B	RS	R1,D2(B2)
Shift Left Single Logical	S LL	89	RS	R1,D2(B2)
Shift Right Double (c)	S RDA	8E	RS	R1,D2(B2)
Shift Right Double Logical	S RDL	8C	RS	R1,D2(B2)
Shift Right Single (c)	S RA	8A	RS	R1,D2(B2)
Shift Right Single Logical	S RL	88	RS	R1,D2(B2)
Signal Processor (c,p)	SIGP	AE	RS	R1,R3,D2(B2)
Start I/O (c,p)	SIO	9C00	S	D2(B2)
Start I/O Fast Release (c,p)	SIOF	9C01	S	D2(B2)
Store	ST	50	RX	R1,D2(X2,B2)
Store Channel ID (c,p)	STIDC	B203	S	D2(B2)
Store Character	STC	42	RX	R1,D2(X2,B2)
Store Characters under Mask	STCM	BE	RS	R1,M3,D2(B2)
Store Clock (c)	STCK	B205	S	D2(B2)
Store Clock Comparator (p)	STCKC	B207	S	D2(B2)
Store Control (p)	STCTL	B6	RS	R1,R3,D2(B2)
Store CPU Address (p)	STAP	B212	S	D2(B2)
Store CPU ID (p)	STIDP	B202	S	D2(B2)
Store CPU Timer (p)	STPT	B209	S	D2(B2)
Store Halfword	STH	40	RX	R1,D2(X2,B2)
Store Multiple	STM	90	RS	R1,R3,D2(B2)
Store Prefix (p)	STPX	B211	S	D2(B2)
Store Then AND System Mask (p)	STNSM	AC	SI	D1(B1),I2
Store Then OR System Mask (p)	STOSM	AD	SI	D1(B1),I2
Subtract (c)	SR	1B	RR	R1,R2
Subtract (c)	S	5B	RX	R1,D2(X2,B2)
Subtract Decimal (c)	SP	FB	SS	D1(L1,B1),D2(L2,B2)
Subtract Halfword (c)	SH	4B	RX	R1,D2(X2,B2)
Subtract Logical (c)	SLR	1F	RR	R1,R2
Subtract Logical (c)	SL	5F	RX	R1,D2(X2,B2)
Supervisor Call	SVC	0A	RR	I
Test and Set (c)	TS	93	S	D2(B2)
Test Channel (c,p)	TCH	9F00	S	D2(B2)
Test I/O (c,p)	TIO	9D00	S	D2(B2)
Test under Mask (c)	TM	91	SI	D1(B1),I2
Translate	TR	DC	SS	D1(L,B1),D2(B2)
Translate and Test (c)	TRT	DD	SS	D1(L,B1),D2(B2)
Unpack	UNPK	F3	SS	D1(L1,B1),D2(L2,B2)
Write Direct (p)	WRD	84	SI	D1(B1),I2
Zero and Add Decimal (c)	ZAP	F8	SS	D1(L1,B1),D2(L2,B2)

Floating-Point Instructions

NAME	MNEMONIC	OP CODE	FOR MAT	OPERANDS
Add Normalized, Extended (c,x)	AXR	36	RR	R1,R2
Add Normalized, Long (c)	ADR	2A	RR	R1,R2
Add Normalized, Long (c)	AD	6A	RX	R1,D2(X2,B2)
Add Normalized, Short (c)	AER	3A	RR	R1,R2
Add Normalized, Short (c)	AE	7A	RX	R1,D2(X2,B2)
Add Unnormalized, Long (c)	AWR	2E	RR	R1,R2
Add Unnormalized, Long (c)	AW	6E	RX	R1,D2(X2,B2)
Add Unnormalized, Short (c)	AUR	3E	RR	R1,R2
Add Unnormalized, Short (c)	AU	7E	RX	R1,D2(X2,B2)

c. Condition code is set.
n. New condition code is loaded.

p. Privileged instruction.
x. Extended precision floating-point.

問題

- 1. 不同系統的電腦，其組合語言都不一樣
- 2. 不容易閱讀理解
- 3. 不容易設計演算

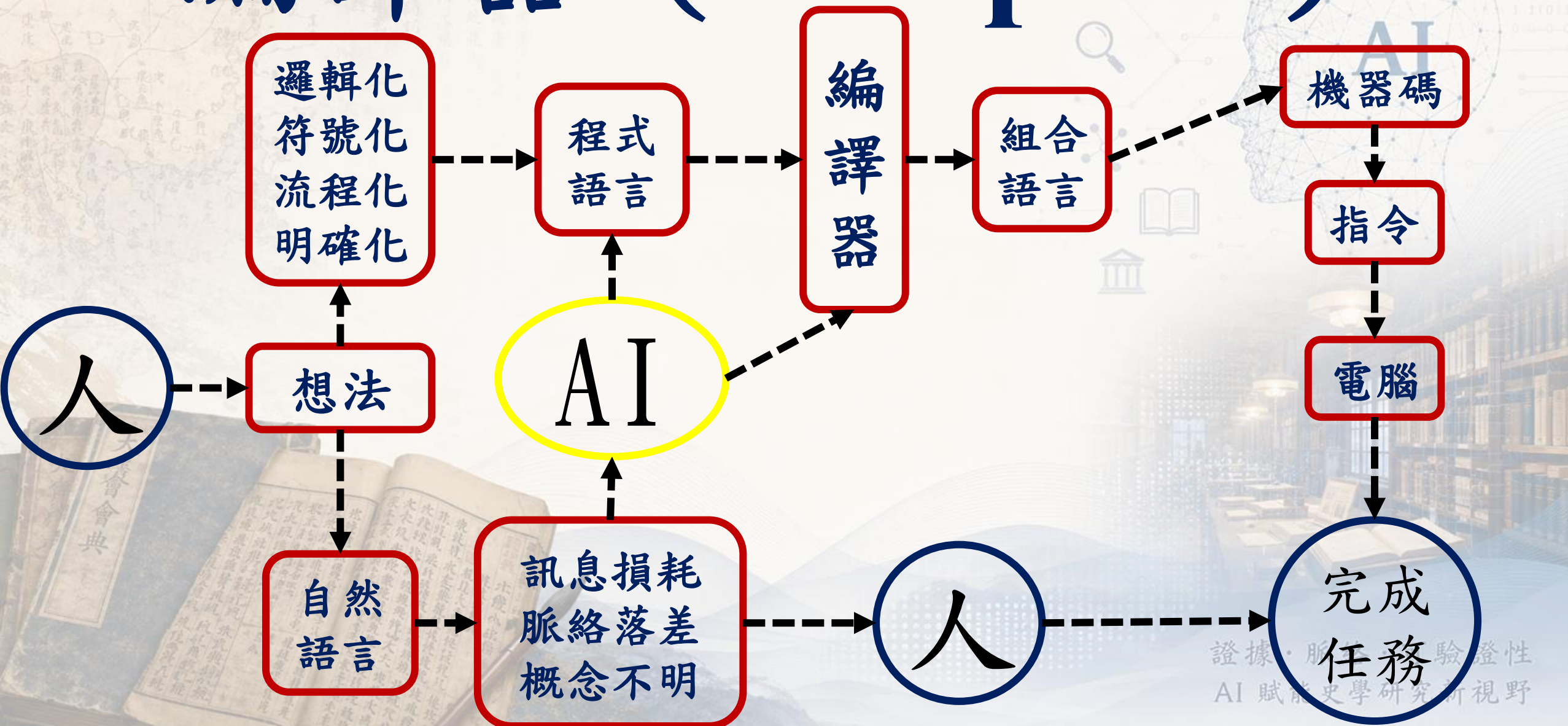
高階語言

• 如 Pascal 、
C 、 C++ 、 C# 、
Java 等



證物 性
AI 賦能 新視野

編譯器 (compiler)



Pascal

```
program yang;  
var i,j,k:integer;  
    a:array [1..7,1..13] of integer;  
Begin  
    for i:=1 to 7 do  
        Begin  
            for j:=1 to 13 do  
                a[a,j]:=0;  
                a[i,7-(i-1)]:=1;  
                a[i,7+(i-1)]:=1;  
                for k:=1 to i-2 do  
                    begin  
                        j:=7-(i-1)+2*k;  
                        a[i,j]:=a[i-1,j-1]+a[i-1,j+1];  
                    end;  
                for j:=1 to 13 do  
                    if a[i,j]=0 then  
                        write('')  
                    else  
                        write(a[i,j]:2);  
                writeln;  
            end;  
        readln;  
    end.
```

C

```
1  /*字元取代*/
2  #include<stdio.h>
3  #include<stdlib.h>
4  void change ( char *,char , char);
5  int main (){
6      char str1[] = "This is fun!";
7      char x,y;
8      printf("%s\n",str1);
9      printf("請輸入要被取代的英文字:");
10     scanf("%c",&x);
11     printf("請輸入要取代的英文字:");
12     scanf("%c",&y);
13     change( str1 , x , y);
14     printf("%s\n",str1);
15     return 0;
16 }
17 void change ( char * str1 , char x , char y)
18 {
19     while( *str1 != '\0')
20     {
21         if( *str1 == x)
22             *str1 = y;
23         str1++;
24     }
25 }
```

程式語言的演進

- 適應不同場景
- 節省資源(排程、記憶體…等等)
- 易讀性(易讀性越高，編譯器越複雜)
- —> 各種軟體、遊戲、操作系統、網站…等等

Python

```
def update_user_profile(user, data):  
    # 1. 檢查存在性 (隱含 Else: user 存在)  
    if user is None:  
        print("使用者不存在")  
        return  
  
    # 2. 檢查狀態 (隱含 Else: user 是 active 的)  
    if not user.is_active:  
        print("帳號未啟用")  
        return  
  
    # 3. 檢查資料 (隱含 Else: data 有內容)  
    if not data:  
        print("沒有資料可更新")  
        return  
  
    # 4. 核心邏輯 (完全平展，無縮排)  
    user.update(data)  
    user.save()
```

✓ 0.0s

為了方便程式設計

(環境設定、除錯、維護、版本控制、分工…
等等)

證據·脈絡·可驗證性
AI 賦能史學研究新視野

VSCode

- 是一款由微軟開發的免費、開源的輕量級程式碼編輯器，廣泛用於編寫和管理程式碼。它功能強大，支持多種程式語言（如Python、JavaScript、C++等），同時也有許多可擴展的插件和工具，適合初學者和專業開發者。



Git/Github

- Git -> 為了團隊合作，完成工作做
- Github-> 為了直接抄別人的作業/成果



git



證據 可驗證性
AI 賦 學研究新視野

Colab

- 是Google提供的基於雲端的免費Jupyter Notebook環境。使用者只需透過網頁瀏覽器，即可編寫與執行Python程式碼，無需安裝任何軟體。它特別適合資料科學、機器學習和深度學習，因為它免費提供強大的GPU和TPU運算資源，且能與Google雲端硬碟(Drive)整合。



證據·脈絡·可驗證性
AI 賦能史學研究新視野

生成式AI出現以後 ->Vibe Coding ->直接講

證據·脈絡·可驗證性
AI 賦能史學研究新視野

站在前人的肩膀上

- 我們都是有科技富爸爸，當然就要學習更美好的事物
- 「人生苦短，我用 Python」(Life is short, you need Python)
- 「人生苦短，了解就好。生命就是要浪費在你/妳覺得美好的事物上」
- 但是... 我們要先知道前一代人做過什麼/了解前人的足跡。

關於史學研究方法

- 從中世紀以來，史學研究實際操作方法本質上都沒有進步。
- 歷史學必須了解程式語言，是史學研究方法現代化的第一步。-> 了解現代數位工具的原理-> 開發有利/方便於歷史研究的工具
- 現代化後，才能進入AI時代。
- 我是黑暗時代最後的歷史學家。
- 下一代/大學生們有可能成為AI文藝復興百花齊放的新時代史家
- 方法上/工具上的變革，才是真正的典範轉移。
- Thomas Kuhn所說的典範轉移，正在發生。



Google COLAB

Download Visual Studio | (272) VSCode 如何建置 | Facebook | Download Python | Pyth | 收件匣 - a090272@gmail | 收件匣 (2) - a090272@g | Colab - Google 搜尋

google.com/search?q=Colab&rlz=1C1GCEA_enTW1137TW1137&oq=&gs_lcrp=EgZjaHJvbWUqBggAEEUYOzIGCAAQRrg7Mg4IARBFGCcYORiABBiKBTUJCAIQxgnGPAFMgclAxAAGI...

Google Colab

AI 模式 全部 影片 新聞 地圖 圖片 購物 更多 工具

Google
https://colab.research.google.com › ...
歡迎使用Colab - Colab
Colab 筆記本可讓你在單一文件中結合可執行的程式碼和RTF 格式，並附帶圖片、HTML、LaTeX 等其他格式的內容。你建立的Colab 筆記本會儲存到你的Google 雲端硬碟帳戶中...

筆記本載入錯誤
載入這個筆記本時發生錯誤。請確認檔案可以存取，然後再試一次 ...

Colab 付費服務定價
選擇合適的Colab 方案。無論是學生、業餘開發人員或機器學習研究 ...

google.com 的其他相關資訊 »

Google Colab
https://colab.research.google.com · 翻譯這個網頁
Welcome To Colab - Colab
Colab, or "Colaboratory", allows you to write and execute Python in your browser, with Whether you're a student, a data scientist or an AI researcher, Colab ...

johnsontac.com
https://johnsontac.com › posts › what-is-google-colab
【早點知道就好了】Google Colab是什麼？
2025年12月1日 — Google Colab (全稱為：Google Colaboratory)，這是個讓你不用煩惱環境設定，就可以在線上編寫程式碼的Google 服務，主要用來進行數據分析、機器學習等，Python ...

STEAM學習網
https://steam.oxxostudio.tw › python › info › online-editor
使用Google Colab - Python 教學| STEAM 教育學習網

Google Colab

問問 Gemini

colab.research.google.com/?hl=zh-tw

哲維

台灣研究機構 成功大學 AI與人文 數據分析 健康資訊 德國期刊 常用網頁 數位史學 Geschichte-Histor... 德國史料檔案 德國媒體 正義歷史 UNI Bonn 計算機、控制器、...

所有書籤

CO

搜尋筆記本

哲維

+ 新增筆記本

上傳筆記本

範例筆記本

查看全部

CO

歡迎使用 Colab

瞭解如何開始使用 Colab。

A

Antigravity 代理

掌握基本概念，瞭解如何在 Colab 建立及使用受管理代理。

Lyria

Lyria

使用功能強大的 Lyria 模型生成專屬音樂。

即時翻譯

直接在筆記本中建立即時語言翻譯工具。

近期

Colab 新功能

繁體中文文本_AI原理14階段整合教學_COLAB.ipynb

9月2日

繁體中文文本_AI原理14階段整合教學_COLAB.ipynb

9月2日

顯示更多...

Colab CLI

瞭解如何直接在終端機使用 Colab 的強大功能。[閱讀完整公告。](#)

圖表模式和全螢幕輸出分享功能

使用自然語言即時生成互動式儀表板，並透過單一連結以獨立網頁形式分享。[查看功能示範。](#)

Colab MCP 伺服器

Colab 全新的 MCP 伺服器可讓本機代理存取筆記本，控管開發生命週期。[查看開放原始碼版本。](#)

G4 GPU 支援

Colab 現已針對 Colab Pro+ 訂閱用戶，在 G4 機型上支援 NVIDIA RTX Pro 6000 Blackwell 伺服器版 GPU。[升級至 Pro+。](#)

顯示更多...

下載內容

VS Code

PyCharm

Colab CLI

Colab MCP

Colab 付費產品 · [按這裡取消合約](#)

開啟筆記本

範例 >

最近 >

Google 雲端硬碟 >

GitHub >

上傳 >



瀏覽

或者將檔案拖曳到這裡

取消

各·可驗證性
學研究新視野

> 桌面

> 筆記

> 詹峻軒報帳用

> 德國掃描圖書資料

> 德國檔案館藏台灣史料

OneDrive - 國立成功大學

> Attachments

> NCKU-HIS-智慧人文

> 台灣醫師名鑑數據資料

> 助理工作

> 待整理文章資料

> 博士論文創建表單之史

> 博士論文德翻中



Colab AI原理教學程式



pa-aa-dossier-markdown



安裝檔



範例資料



簡報

> 🖥 桌面

> 📁 筆記

> 📁 詹峻軒報帳用

> 📁 德國掃描圖書資料

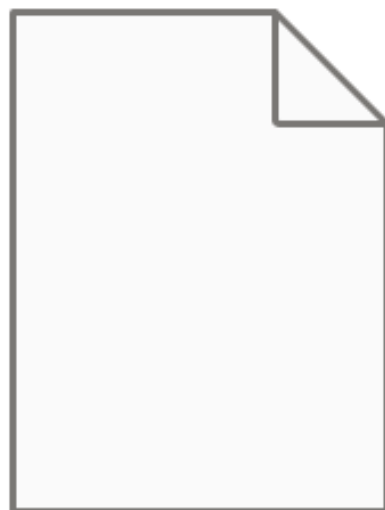
> 📁 德國檔案館藏台灣史料

✓ 🌩 OneDrive - 國立成功大學

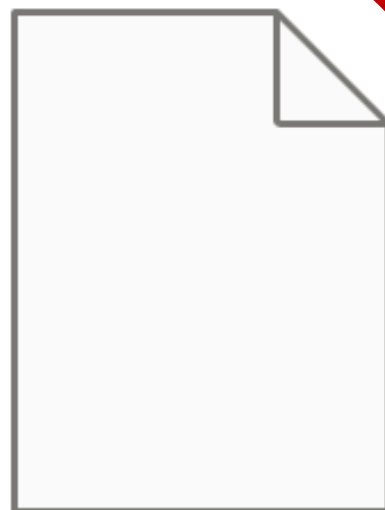
> 📁 Attachments

> 📁 NCKU-HIS-智慧人文

> 📁 台灣醫師名鑑數據資料



✓ 清大歷史RAG_Colab.ipynb



✓ 繁體中文文本_AI原理14階段
整合教學_COLAB.ipynb



繁體中文文本：從文件清理到 RNN、Attention 與 MoE 的 14 階段 AI 原理教學

這份 Google Colab Notebook 將本系列課程整合成一條可以從頭執行的流程：

1. Word / PDF 去除雜訊並轉成 Markdown。
2. N-gram 分析與報表。
3. 詞性、人物、官職、年代、地名等初步判定，輸出 TEI/XML 與 DocuSky 匯入表。
4. BPE 分析與報表。
5. 比較 N-gram 與 BPE。
6. 輸出 BPE TOKEN 字詞表。
7. One-hot 編碼。
8. Word2Vec 風格的淺層 FFNN：One-hot → Linear → 向量。
9. RNN 訓練、誤差與參數調整。
10. Multi-Head Attention 訓練並比較 RNN。
11. Mixture of Experts (MoE) 訓練並比較 RNN、Attention。
12. 使用三種模型進行簡易對話測試。
13. 加入新文本，微調 Attention 與 MoE，比較微調前後並測試對話。
14. 彙整各階段向量維度、參數量與誤差。

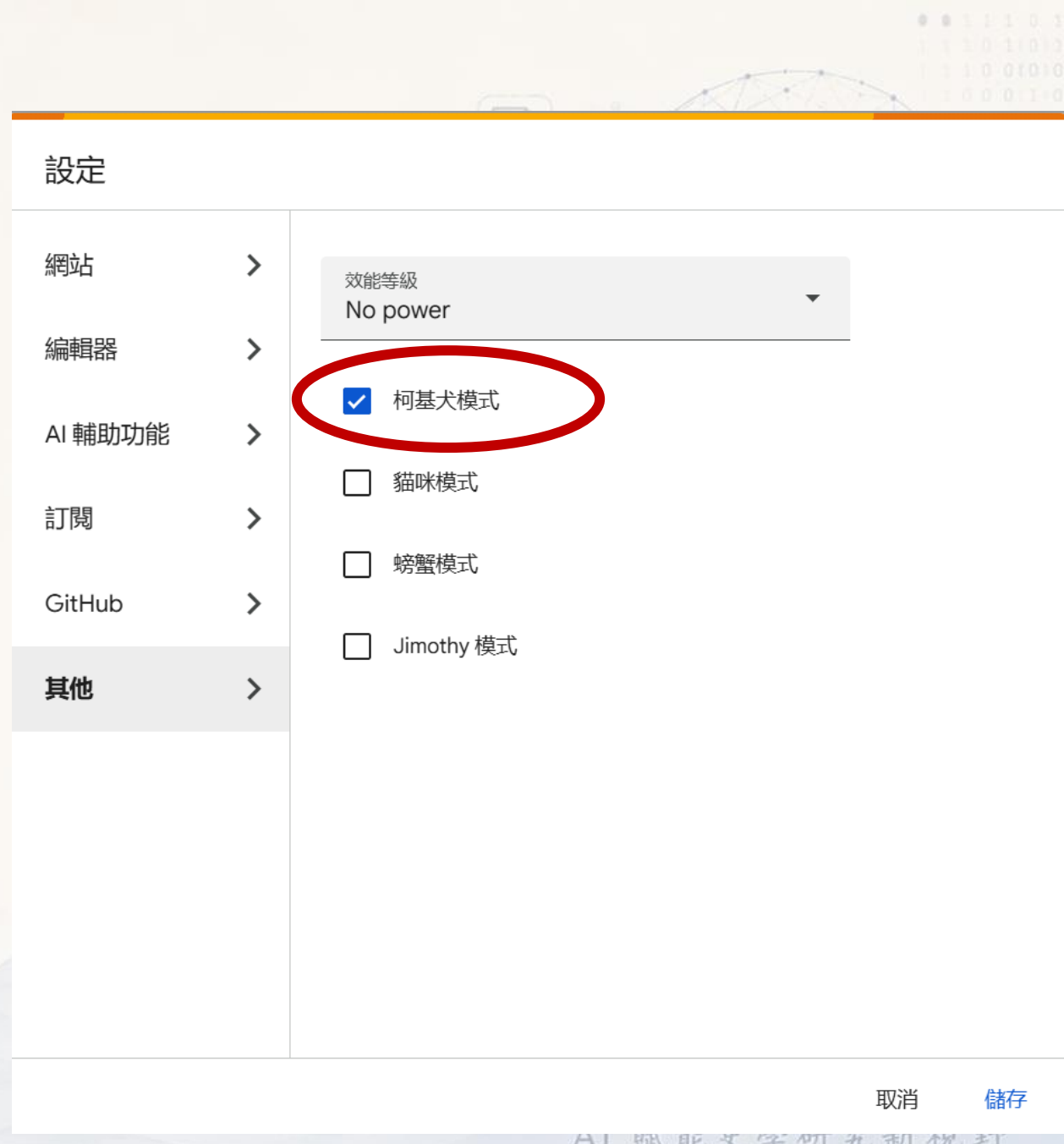
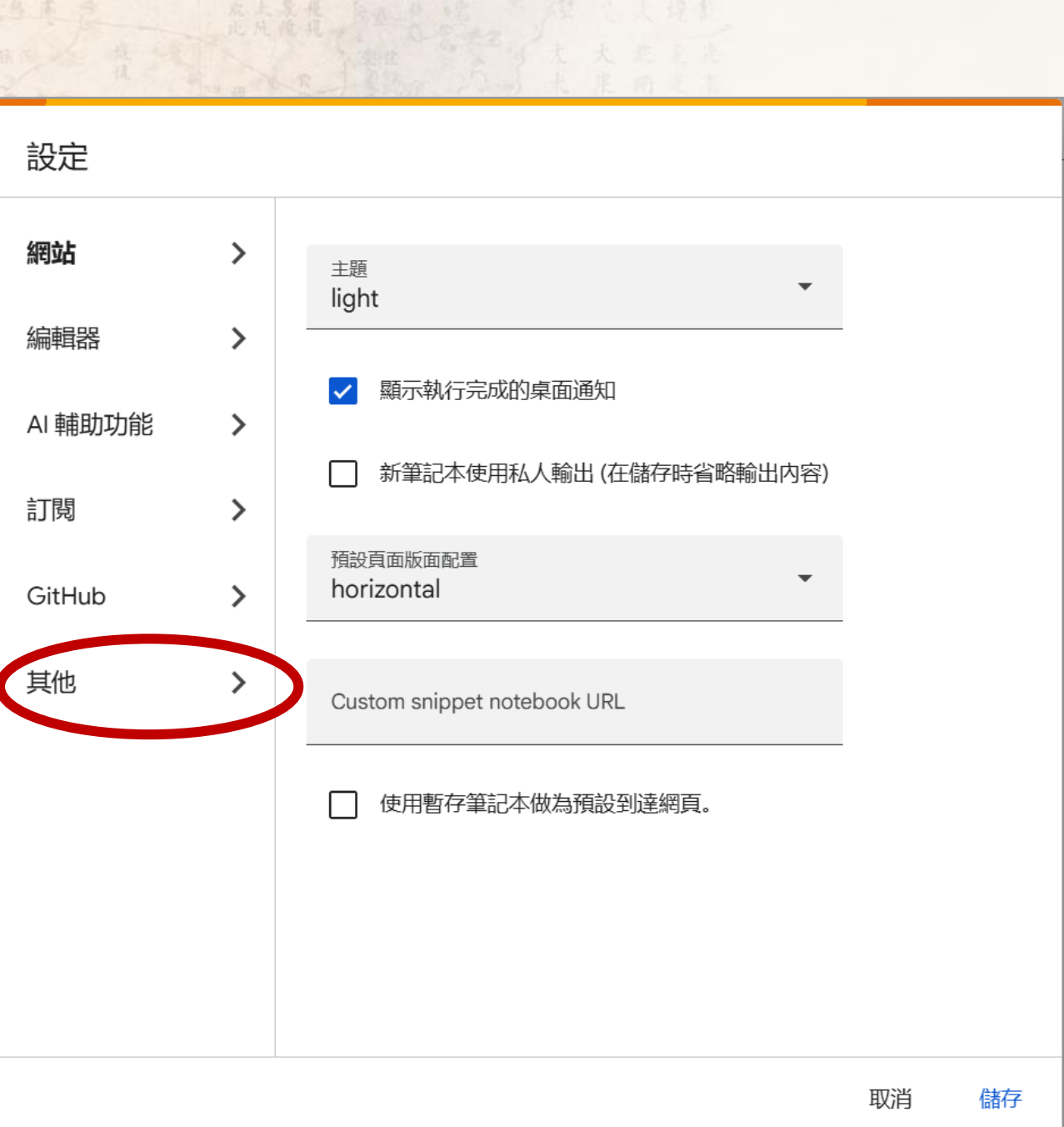
所有報表會集中在 [/content/AI原理整合教學輸出](#)，最後可一次下載 ZIP。

****2026-09-02 更新：****第13階段會自動偵測新文本的 `<unk>` / `??` 字元，保留原 BPE ID 並擴充新字元 TOKEN、Attention / MoE Embedding 與輸出層，避免《水經注》測試結果出現大量問號。

教學目的在於理解原理，不是建立可取代大型語言模型的產品。自動詞性與實體標註必須人工複核；模型生成文字不是史料原文，不能用來補寫缺文。

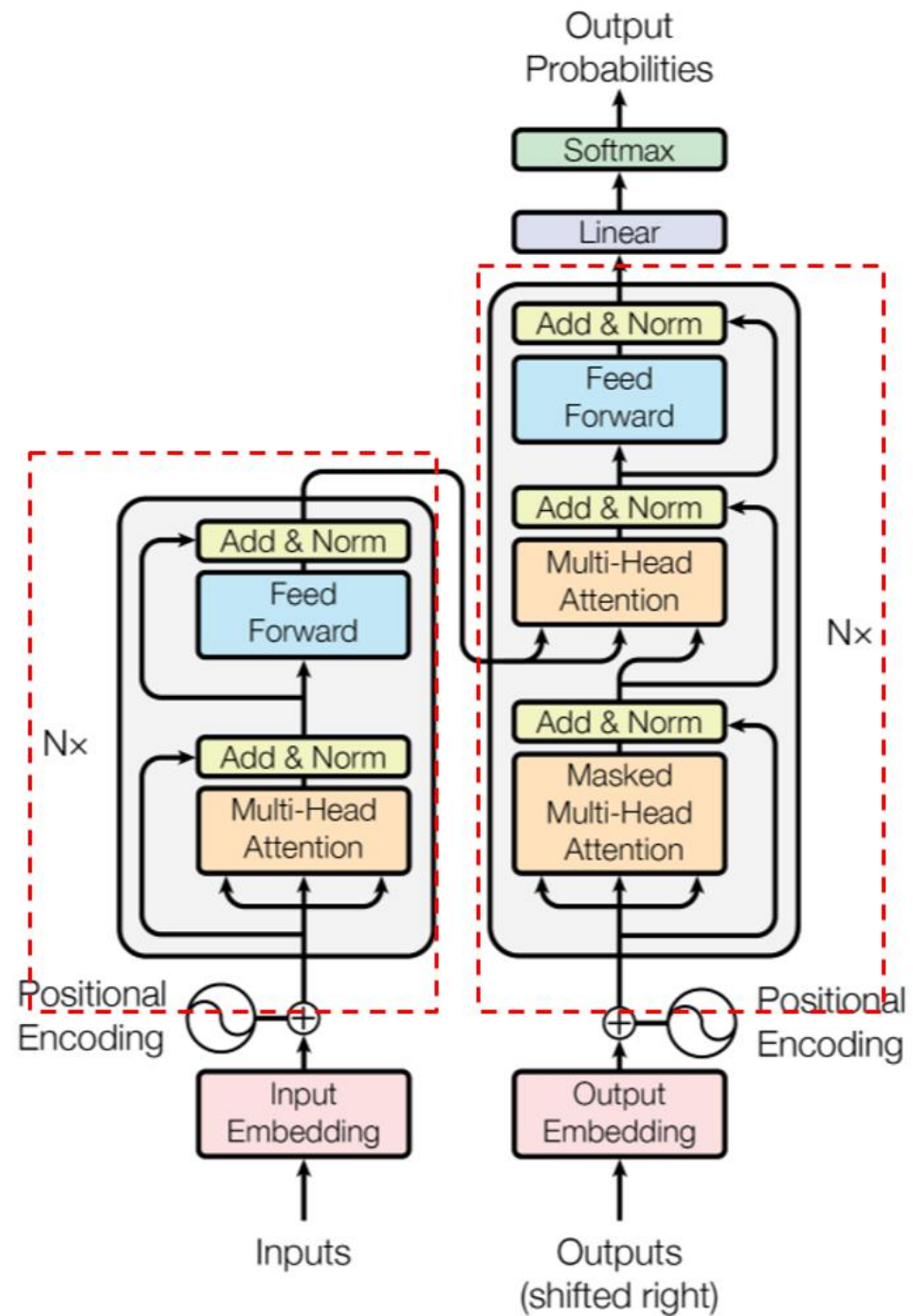
休田亡式與二話番西阳半





Transformer

AI模型架構





$$y = f(x)$$

證據·脈絡·可驗證性
AI 賦能史學研究新視野

函式

$$f(x)$$

證據·脈絡·可驗證性
AI 賦能史學研究新視野

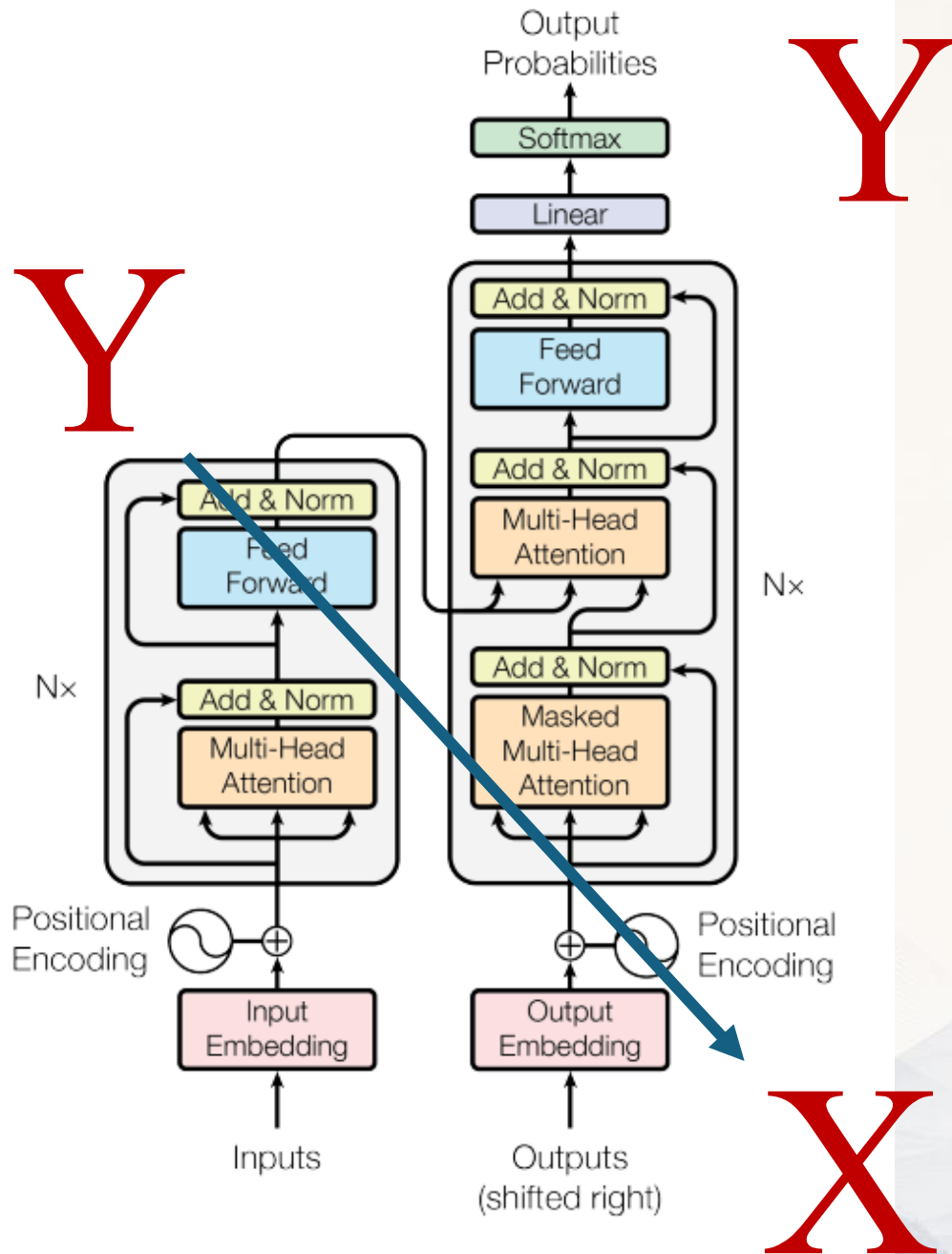


Figure 1: The Transformer - model architecture.

LLM 大型語言模型

- 本質是文字接龍、一個龐大的數學函數
- 使用者 -> 問題 -> 編碼(**Tokenizer**) -> 向量化(**Embedding**) -> LLM 計算
- LLM 計算 -> 向量 -> 解碼 -> 逐字回答 -> 使用者

TOKEN的原理

- **Token**是最基本單元，它是一個一個代表字詞的編號
- Token並非詞、也並非一對一的關係，而是AI可以處理的最小單位
- 例如：我=1、要=2、吃飯=3。
- 1、2、3就是Token
- Token和單詞並非一一對應。
- 例如：吃飯，可以拆解成：吃=4、飯=5。
- 這樣設計的好出在於：1.節省算力資源、2.幫助模型認識不認識的詞彙。
- 將文字轉化為數字、又將數字轉化成文字，這樣的過程稱之為Tokenizer。
- 平均來說，1Token=1.5~2個中文、0.75個英文

- Tokenizer 將文字轉化成代表字詞的編號 -> 詞典的概念
- Tokenizer的實現方式各不相同，例如 tiktoken (OpenAI)、WordPiece (google)
- **Tiktoken = BPE 算法**
- **WordPiece = WordPiece算法**
- Tokenizer庫的數量大小 稱之為 Vocab Size
- GPT-2的Tokenizer庫 Vocab Size只認識50257個Token
- GPT-4o的Tokenizer庫 Vocab Size認識20萬個Token
- Vocab Size越大，對於語言處理就越精細
- Tokenizer 是一種預處理，而非大模型本身，也非訓練過程
- 訓練大模型時，Tokenizer 本身是不變的。
- Token是LLM處理文本的最基本單位

<https://platform.openai.com/tokenizer>

r

OpenAI Platform

[API Docs](#)

[Log in](#)

[Sign up](#)

Tokenizer

Learn about language model tokenization

OpenAI's large language models process text using **tokens**, which are common sequences of characters found in a set of text. The models learn to understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens.

[Learn more.](#)

You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

GPT-5.x & O1/3

GPT-4 & GPT-3.5 (legacy)

GPT-3 (legacy)

Enter some text

Tokenizer

Learn about language model tokenization

OpenAI's large language models process text using **tokens**, which are common sequences of characters found in a set of text. The models learn to understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens.

[Learn more.](#)

You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

GPT-5.x & O1/3

GPT-4 & GPT-3.5 (legacy)

GPT-3 (legacy)

Ich bin CHE-CHE

Clear

Show example

Tokens

5

Characters

15

· 脈絡 · 可驗證性
能史學研究新視野

Clear

Show example

Tokens

Characters

5

15

Ich bin CHE-CHE

Text

Token IDs

Clear

Show example

Tokens

Characters

5

15

[20444, 6958, 73642, 10050, 2895]

Text

Token IDs

BPE計算

- 將句子切分片段->將片段映射到 Token ID (這過程在訓練時是自動化的)
- (Byte Pair Encoding, 位元組對編碼) 是一種資料壓縮演算法。其核心邏輯是透過「統計相鄰字元對的出現頻率」並「反覆合併最高頻率的組合」，將頻繁出現的字串片段壓縮成單一 Token，以控制詞表大小並減少序列長度。
- 在AI訓練時，透過BPE計算，將經常出現的詞組/字母串，切分成token。例如：helpful = help + ful。

教學目的在於理解原理，不是建立可取代大型語言模型的產品。自動詞性與實體標註必須人工複核；模型生成文字不是史料原文，不能用來補寫缺文。



指令 + 程式碼 + 文字 全部執行

RAM 磁碟



檔案



檔案



..

AI原理整合教學輸出

sample_data

水經注.md

洛陽伽藍記.md

繁體中文文本_AI原理14階段整合...

使用方式與三項重要限制

1. 建議在 Colab 選擇「執行階段 → 變更執行階段類型 → T4 GPU」，再依序執行。
2. 一般 DOCX、文字型 PDF 可直接擷取；掃描型 PDF 會嘗試繁體中文 OCR。手寫、古籍版面或低畫質影像仍需人工校對。
3. 本 Notebook 輸出標準 TEI P5，並另外輸出 DocuSky 表格轉換工具可使用的 Excel。DocuSky 建庫格式是 DocuXML，因此請在平台的「表格文本轉換工具」中對應 `corpus`、`filename`、`title`、`text` 等欄位後轉成 DocuXML。

DocuSky 表格文本轉換工具：<https://docusky.org.tw/DocuSky/devtools/CSVXLS2DocuXML/>

0. 安裝套件與繁體中文字型

```
[1] 40 秒
1 import importlib.util, subprocess, sys
2
3 packages = {
4     "docx": "python-docx", "pdfplumber": "pdfplumber", "pymupdf": "pymupdf",
5     "pytesseract": "pytesseract", "sentencepiece": "sentencepiece", "google.protobuf": "protobuf",
6     "jieba": "jieba", "openpyxl": "openpyxl", "lxml": "lxml"
7 }
8 missing = [pip_name for module, pip_name in packages.items() if importlib.util.find_spec(module) is None]
9 if missing:
10     subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", *missing])
11
12 # 圖表繁體中文; OCR 語言包若安裝失敗, 不中止整份 Notebook.
13 subprocess.run(["apt-get", "update", "-qq"], check=False)
14 subprocess.run(["apt-get", "install", "-y", "-qq", "fonts-noto-cjk", "tesseract-ocr", "tesseract-ocr-chi-tra"], check=False)
15 print("套件安裝完成。若上方僅有 apt 警告, 可繼續執行; 掃描 PDF 的 OCR 會另行檢查。")
16
```

... 套件安裝完成。若上方僅有 apt 警告, 可繼續執行; 掃描 PDF 的 OCR 會另行檢查。

磁碟 64.96 GB 可用

```
[2] 7秒
1 from __future__ import annotations
2
3 import copy, io, math, random, re, shutil, time, warnings
4 from collections import Counter, defaultdict
5 from pathlib import Path
6
7 import jieba
8 import jieba.posseg as pseg
9 import matplotlib.pyplot as plt
10 from matplotlib import font_manager
11 import numpy as np
12 import pandas as pd
13 import pdfplumber
14 import pymupdf
15 import pytesseract
16 import sentencepiece as spm
17 import torch
18 import torch.nn.functional as F
19 from docx import Document
20 from google.colab import files
21 from IPython.display import display
22 from lxml import etree
23 from scipy import sparse
24 from torch import nn
25 from torch.utils.data import DataLoader, TensorDataset
26
27 warnings.filterwarnings("ignore", category=UserWarning)
28 SEED = 42
29 random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
30 if torch.cuda.is_available(): torch.cuda.manual_seed_all(SEED)
31 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

檔案



[2] 7秒



..

AI原理整合教學輸出

sample_data

水經注.md

洛陽伽藍記.md

繁體中文文本_AI原理14階段整合...

磁碟

64.96 GB 可用

```
30 if torch.cuda.is_available(): torch.cuda.manual_seed_all(SEED)
31 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
32 OUTPUT_DIR = Path("/content/AI原理整合教學輸出")
33 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
34
35 font_candidates = [Path("/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"), Path("/usr/share/fonts/opentype/noto/NotoSerifCJK-Regular.ttc")]
36 font_path = next((p for p in font_candidates if p.exists()), None)
37 if font_path:
38     font_manager.fontManager.addfont(str(font_path))
39     FONT_NAME = font_manager.FontProperties(fname=str(font_path)).get_name()
40     plt.rcParams["font.family"] = FONT_NAME
41     plt.rcParams["font.sans-serif"] = [FONT_NAME]
42 else:
43     FONT_NAME = "sans-serif"
44 plt.rcParams["axes.unicode_minus"] = False
45 print(f"裝置: {DEVICE} | 繁體中文字型: {FONT_NAME} | 輸出: {OUTPUT_DIR}")
46
47 def ask_int(prompt, default, low, high):
48     while True:
49         answer = input(f"{prompt} [預設 {default}]: ").strip()
50         if not answer: return default
51         try:
52             value = int(answer)
53             if low <= value <= high: return value
54         except ValueError: pass
55         print(f"請輸入 {low} 至 {high} 的整數。")
56
57 def ask_float(prompt, default, low, high):
58     while True:
59         answer = input(f"{prompt} [預設 {default}]: ").strip()
60         if not answer: return default
61         try:
62             value = float(answer)
63             if low <= value <= high: return value
```



繁體中文文本 AI原理14階段整合...

```

/usr/local/lib/python3.13/dist-packages/jieba/_init_.py:44: SyntaxWarning: invalid escape sequence '\.'
    re_han_default = re.compile("(\\uE00-\\u9FD5a-zA-Z0-9+#[\\._%\\-\\+)*", re.U)
/usr/local/lib/python3.13/dist-packages/jieba/_init_.py:46: SyntaxWarning: invalid escape sequence '\s'
    re_skip_default = re.compile("(\\r\\n\\s)", re.U)
/usr/local/lib/python3.13/dist-packages/jieba/finalseg/_init_.py:78: SyntaxWarning: invalid escape sequence '\.'
    re_skip = re.compile("[a-zA-Z0-9]+(?:\\.\\d+)?%?")
/usr/local/lib/python3.13/dist-packages/jieba/posseg/_init_.py:16: SyntaxWarning: invalid escape sequence '\.'
    re_skip_detail = re.compile("(\\[.0-9\\+][a-zA-Z0-9\\+)*")
/usr/local/lib/python3.13/dist-packages/jieba/posseg/_init_.py:17: SyntaxWarning: invalid escape sequence '\.'
    re_han_internal = re.compile("(\\uE00-\\u9FD5a-zA-Z0-9+#[\\._\\+)*")
/usr/local/lib/python3.13/dist-packages/jieba/posseg/_init_.py:18: SyntaxWarning: invalid escape sequence '\s'
    re_skip_internal = re.compile("(\\r\\n\\s)")
/usr/local/lib/python3.13/dist-packages/jieba/posseg/_init_.py:21: SyntaxWarning: invalid escape sequence '\.'
    re_num = re.compile("(\\[.0-9\\+)*")

```

處理策略：

磁碟 64.96 GB 可用

 變數 終端機

T4 (Python 3)

指令 + 程式碼 + 文字 | 全部執行

檔案



▼

```
re_num = re.compile("[\.\d-9]+")
```

裝置: cuda | 繁體中文字型: Noto Sans CJK JP | 輸出: /content/AI原理整合教學輸出

- 1. Word / PDF 去除雜訊並轉成 Markdown

處理策略：

- DOCX: 依段落順序擷取，保留 Word Heading 樣式為 Markdown 標題。
- 文字型 PDF: 依頁擷取文字。
- 掃描 PDF: 若文字層過少，嘗試 `chi_tra` OCR。
- 自動移除重複頁首 / 頁尾、單獨頁碼、控制字元與過多空白。
- 不確定是否為正文的短行只做保守處理，避免誤刪史料。

12
利

```

1 PAGE_NUMBER_RE = re.compile(r"^\s*(?:第\s*)?[0-9 0-9一二三四五六七八九十百千IVXLCDMiVxIcdm]+(?:\s*頁)?\s*$")
2
3 def decode_text(data):
4     for enc in ("utf-8-sig", "utf-8", "cp950", "big5", "utf-16"):
5         try: return data.decode(enc)
6         except UnicodeDecodeError: pass
7     return data.decode("utf-8", errors="replace")
8
9 def extract_docx(data):
10     doc = Document(io.BytesIO(data)); lines = []
11     for paragraph in doc.paragraphs:
12         text = paragraph.text.strip()
13         if not text: lines.append(""); continue
14         style = (paragraph.style.name or "").lower()
15         match = re.search(r"heading\s*(\d+)", style)
16         lines.append(("#" * min(6, int(match.group(1))) + " " if match else "") + text)
17     return ["\n".join(lines), "DOCX文字層"]
18
19 def main():
20     # 从 docx 文件中提取文字层
21     docx_data = open("docx_data.docx", "rb").read()
22     docx_text, docx_header = extract_docx(docx_data)
23     # 解码文字层
24     docx_text = decode_text(docx_text)
25     # 输出结果
26     print(docx_header)
27     print(docx_text)
28
29 if __name__ == '__main__':
30     main()

```

檔案

📁 ..

📁 AI原理整合教學輸出

📁 sample_data

📄 水經注.md

📄 洛陽伽藍記.md

📄 繁體中文文本_AI原理14階段整合...

```
91 markdown_text, noise_df, extraction_mode = extract_and_clean(source_filename, source_bytes)
92 plain_text = re.sub(r"^\s*#{1,6}\s+", "", markdown_text, flags=re.MULTILINE)
93 plain_text = re.sub(r"\n{3,}", "\n\n", plain_text).strip()
94 if len(re.sub(r"\s", "", plain_text)) < 20: raise ValueError("清理後正文過短，請檢查檔案或 OCR 品質。")
95
96 md_path = OUTPUT_DIR / "01_清理後文本.md"
97 md_path.write_text(markdown_text, encoding="utf-8")
98 noise_path = OUTPUT_DIR / "01_雜訊清理報表.xlsx"
99 save_excel(noise_path, {"處理摘要":pd.DataFrame([{"來源檔":source_filename,"擷取方式":extraction_mode,"清理後字元":len(plain_text),"移除行數":len(noise_df)}]), "移
100 print(f"擷取方式: {extraction_mode}\n\nMarkdown 預覽: \n{markdown_text[:800]}")
101
```

請上傳主要文本 (DOCX、PDF、TXT 或 Markdown):

選擇檔案 未選擇任何檔案

Uploading widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 洛陽伽藍記.md to 洛陽伽藍記.md

已輸出: 01_雜訊清理報表.xlsx

擷取方式: 純文字/Markdown

Markdown 預覽:

作者: "[[楊銓之]]"

出版年: "[[西元547年]]"

序

三墳五典之說。九流百代之言。並理在人區而義兼天外。至於一乘二諦之原。三明六通之旨。西域備詳。東土靡記。自頂日感夢滿月流光。陽門飾豪眉之像。夜臺圖紺髮之形爾來奔競其風遂廣。至晉永嘉。唯有寺四十二所。刻

2. N-gram 分析並匯出報表

從使用者指定的最大 N 依序分析到 N=2。沿用先前規則:

- N > 4: 出現次數至少3次。
- N = 3: 出現次數至少4次。
- N = 2: 出現次數至少5次。



新分頁

新分頁

回到Obsidian

建立新檔案 (Ctrl + N)

開啟檔案 (Ctrl + O)

關閉窗格

> 📁 應用程式

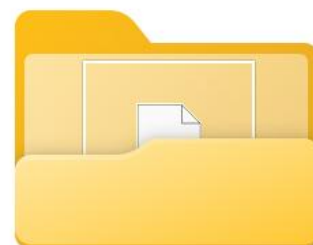
- ❌ 🖥 桌面
- ↓ 下載
- 📁 文件
- 🖼 圖片
- 🎵 音樂
- 🎬 影片
- 📁 H:\
- 📁 I:\
- 📁 G:\

- 📁 課程PPT
- 📁 20260918國科會歷史學門
- 📁 研究室工作會議
- 📁 7. 史學研究的數位工具 第

> 📁 Synology Drive - Chang

✓ 🖥 本機

> 📁 Windows (C:)



🔄 .obsidian



✅ 1954年台灣醫師名鑑



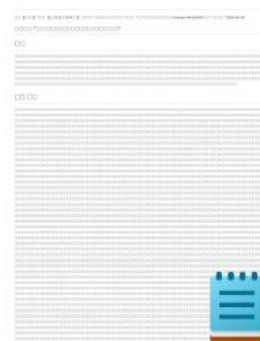
✅ RZ 208_86377 Formosa



✅ 手稿筆記



✅ 法肯豪森將軍手稿



✅ 水經注.md



✅ 洛陽伽藍記.md



✅ 洛陽伽藍記.pdf



✅ 張哲維 當事人視角下的陶德曼調停 新史學 2025 36卷2期.pdf



✅ 當事人視角下的陶德曼調停.md

可驗證性
究新視野

檔案

📁 ..

📁 AI原理整合教學輸出

📁 sample_data

📄 水經注.md

📄 洛陽伽藍記.md

📄 繁體中文文本_AI原理14階段整合...

```
91 markdown_text, noise_df, extraction_mode = extract_and_clean(source_filename, source_bytes)
92 plain_text = re.sub(r"^\s*#{1,6}\s+", "", markdown_text, flags=re.MULTILINE)
93 plain_text = re.sub(r"\n{3,}", "\n\n", plain_text).strip()
94 if len(re.sub(r"\s", "", plain_text)) < 20: raise ValueError("清理後正文過短，請檢查檔案或 OCR 品質。")
95
96 md_path = OUTPUT_DIR / "01_清理後文本.md"
97 md_path.write_text(markdown_text, encoding="utf-8")
98 noise_path = OUTPUT_DIR / "01_雜訊清理報表.xlsx"
99 save_excel(noise_path, {"處理摘要":pd.DataFrame([{"來源檔":source_filename,"擷取方式":extraction_mode,"清理後字元":len(plain_text),"移除行數":len(noise_df)}]), "移
100 print(f"擷取方式: {extraction_mode}\n\nMarkdown 預覽: \n{markdown_text[:800]}")
101
```

請上傳主要文本 (DOCX、PDF、TXT 或 Markdown):

選擇檔案 未選擇任何檔案

Uploading widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 洛陽伽藍記.md to 洛陽伽藍記.md

已輸出: 01_雜訊清理報表.xlsx

擷取方式: 純文字/Markdown

Markdown 預覽:

作者: "[[楊銓之]]"

出版年: "[[西元547年]]"

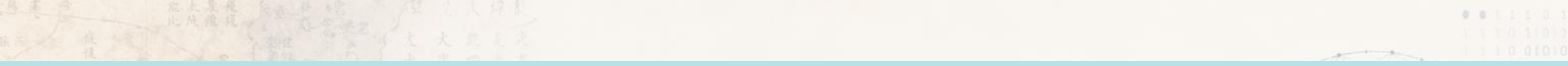
序

三墳五典之說。九流百代之言。並理在人區而義兼天外。至於一乘二諦之原。三明六通之旨。西域備詳。東土靡記。自頂日感夢滿月流光。陽門飾豪眉之像。夜臺圖紺髮之形爾來奔競其風遂廣。至晉永嘉。唯有寺四十二所。刻

2. N-gram 分析並匯出報表

從使用者指定的最大 N 依序分析到 N=2。沿用先前規則:

- N > 4: 出現次數至少3次。
- N = 3: 出現次數至少4次。
- N = 2: 出現次數至少5次。



檔案



- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

2. N-gram 分析並匯出報表

從使用者指定的最大 N 依序分析到 N=2。沿用先前規則：

- N > 4: 出現次數至少3次。
- N = 3: 出現次數至少4次。
- N = 2: 出現次數至少5次。

較長 N-gram 一旦選入，對應位置不再交給較短 N 重複計算，便於觀察長詞優先的候選結果。



```
1 def ngram_min_count(n): return 3 if n > 4 else 4 if n == 3 else 5
2
3 def chinese_segments_with_offsets(text):
4     return [(m.group(), m.start()) for m in re.finditer(r"[\u3400-\u9fff]+", text)]
5
6 max_n = ask_int("N-gram 最大 N", 6, 2, 20)
7 blocked = np.zeros(len(plain_text), dtype=bool)
8 all_rows, selected_rows = [], []
9 for n in range(max_n, 1, -1):
10     occurrences = defaultdict(list)
11     for segment, base in chinese_segments_with_offsets(plain_text):
12         for i in range(max(0, len(segment)-n+1)):
13             start, end = base+i, base+i+n
14             if not blocked[start:end].any(): occurrences[segment[i:i+n]].append((start, end))
15     threshold = ngram_min_count(n)
16     ranked = sorted(occurrences.items(), key=lambda item: (-len(item[1]), item[0]))
17     for gram, spans in ranked:
18         usable_spans = [(start, end) for start, end in spans if not blocked[start:end].any()]
19         row = {"N": n, "N-gram": gram, "次數": len(usable_spans), "門檻": threshold, "達門檻": len(usable_spans) >= threshold}
20     all_rows.append(row)
```





檔案



- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

磁碟 64.96 GB 可用

```
[4] 29 ngram_path = OUTPUT_DIR / '02_N-gram分析報表.xlsx'
    30 save_excel(ngram_path, {"入選N-gram":ngram_selected_df,"全部候選":ngram_all_df,"設定":pd.DataFrame([{"最大N":max_n,"N大於4門檻":3,"N等於3門檻":4,"N等於2門檻":5}]})
    31 display(ngram_selected_df.head(50))
    32
```

...	6	門外一里御道	5	3	True
2	6	五層浮圖一所	4	3	True
3	6	門外三里御道	4	3	True
4	6	宣武皇帝所立	3	3	True
5	6	尚書令樂平王	3	3	True
6	6	爾朱兆入洛陽	3	3	True
7	6	西陽門內御道	3	3	True
8	5	層浮圖一所	4	3	True
9	5	去地二十丈	3	3	True
10	5	在建春門外	3	3	True
11	5	夷來附者處	3	3	True
12	5	捨宅所立也	3	3	True
13	5	斤馬五十匹	3	3	True
14	5	晉太康元年	3	3	True
15	5	為黃門侍郎	3	3	True
16	5	而京師遷鄴	3	3	True
17	5	陳留侯李崇	3	3	True
18	5	黃門侍郎徐	3	3	True
19	4	中書舍人	7	5	True
20	4	道榮傳云	7	5	True

指令 程式碼 文字 全部執行

RAM 磁碟

檔案

上傳 下載 刪除 複製

- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

磁碟 64.96 GB 可用

[4] 7秒

```
29 ngram_path = OUTPUT_DIR / '02_N-gram分析報表.xlsx'
30 save_excel(ngram_path, {"入選N-gram":ngram_selected_df,"全部候選":ngram_all_df,"設定":pd.DataFrame([{"最大N":max_n,"N大於4門檻":3,"N等於3門檻":4,"N等於2門檻":5}]})
31 display(ngram_selected_df.head(50))
32
```

18	5	黃門侍郎	3	5	True
19	4	中書舍人	7	5	True
20	4	道榮傳云	7	5	True
21	4	儀同三司	6	5	True
22	4	因以為名	6	5	True
23	4	爾朱世隆	6	5	True
24	4	建義元年	5	5	True
25	4	里內之人	5	5	True
26	3	州刺史	28	4	True
27	3	所立也	24	4	True
28	3	太原王	21	4	True
29	3	里內有	11	4	True
30	3	大將軍	10	4	True
31	3	中朝時	9	4	True
32	3	爾朱榮	9	4	True
33	3	魏晉曰	9	4	True
34	3	廣陵王	8	4	True
35	3	彭城王	8	4	True
36	3	雲謨最	8	4	True
37	3	像一軀	7	4	True

檔案

- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

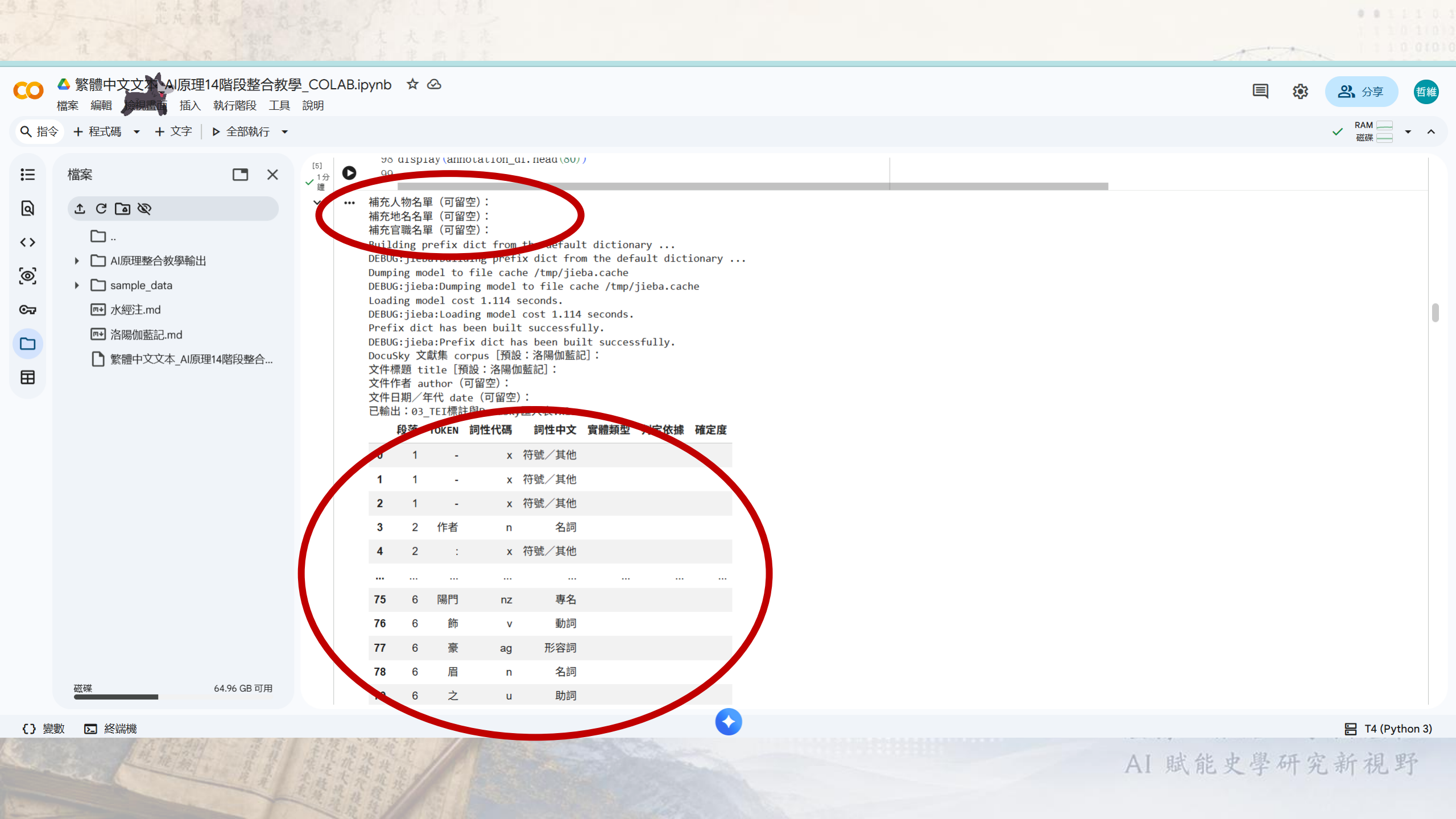
3. 自動詞性與實體判定，輸出 TEI/XML 與 DocuSky 匯入表

自動標註類別包括：

- 詞性：以 jieba POS 為初步判定。
- 人物：`nr` 標記或使用者補充名單。
- 一般地名：`ns` 標記或常見地名後綴。
- 官職：官職詞表與常見官職後綴。
- 年代 / 日期：西元年與「年號 + 中文數字 + 年」等形式。

古典漢文無法只靠現代漢語模型可靠判定，因此輸出 `判定依據` 與 `確定度`，請務必人工複核。

```
[5] 1分 鐘
1 # 可直接在此補充，使用頓號或逗號分隔；留空也能執行。
2 PERSON_LEXICON = set(filter(None, re.split(r"[、, \s]+", input("補充人物名單（可留空）：").strip()))
3 PLACE_LEXICON = set(filter(None, re.split(r"[、, \s]+", input("補充地名名單（可留空）：").strip()))
4 ROLE_LEXICON = set(filter(None, re.split(r"[、, \s]+", input("補充官職名單（可留空）：").strip()))
5
6 for gram in ngram_selected_df.get("N-gram", pd.Series(dtype=str)).astype(str):
7     jieba.add_word(gram, freq=max(10, int(ngram_selected_df.loc[ngram_selected_df["N-gram"]==gram, "次數"].max())), tag="nz")
8
9 POS_ZH = {"n": "名詞", "nr": "人名", "ns": "地名", "nt": "機構 / 團體", "nz": "專名", "v": "動詞", "a": "形容詞", "d": "副詞", "m": "數詞", "q": "量詞", "r": "代詞", "p": "介詞", "c": "連詞",
10 ROLE_SUFFIX = ("太守", "刺史", "尚書", "侍中", "御史", "將軍", "司馬", "令", "尹", "守", "丞", "使", "官", "侯", "王", "帝", "后")
11 PLACE_SUFFIX = ("州", "府", "郡", "縣", "城", "京", "都", "里", "村", "山", "水", "河", "江", "湖", "關", "寺", "宮", "臺")
12 DATE_RE = re.compile(r"(?:西元|公元)?\d{3,4}年|一二三四五六七八九十百千元初+年(?:[一二三四五六七八九十]+月)?(?:永和|建武|太和|延昌|熙平|神龜|正光|孝昌|武泰|永安|張)"
13
14 def classify_token(word, flag):
15     if DATE_RE.fullmatch(word): return "date", "年代 / 日期規則", "high"
16     if word in PERSON_LEXICON: return "persName", "使用者人物詞表", "high"
17     if word in PLACE_LEXICON: return "placeName", "使用者地名詞表", "high"
18     if word in ROLE_LEXICON: return "roleName", "使用者官職詞表", "high"
19     if flag == "nr" and 2 <= len(word) <= 4: return "persName", "jieba nr", "medium"
20     if flag == "ns" or (2 <= len(word) <= 8 and word.endswith(PLACE_SUFFIX)): return "placeName", "地名標記 / 後綴", "low" if flag != "ns" else
```





檔案

- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

4. BPE 分析並匯出結果

BPE 從單字元開始，反覆合併常一起出現的相鄰單位。這裡使用 SentencePiece 的 BPE 模式；它直接處理 Unicode 字串，適合沒有明確空格邊界的繁體中文。

[6]
✓ 2 秒

```
1 required_characters = len(set("".join(paragraphs))) + 16
2 recommended_bpe_vocab = max(800, min(16000, required_characters + 300))
3 bpe_vocab_requested = ask_int("BPE 詞彙表目標大小", recommended_bpe_vocab, 50, 32000)
4 bpe_vocab_training = max(bpe_vocab_requested, required_characters)
5 if bpe_vocab_training != bpe_vocab_requested:
6     print(f"目標詞彙量小於基本字元需求，已自動調整為 {bpe_vocab_training}")
7 corpus_path = OUTPUT_DIR / "04_bpe_corpus.txt"
8 corpus_path.write_text("\n".join(paragraphs), encoding="utf-8")
9 model_prefix = str(OUTPUT_DIR / "04_繁體中文BPE")
10 spm.SentencePieceTrainer.train(
11     input=str(corpus_path), model_prefix=model_prefix, model_type="bpe",
12     vocab_size=bpe_vocab_training, character_coverage=1.0, hard_vocab_limit=False,
13     bos_id=-1, eos_id=-1, pad_id=-1, unk_id=0,
14     input_sentence_size=0, shuffle_input_sentence=False,
15 )
16 sp = spm.SentencePieceProcessor(model_file=model_prefix + ".model")
17 bpe_sequences = [sp.encode(p, out_type=int) for p in paragraphs]
18 bpe_ids = [i for seq in bpe_sequences for i in seq]
19 bpe_counts = Counter(bpe_ids)
20 bpe_vocab_size = sp.vocab_size()
21 base_bpe_vocab_size = bpe_vocab_size
22 extra_token_to_id = {}
23 current_id_to_token = {i:sp.id_to_piece(i) for i in range(base_bpe_vocab_size)}
24
25 def encode_current_vocabulary(text):
26     # 保留 SentencePiece 舊 ID; 已擴充的未知字元改用新增 ID。
27     ids = []
28     for i, c in enumerate(text):
29         if c in current_id_to_token:
30             ids.append(current_id_to_token[c])
31         else:
32             new_id = base_bpe_vocab_size + i
33             current_id_to_token[c] = new_id
34             ids.append(new_id)
```

檔案

- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

[6]
✓ 2秒

63 save_excel(bpe_path, { BPE_TOKEN字詞表 :bpe_vocab_df, 段落編碼 :bpe_encoded_df, 設定 :pd.DataFrame({ 使用者要求詞彙數 :bpe_vocab_requested, 訓練使用詞彙數 :bpe_vocab_t

64 display(bpe_vocab_df.head(60))

65

...

BPE 詞彙表目標大小 [預設 3034]:
已輸出: 04_BPE分析報表.xlsx

TOKEN_ID	TOKEN	文本次數	模型分數	字元長度	特殊TOKEN	
856	856	。	4726	-855.0	1	False
0	0	<unk>	818	0.0	5	True
857	857	之	344	-856.0	1	False
859	859	為	223	-858.0	1	False
871	871	王	181	-870.0	1	False
858	858	有	169	-857.0	1	False
862	862	以	153	-861.0	1	False
867	867	人	147	-866.0	1	False
860	860	寺	134	-859.0	1	False
869	869	於	134	-868.0	1	False
863	863	不	132	-862.0	1	False
872	872	其	124	-871.0	1	False
864	864	曰	121	-863.0	1	False
895	895	國	109	-894.0	1	False
892	892	至	108	-891.0	1	False
865	865	也	104	-864.0	1	False
879	879	者	102	-878.0	1	False
868	868	中	88	-867.0	1	False

檔案

上傳 下載 刪除 複製

AI原理整合教學輸出

sample_data

水經注.md

洛陽伽藍記.md

繁體中文文本_AI原理14階段整合...

5. 比較 N-gram 與 BPE

N-gram 依固定長度統計; BPE 依資料反覆合併, TOKEN 長度可變。兩者的『詞』都只是演算法單位, 不必然等於語言學詞彙。

[7]
0
秒

```
1 ngram_terms = set(ngram_selected_df.get("N-gram",pd.Series(dtype=str)).astype(str))
2 bpe_terms = {sp.id_to_piece(i).replace("_","") for i in range(bpe_vocab_size) if not sp.is_unkown(i)} - {" "}
3 overlap = ngram_terms & bpe_terms
4 plain_chars = len(re.sub(r"\s", "", plain_text))
5 comparison_df = pd.DataFrame([
6     ("N-gram", "固定長度2至最大N", len(ngram_terms), int(ngram_selected_df.get("次數",pd.Series(dtype=int)).sum()), np.mean([len(x) for x in ngram_terms]) if ngr
7     ("BPE", "資料驅動可變長度", len(bpe_terms), len(bpe_ids), np.mean([len(x) for x in bpe_terms]) if bpe_terms else 0),
8 ], columns=["方法", "切分原理", "詞彙種類數", "文本TOKEN / 入選次數", "平均詞彙字元長度"])
9 overlap_df = pd.DataFrame({"兩者完全相同詞形":sorted(overlap, key=lambda x: (-len(x), x))})
10 summary_compare_df = pd.DataFrame({"正文非空白字元":plain_chars, "N-gram與BPE完全重合詞數":len(overlap), "Jaccard":len(overlap)/max(1, len(ngram_terms|bpe_terms)), "BPE
11 compare_path = OUTPUT_DIR / "05_N-gram與BPE比較.xlsx"
12 save_excel(compare_path, {"方法比較":comparison_df, "數值摘要":summary_compare_df, "重合詞形":overlap_df})
13
14 plot_df = comparison_df.set_index("方法")[["詞彙種類數", "平均詞彙字元長度"]]
15 fig, axes=plt.subplots(1, 2, figsize=(12, 4)); plot_df["詞彙種類數"].plot.bar(ax=axes[0], color=["#4C78A8", "#F58518"]); plot_df["平均詞彙字元長度"].plot.bar(ax=axes[1], cc
16 axes[0].set_title("詞彙種類數"); axes[1].set_title("平均 TOKEN 字元長度")
17 for ax in axes: ax.set_xlabel(""); ax.tick_params(axis="x", rotation=0); ax.grid(axis="y", alpha=.25)
18 plt.tight_layout(); plt.savefig(OUTPUT_DIR/"05_N-gram與BPE比較.png", dpi=160, bbox_inches="tight"); plt.show()
19 display(comparison_df); display(summary_compare_df)
20
```

已輸出: 05_N-gram與BPE比較.xlsx

詞彙種類數

平均 TOKEN 字元長度

3000

2.5

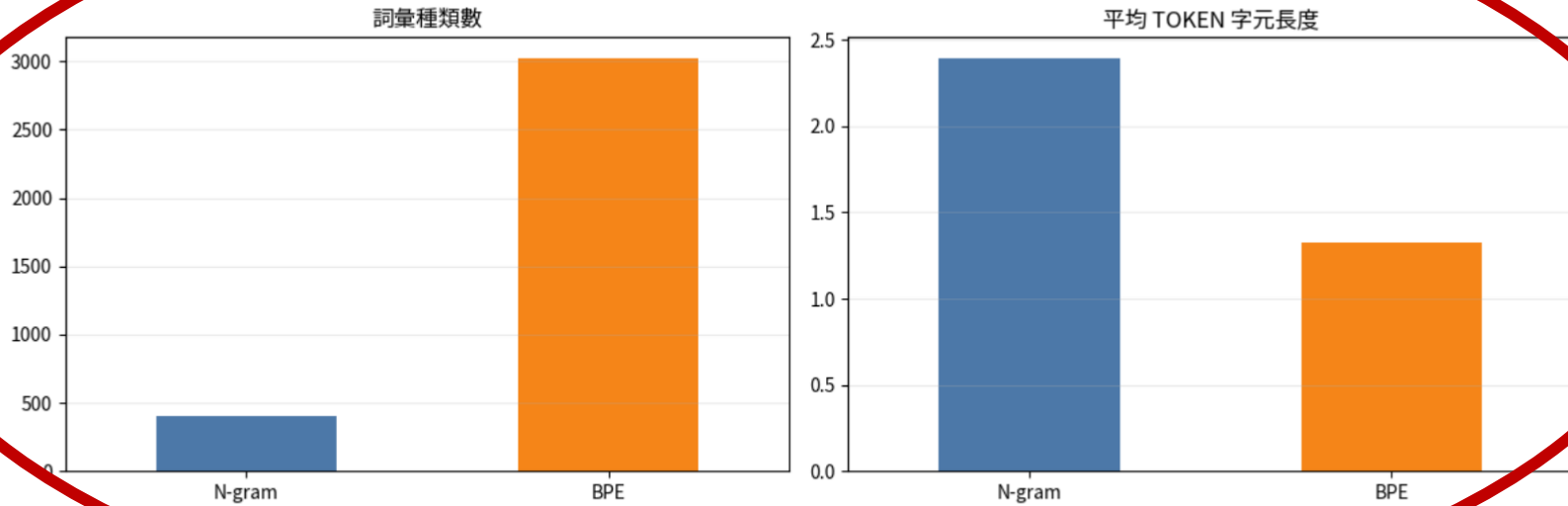
檔案

上傳 下載 刪除 複製

- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

```
18 plt.tight_layout(); plt.savefig('05_N-gram與BPE比較.png', dpi=100, bbox_inches='tight'); plt.show()
19 display(comparison_df)
20 display(summary_compare_df)
```

已輸出: 05_N-gram與BPE比較.xlsx

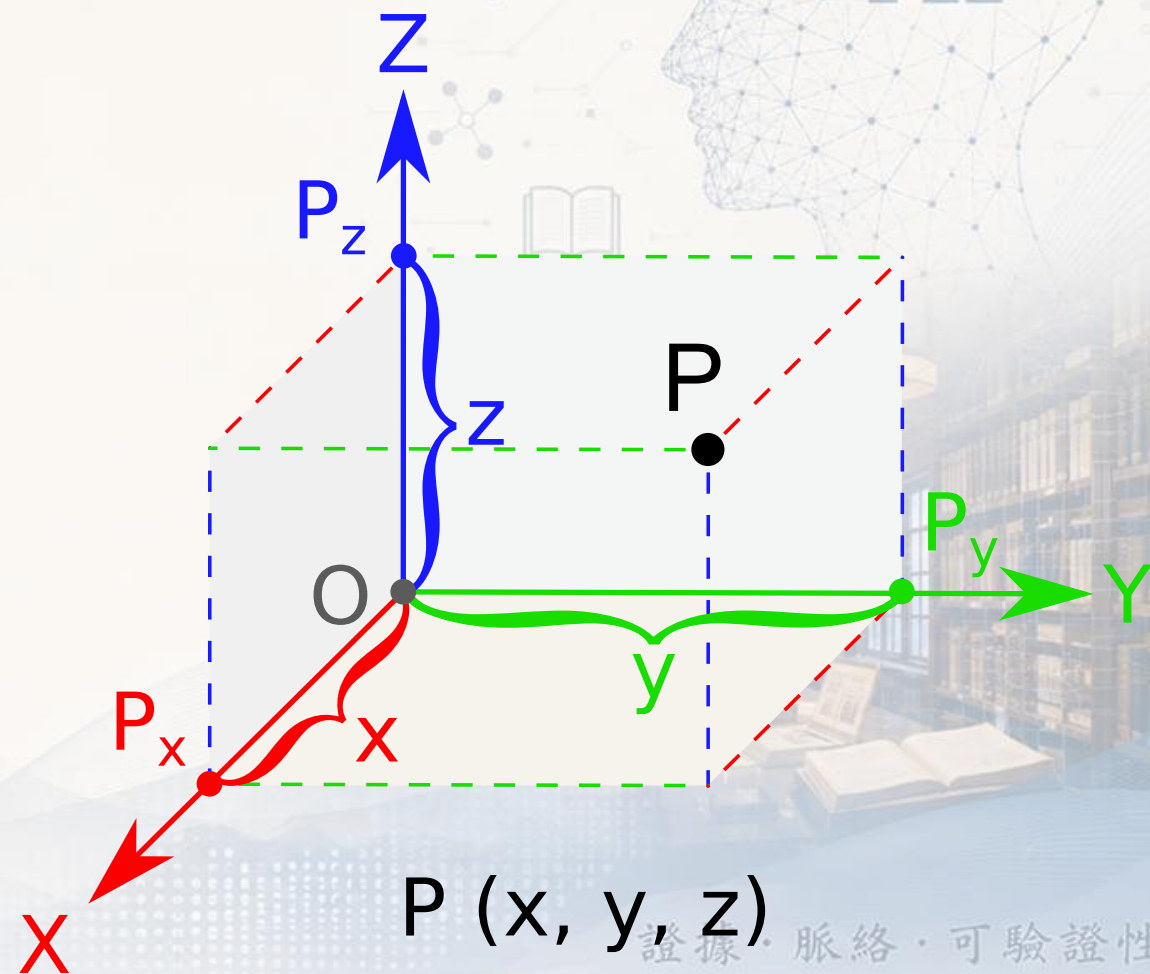


方法	切分原理	詞彙種類數	文本TOKEN／入選次數	平均詞彙字元長度
0	N-gram 固定長度2至最大N	406	3095	2.394089
1	BPE 資料驅動可變長度	3024	29896	1.326058
正文非空白字元 N-gram與BPE完全重合詞數 Jaccard BPE字元／TOKEN				
0	34604	293	0.093401	1.157479

6. 輸出 BPE TOKEN 字詞表

向量概念

- 向量就是有大小有方向的量，可以用數組來表示
- $\{1\}$ -> 一維
- $\{1,1\}$ -> 二維
- $\{1,1,1\}$ -> 三維
- $\{1,1,1,1\}$ -> 四維
- 向量有多少個數字，代表有多少維度。每一維代表一種屬性。
- 在AI中向量維度通常是上千維度



$P(x, y, z)$

證據·脈絡·可驗證性
AI 賦能史學研究新視野

向量化 Embedding

- 將 token 向量化，可以辨識其意涵的相似性
- 向量化是利用不同類別，辨識 token 在不同類別的向量數值
- 類別的數量多寡稱之為向量長度對 AI 模型的精確程度有很大的影響
- GPT2 有 1600 向量維度、deepseek R1 有 7168 向量維度
- 維度越高代表模型表述世界越精細

	語言	形狀	顏色	類別
Apple	1	1	1	1
蘋果	2	1	1	1
香蕉	2	2	2	1
芭樂	2	1.2	3	1
榴槤	2	3	3.4	1

證據 · 驗證性
AI 賦能史學研究新視野

向量化本身是大語言模型的一部分，它是訓練出來的，而不是人工可以干預的，但token是可以人工干預的。

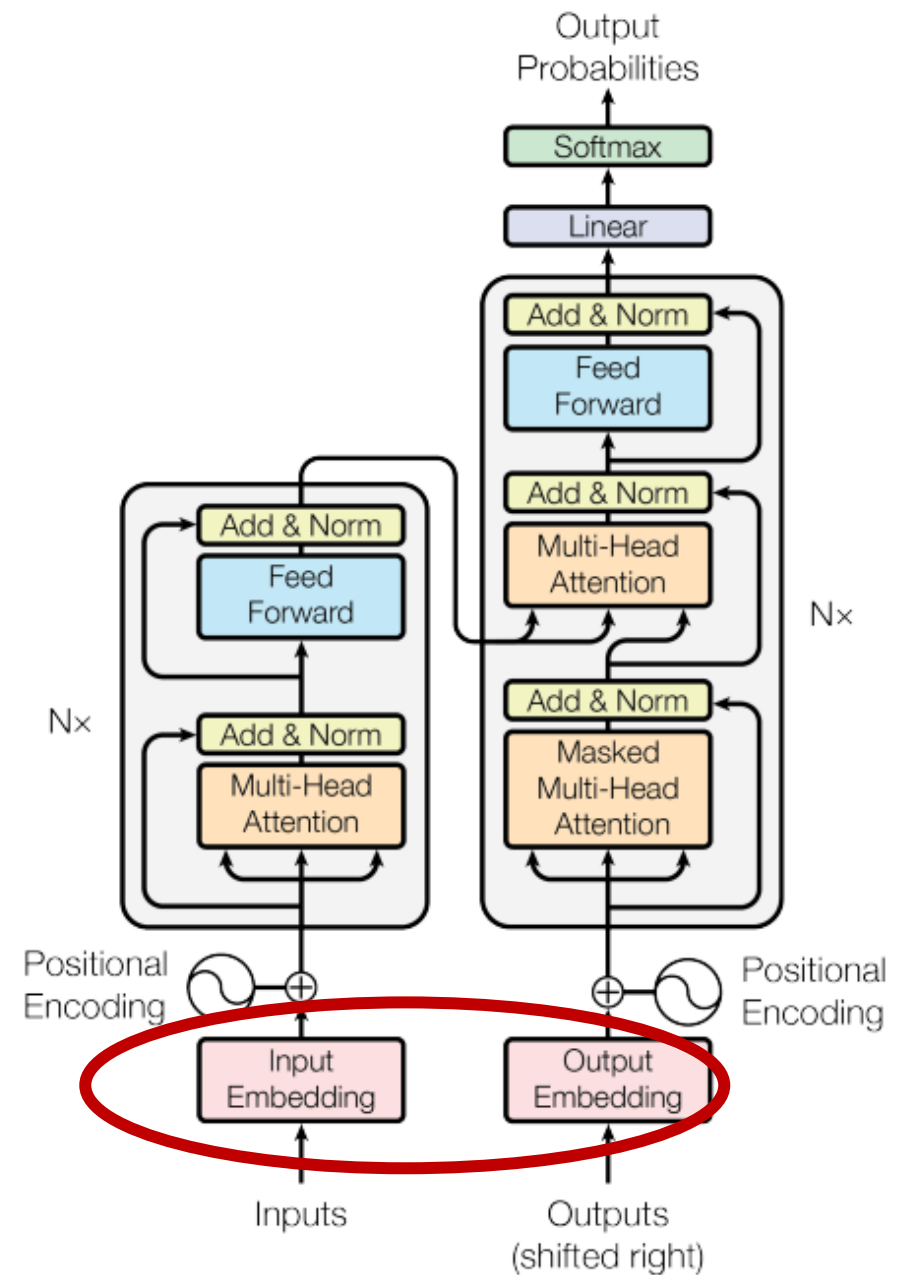


Figure 1: The Transformer - model architecture.

- 因為是訓練出來的，所以實際上向量維度數字的意義人類暫時無法解釋
- 甚至無法確認是一個數字代表一個特徵，還是多個數字代表一個特徵
- 這就是人類還無法解釋AI的根本原因
- 解釋這些向量維度？？？也是一門學問

	語言	形狀	顏色	類別
Apple	1	1	1	1
蘋果	2	1	1	1
香蕉	2	2	2	1
芭樂	2	1.2	3	1
榴槤	2	3	3.4	1



	???	???	???	???
Apple	1	1	1	1
蘋果	2	1	1	1
香蕉	2	2	2	1
芭樂	2	1.2	3	1
榴槤	2	3	3.4	1

從TOKEN到向量化

- 將每個token視為向量，該token在自己的向量參數為1，其餘為0，Vocab Size有多少就等於有多少的向量，這方法叫做：**One-hot編碼**
- 再透過線性層算法進行參數調整。現行最廣泛的算法是
Word2Vec 維度

	Token	Apple	蘋果	香蕉	芭樂	榴槤
Apple	1	1	0	0	0	0
蘋果	2	0	1	0	0	0
香蕉	3	0	0	1	0	0
芭樂	4	0	0	0	1	0
榴槤	5	0	0	0	0	1

向量化

線性層

	語言	形狀	顏色	類別
Apple	1	1	1	1
蘋果	2	1	1	1
香蕉	2	2	2	1
芭樂	2	1.2	3	1
榴槤	2	3	3.4	1

Word2Vec

近朱者赤，近墨者黑

- 看一個字周圍常出現什麼字，就能猜出這個字的意思。
- 如果「珍奶」、「咖啡」、「可樂」的旁邊經常出現「喝」、「去冰」、「杯」，電腦就會知道這三樣東西是同一類的（都是飲料）。

• 1. 核心目標函數 (Objective Function)

$$\mathcal{L}(\theta) = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t)$$

• 2. 字詞預測的機率公式 (Softmax)

$$P(w_O | w_I) = \frac{\exp(v'_{w_O} \top v_{w_I})}{\sum_{w=1}^V \exp(v'_w \top v_{w_I})}$$

• 3. 負採樣 (Negative Sampling)

$$\log \sigma(v'_{w_O} \top v_{w_I}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-v'_{w_i} \top v_{w_I})]$$

檔案

↑ ↺ 📁 🗑

..

▶ AI原理整合教學輸出

▶ sample_data

📄 水經注.md

📄 洛陽伽藍記.md

📄 繁體中文文本_AI原理14階段整合...

6. 輸出 BPE TOKEN 字詞表

[8]
0
秒

```
1 token_table_path = OUTPUT_DIR / "06_BPE_TOKEN字詞表.xlsx"
2 save_excel(token_table_path, {"TOKEN字詞表":bpe_vocab_df.sort_values("TOKEN_ID"), "依頻率排序":bpe_vocab_df})
3 print("BPE 模型與詞彙檔也已保存在輸出資料夾: 04_繁體中文BPE.model /.vocab")
4
```

已輸出: 06_BPE_TOKEN字詞表.xlsx
BPE 模型與詞彙檔也已保存在輸出資料夾: 04_繁體中文BPE.model /.vocab

7. 將 TOKEN 表進行 One-hot 編碼

若詞彙量為 V ，第 i 個 TOKEN 的 One-hot 是長度 V 、只有第 i 格為 1 的向量。完整矩陣是 $V \times V$ ，大型詞表非常浪費記憶體，因此實務上使用稀疏矩陣或直接保存 `TOKEN_ID`。

[9]
0
秒

```
1 onehot_sparse = sparse.eye(bpe_vocab_size, dtype=np.int8, format="csr")
2 preview_size = min(40, bpe_vocab_size)
3 onehot_preview_df = pd.DataFrame(onehot_sparse[:preview_size, :preview_size].toarray(), columns=[sp.id_to_piece(i) for i in range(preview_size)])
4 onehot_preview_df.insert(0, "TOKEN", [sp.id_to_piece(i) for i in range(preview_size)])
5 onehot_map_df = pd.DataFrame({"TOKEN_ID":range(bpe_vocab_size), "TOKEN":[sp.id_to_piece(i) for i in range(bpe_vocab_size)], "非零位置":range(bpe_vocab_size), "值":1})
6 sparse.save_npz(OUTPUT_DIR/"07_One-hot完整稀疏矩陣.npz", onehot_sparse)
7 onehot_path = OUTPUT_DIR/"07_One-hot編碼報表.xlsx"
8 save_excel(onehot_path, {"TOKEN索引對照":onehot_map_df, "前40項預覽":onehot_preview_df, "概念摘要":pd.DataFrame([{"詞彙量V":bpe_vocab_size, "完整矩陣形狀":f"{bpe_vocab_size}"})})
9 display(onehot_preview_df)
10
```

已輸出: 07_One-hot編碼報表.xlsx

TOKEN	<unk>	陽門	高祖	所立	京師	御道	太后	所立也	以為	...	陽王	元年	北有	將軍	年中	捨宅	時人	此寺	魏晉	里內有
0	<unk>	1	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

CO

繁體中文文本_AI原理14階段整合教學_COLAB.ipynb

☆

☁

檔案編輯檢視畫面插入執行階段工具說明

Q 指令 + 程式碼 + 文字 ▶ 全部執行

☰

檔案

⬆ ⬇ ⬅ ➡

..

▶ AI原理整合教學輸出

▶ sample_data

📄 水經注.md

📄 洛陽伽藍記.md

📄 繁體中文文本_AI原理14階段整合...

磁碟 64.96 GB 可用

▶

[9] 0 秒

```
3 onehot_preview_df = pd.DataFrame(onehot_sparse[:preview_size, :preview_size].toarray(), columns=[sp.id_to_piece(i) for i in range(preview_size)])
4 onehot_preview_df.insert(0, "TOKEN", [sp.id_to_piece(i) for i in range(preview_size)])
5 onehot_map_df = pd.DataFrame({"TOKEN_ID": range(bpe_vocab_size), "TOKEN": [sp.id_to_piece(i) for i in range(bpe_vocab_size)], "非零位置": range(bpe_vocab_size), "值": 1})
6 sparse.save_npz(OUTPUT_DIR/"07_One-hot完整稀疏矩陣.npz", onehot_sparse)
7 onehot_path = OUTPUT_DIR/"07_One-hot編碼報表.xlsx"
8 save_excel(onehot_path, {"TOKEN索引對照": onehot_map_df, "前40項預覽": onehot_preview_df, "概念摘要": pd.DataFrame([{"詞彙量V": bpe_vocab_size, "完整矩陣形狀": f"{bpe_vocab_size}"})})
9 display(onehot_preview_df)
10
```

已輸出: 07_One-hot編碼報表.xlsx

	TOKEN	<unk>	陽門	高祖	所立	京師	御道	太后	所立也	以為	...	陽王	元年	北有	將軍	年中	捨宅	時	此寺	魏晉	里內有
0	<unk>	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	陽門	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	高祖	0	0	1	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	所立	0	0	0	1	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	京師	0	0	0	0	1	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
5	御道	0	0	0	0	0	1	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6	太后	0	0	0	0	0	0	1	0	0	...	0	0	0	0	0	0	0	0	0	0
7	所立也	0	0	0	0	0	0	0	1	0	...	0	0	0	0	0	0	0	0	0	0
8	以為	0	0	0	0	0	0	0	0	1	...	0	0	0	0	0	0	0	0	0	0
9	東有	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
10	刺史	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
11	一所	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
12	洛陽	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
13	南有	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
14	尚書	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
15	里內	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

{} 變數

🖥 終端機

T4 (Python 3)

證據·脈絡·可驗證性
AI 賦能史學研究新視野

檔案

..
AI原理整合教學輸出
sample_data
水經注.md
洛陽伽藍記.md
繁體中文文本_AI原理14階段整合...

8. Word2Vec 風格 FFNN: One-hot → Linear → TOKEN 向量

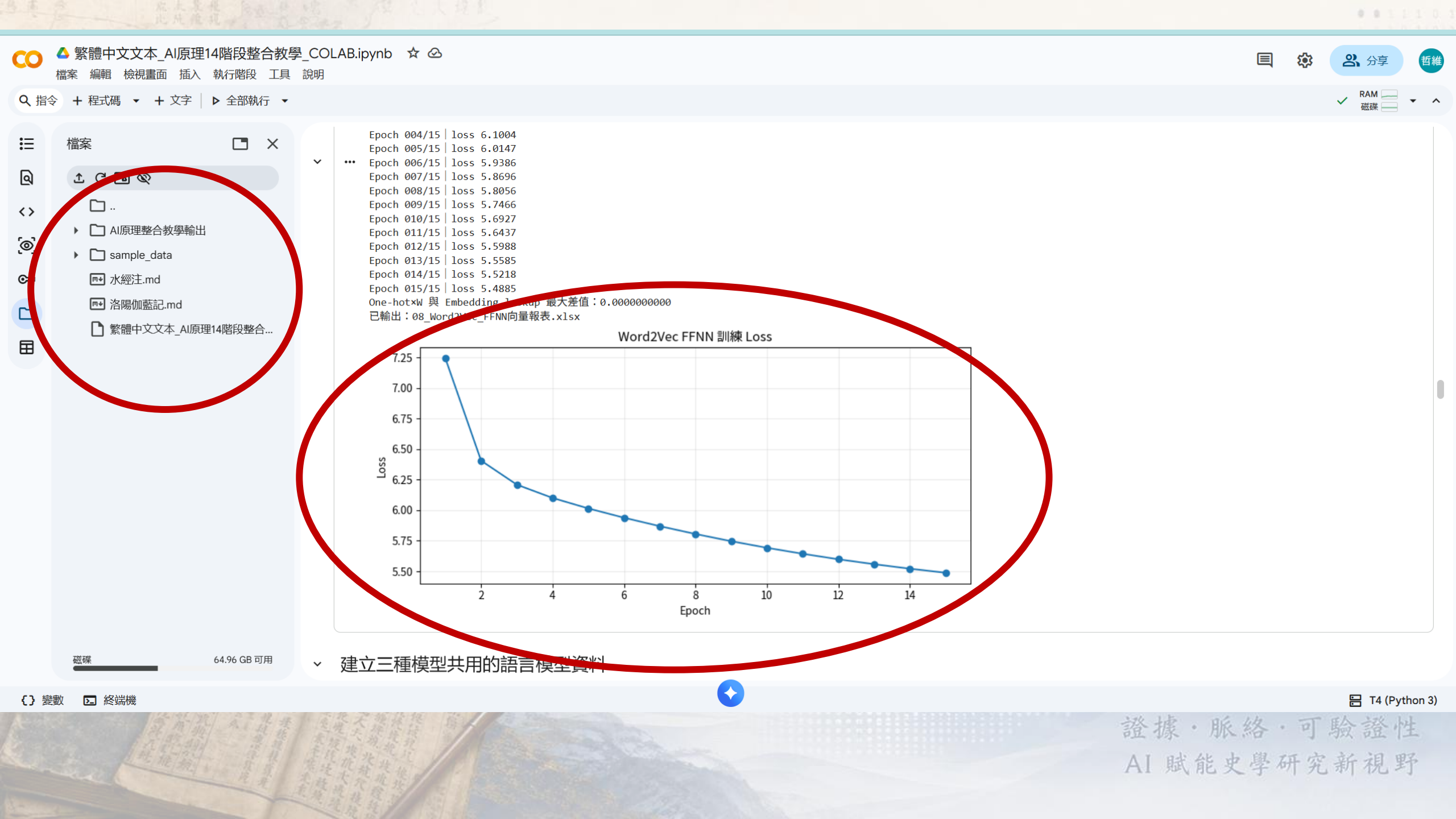
對第 i 個 One-hot 向量 x_i 使用線性層權重 W :

$$h = x_i W$$

因為 One-hot 只有一格為1, 所以結果等同「取出 W 的第 i 列」。 `nn.Embedding` 正是較有效率的查表實作。本教學採 Skip-gram: 用中心 TOKEN 預測附近 TOKEN, 使相似上下文的 TOKEN 向量逐漸接近。

[10]
✓ 44
秒

```
1 embedding_dim = ask_int("Word2Vec TOKEN 向量維度", 64, 8, 512)
2 word2vec_window = ask_int("Skip-gram 上下文窗口", 3, 1, 10)
3 word2vec_epochs = ask_int("Word2Vec 訓練輪數", 15, 1, 200)
4 word2vec_lr = ask_float("Word2Vec 學習率", .003, .00001, .1)
5 max_pairs = ask_int("最多 Skip-gram 配對", 200000, 100, 2000000)
6
7 pairs=[]
8 for seq in bpe_sequences:
9     for i, center in enumerate(seq):
10         for j in range(max(0, i-word2vec_window), min(len(seq), i+word2vec_window+1)):
11             if i!=j: pairs.append((center, seq[j]))
12 if len(pairs)<20: raise ValueError("Skip-gram 配對少於20, 請增加文本。")
13 rng=np.random.default_rng(SEED); rng.shuffle(pairs); pairs=pairs[:max_pairs]
14 pair_array=np.asarray(pairs, dtype=np.int64)
15 pair_loader=DataLoader(TensorDataset(torch.from_numpy(pair_array[:, 0]), torch.from_numpy(pair_array[:, 1])), batch_size=512, shuffle=True)
16
17 class SkipGramFFNN(nn.Module):
18     def __init__(self, vocab, dim):
19         super().__init__(); self.input_linear=nn.Embedding(vocab, dim); self.output_linear=nn.Linear(dim, vocab)
20     def forward(self, token_ids): return self.output_linear(self.input_linear(token_ids))
21
22 word2vec_model=SkipGramFFNN(bpe_vocab_size, embedding_dim).to(DEVICE)
```



檔案

- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

磁碟 64.96 GB 可用

```
Epoch 004/15 | loss 6.1004
Epoch 005/15 | loss 6.0147
Epoch 006/15 | loss 5.9386
Epoch 007/15 | loss 5.8696
Epoch 008/15 | loss 5.8056
Epoch 009/15 | loss 5.7466
Epoch 010/15 | loss 5.6927
Epoch 011/15 | loss 5.6437
Epoch 012/15 | loss 5.5988
Epoch 013/15 | loss 5.5585
Epoch 014/15 | loss 5.5218
Epoch 015/15 | loss 5.4885
One-hotxw 與 Embedding 1-step 最大差值: 0.0000000000
已輸出: 08_Word2Vec_FFNN向量報表.xlsx
```

Word2Vec FFNN 訓練 Loss

Epoch	Loss
1	7.25
2	6.40
3	6.20
4	6.10
5	6.00
6	5.95
7	5.88
8	5.82
9	5.75
10	5.70
11	5.65
12	5.60
13	5.55
14	5.52
15	5.49

建立三種模型共用的語言模型資料

名稱

01_清理後文本.md

01_雜訊清理報表.xlsx

02_N-gram分析報表.xlsx

03_TEI標註與DocuSky匯入表.xlsx

03_自動標註_TEI_P5.xml

04_bpe_corpus.txt

04_BPE分析報表.xlsx

04_繁體中文BPE.model

04_繁體中文BPE.vocab

05_N-gram與BPE比較.png

05_N-gram與BPE比較.xlsx

06_BPE_TOKEN字詞表.xlsx

07_One-hot完整稀疏矩陣.npz

07_One-hot編碼報表.xlsx

08_Word2Vec_FFNN向量報表.xlsx

08_Word2Vec訓練Loss.png

09_RNN訓練與誤差報表.xlsx



編碼(Encoder)

- X = 我是張哲維
- 「我是張哲維」的涵義矩陣 ->
- $N \times$ = 重複多次
- 運算函式稱之為模型參數。
- 訓練模型就是透過自動化的方式，調整這些參數。
- 讓運算結果符合人們的預期。
- 這是提取意義的過程
- (對於數位人文標註詞語有用)

0.1	-0.2	3	-0.15
4	0.7	1.7	-0.6
0.8	-2	1.1	2.3
0.9	5.2	1.2	-0.75

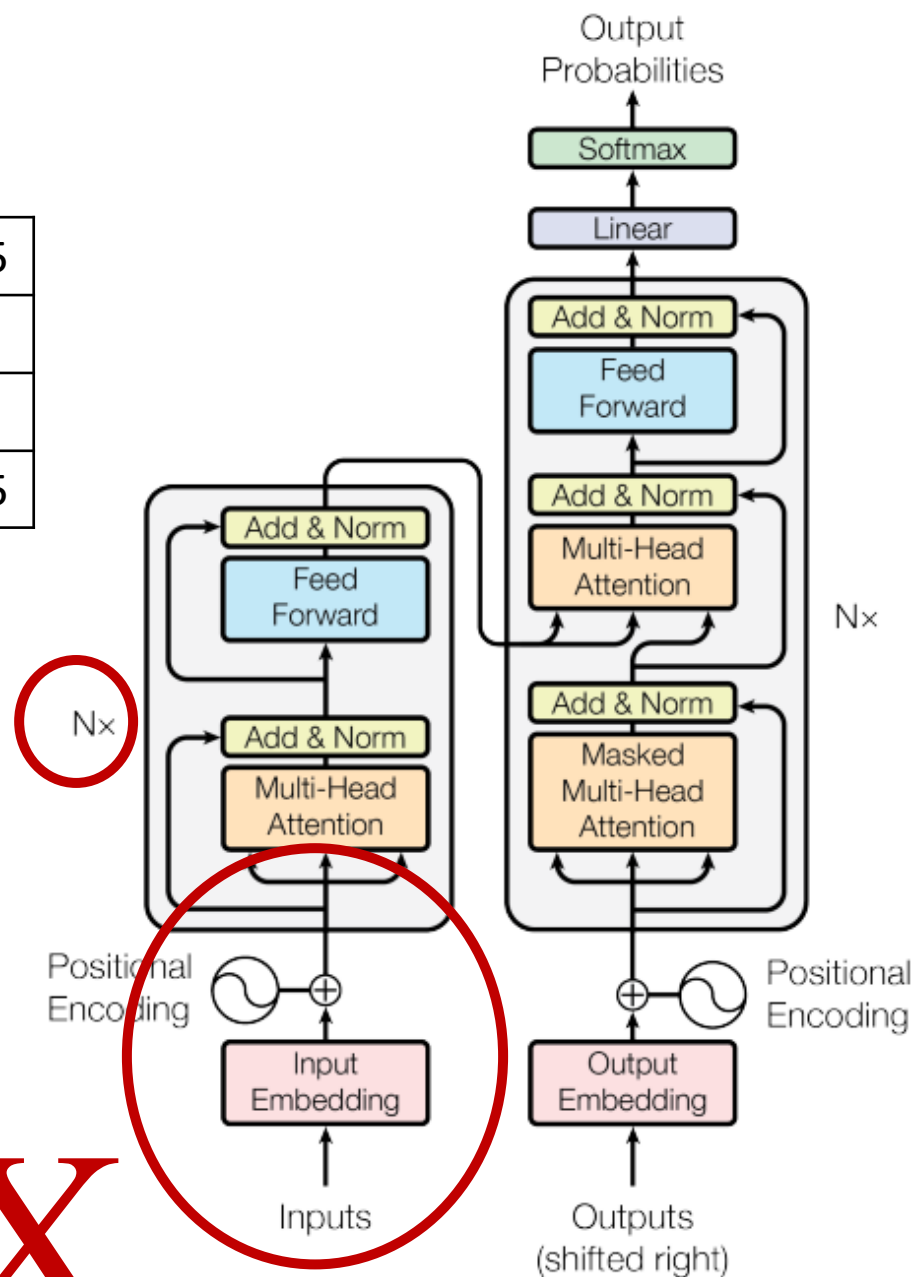


Figure 1: The Transformer - model architecture.

AI 訓練



繁體中文文本_AI原理14階段整合教學_COLAB.ipynb ☆

檔案 編輯 檢視畫面 插入 執行階段 工具 說明



分享



哲維

指令 + 程式碼 + 文字 | 全部執行

✓ RAM 磁碟



檔案



建立三種模型共用的語言模型資料



上傳 下載 刪除 複製

- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

[11]

✓ 2秒



```
context_length=ask_int("共同最大上下文長度", 24, 2, 128)
batch_size=ask_int("共同批次大小", 64, 4, 1024)
3 max_lm_samples=ask_int("最多語言模型樣本", 200000, 100, 2000000)
4 pad_id=bpe_vocab_size
5 model_vocab_size=bpe_vocab_size+1
6 pad_vector=np.zeros((1, embedding_dim), dtype=np.float32)
7 initial_embeddings=np.vstack([word2vec_vectors, pad_vector]).astype(np.float32)
8
9 def sequences_to_dataset(sequences, max_samples=max_lm_samples, split=True):
10     contexts, lengths, targets=[], [], []
11     for seq in sequences:
12         for i in range(1, len(seq)):
13             context=seq[max(0, i-context_length):i]
14             contexts.append(context+[pad_id]*(context_length-len(context))); lengths.append(len(context)); targets.append(seq[i])
15     if len(targets)<20: raise ValueError("語言模型樣本少於20, 請增加文本。")
16     idx=np.arange(len(targets)); rng=np.random.default_rng(SEED); rng.shuffle(idx); idx=idx[:max_samples]
17     X=np.asarray(contexts, np.int64)[idx]; L=np.asarray(lengths, np.int64)[idx]; y=np.asarray(targets, np.int64)[idx]
18     if not split: return TensorDataset(torch.from_numpy(X), torch.from_numpy(L), torch.from_numpy(y))
19     cut=min(len(idx)-1, max(1, int(len(idx)*.9)))
20     return (TensorDataset(torch.from_numpy(X[:cut]), torch.from_numpy(L[:cut]), torch.from_numpy(y[:cut])),
21                     TensorDataset(torch.from_numpy(X[cut:]), torch.from_numpy(L[cut:]), torch.from_numpy(y[cut:])))
22
23 train_ds, valid_ds=sequences_to_dataset(bpe_sequences)
24 train_loader=DataLoader(train_ds, batch_size=batch_size, shuffle=True); valid_loader=DataLoader(valid_ds, batch_size=batch_size)
25 print(f"訓練樣本: {len(train_ds):,} | 驗證樣本: {len(valid_ds):,}")
26
27 def count_parameters(model): return sum(p.numel() for p in model.parameters())
```

Loading...

磁碟 64.96 GB 可用



變數



終端機

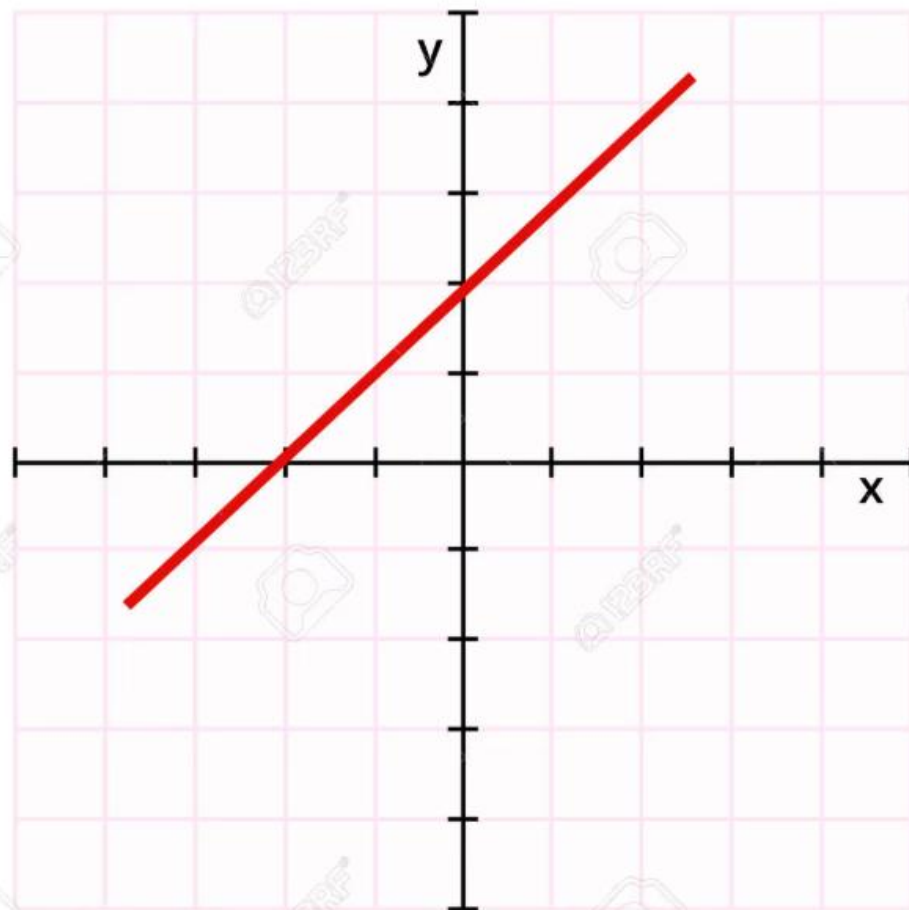


T4 (Python 3)

線性層 linear

- 透過這兩者 就能做出 數字識別、圖像分類
- 本質上都是在做加法和乘法的基礎運算
- $y=ax+b$... 這稱之為線性函式
- 當事物可以用數字表示，那麼結論也可以用線性函式來表示

Linear Function



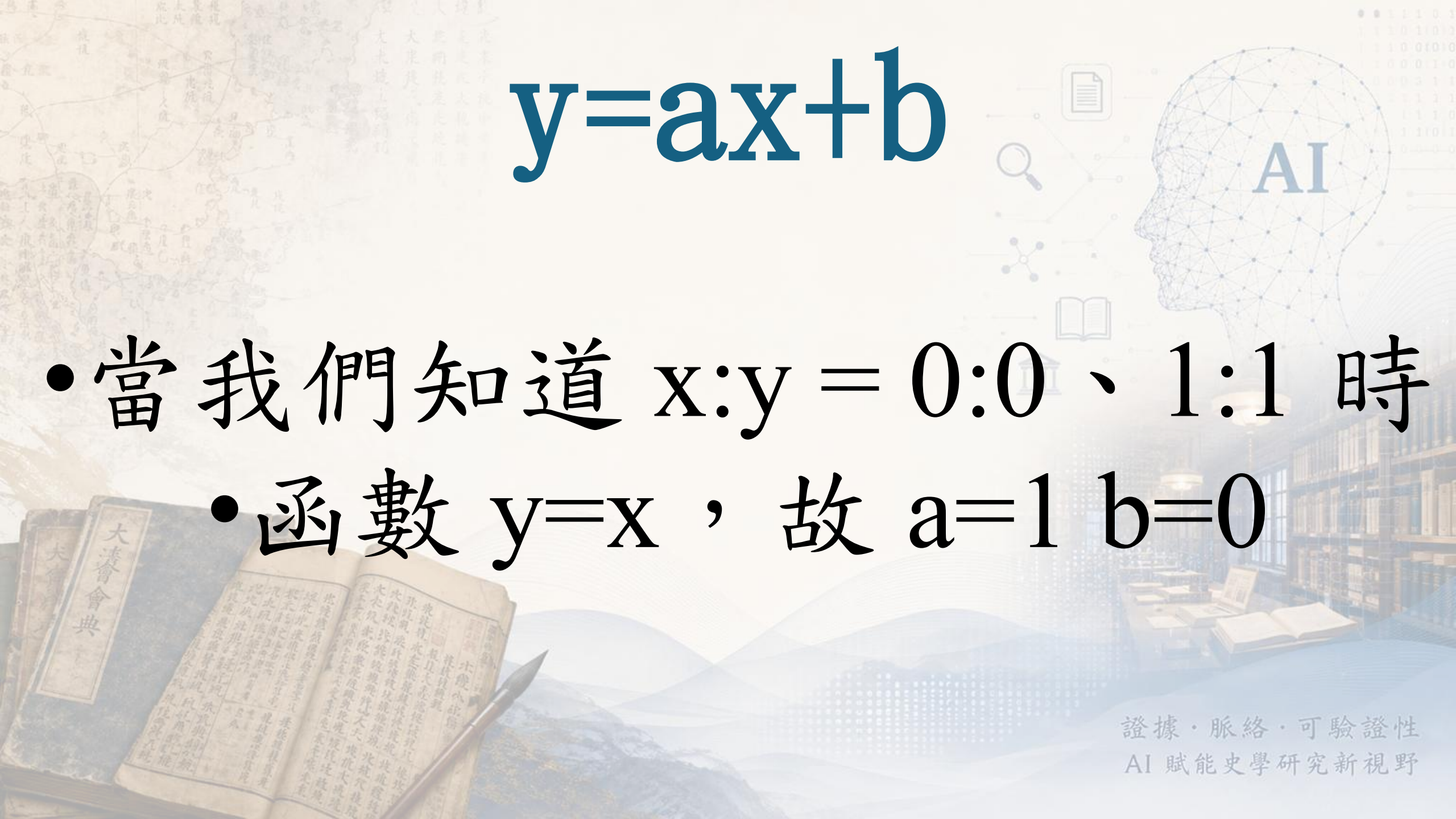
變量	包子	饅頭	肉粽	固定消耗
數量	1	2	3	
熱量	a1	a2	a3	b1
時間	a4	a5	a6	b2
總熱量	$1*a1+2*a2+3*a3+b1$			
總耗時	$1*a4+2*a5+3*a6+b2$			

參數

- a_1 、 a_2 、 a_3 、 a_4 、 a_5 、 a_6 → 權重 Weight
- B_1 、 b_2 偏置 Bias
- 輸入 = 變量 (三維向量)
- 輸出 = 結果 (二維向量)

線性函式

證據·脈絡·可驗證性
AI 賦能史學研究新視野


$$y=ax+b$$

- 當我們知道 $x:y = 0:0$ 、 $1:1$ 時
- 函數 $y=x$ ，故 $a=1$ $b=0$

模型中參數都是算出來的

$$1*a1+2*a2+3*a3+b1 = y1(\text{總熱量})$$

$$1*a4+2*a5+3*a6+b2 = y2(\text{總耗時})$$

如果已經有了實際的熱量和耗時

那麼就可以回推參數

回推參數的過程

就是訓練模型的過程

模型參數大小

**Hugging Face**

Models Datasets Spaces Buckets NEW Docs Enterprise Pricing Log In Sign Up

Main Tasks Libraries Languages Licenses Other

Tasks

Text Generation

Any-to-Any

Image-Text-to-Text

Image-to-Text

Image-to-Image

Text-to-Image

Text-to-Video

Text-to-Speech

+ 44

Parameters

<1B

6B

12B

32B

128B

> 500B

Libraries

PyTorch

TensorFlow

JAX

Transformers

Diffusers

GGUF

MLX

MLX

Transformers.js

Safetensors

sentence-transformers

ONNX

+ 42

Apps

vLLM

llama.cpp

MLX LM

LM Studio

Ollama

Jan

Draw Things

+ 8

Inference Providers

Groq

Novita

Cerebras

SambaNova

Models 2,879,868

Full-text search

Inference Available

Sort: Trending

openbmb/MiniCPM-V-4.6

Image-Text-to-Text · 1B · Updated about 23 hours ago · 28.6k · 619

HiDream-ai/HiDream-01-Image

Image-Text-to-Image · 9B · Updated about 24 hours ago · 13.6k · 350

Zyphra/ZAYA1-8B

9B · Updated 4 days ago · 144k · 508

unsloth/Qwen2.5-72B-MTP-GGUF

Image-Text-to-Text · 27B · Updated 1 day ago · 134k · 176

SeeSee21/Z-Anime

Text-to-Image · 6B · Updated 19 days ago · 14.5k · 384

froggeric/Qwen-Fixed-Chat-Templates

Updated about 15 hours ago · 220

TenStrip/LTX2.3-10Eros

Image-to-Video · Updated 5 days ago · 122k · 270

Jiunsong/supergemma4-26b-uncensored-gguf-v2

Text Generation · 25B · Updated Apr 12 · 274k · 600

SulphurAI/Sulphur-2-base

Text-to-Video · 9B · Updated 7 days ago · 875k · 1k

Supertone/supertonic-3

Text-to-Speech · Updated 10 days ago · 16.5k · 260

deepseek-ai/DeepSeek-V4-Pro

Text Generation · 862B · Updated 10 days ago · 2.97M · 3.98k

unsloth/Qwen3.6-35B-A3B-MTP-GGUF

Image-Text-to-Text · 36B · Updated 1 day ago · 124k · 158

circlestone Labs/Anima

Updated 1 day ago · 2502k · 1.3k

ResembleAI/gramabox

Text-to-Speech · Updated 3 days ago · 869 · 97

Qwen/Qwen3.6-35B-A3B

Image-Text-to-Text · 36B · Updated 22 days ago · 5.26M · 1.78k

Qwen/Qwen3.6-27B

Image-Text-to-Text · 28B · Updated 22 days ago · 3.26M · 1.3k

八千億參數

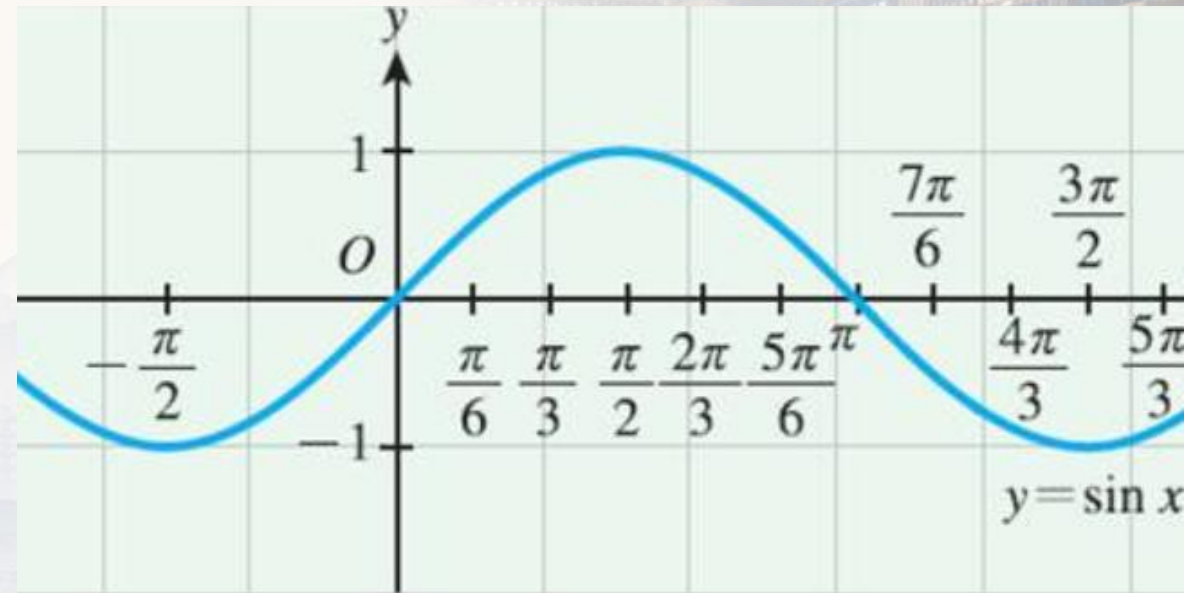
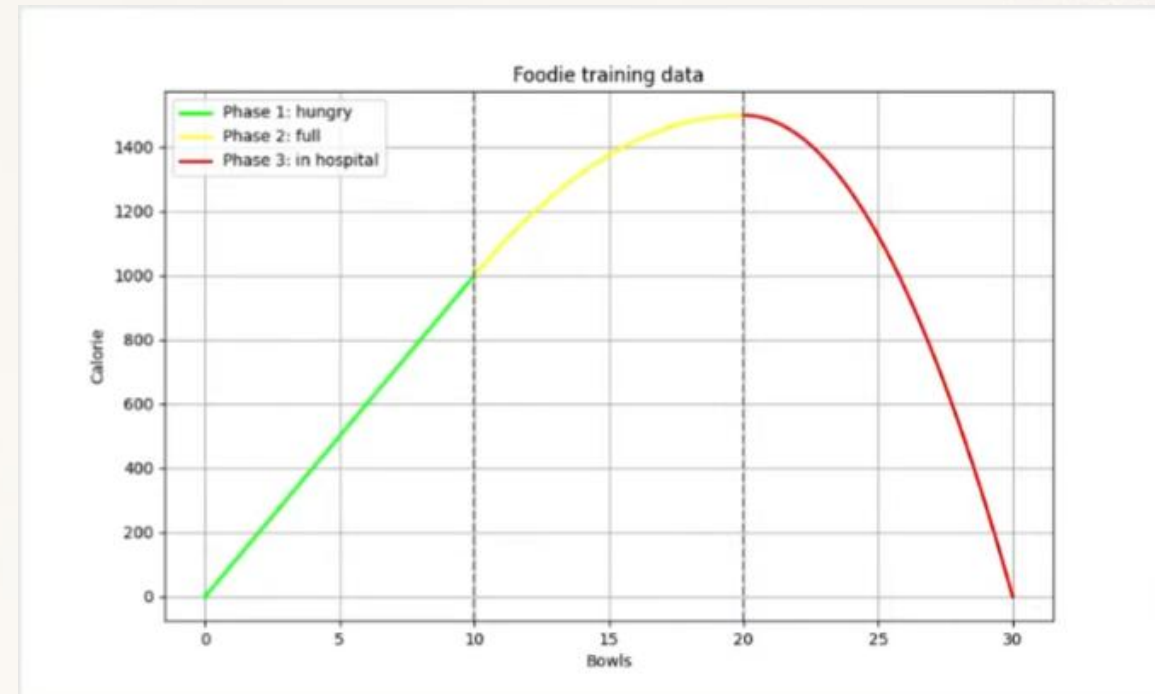
讓AI訓練成我們想要的樣子



- $aX + b = Y$
- 訓練目的在於找出 a 和 b 的值

如何呈現複雜的真實情況

- 以代數來看，任何線性都能用公式表示。
- 但在工程上，過於複雜的公式計算量過於龐大
- 工程模塊化 = 函式越簡單越好
- 工程講究的是，用簡單的功能搭建出複雜的功能
- 過於複雜的公式，在工程上沒有擴展性
- -> 線性函式 + 非線性函式 + 線性函式

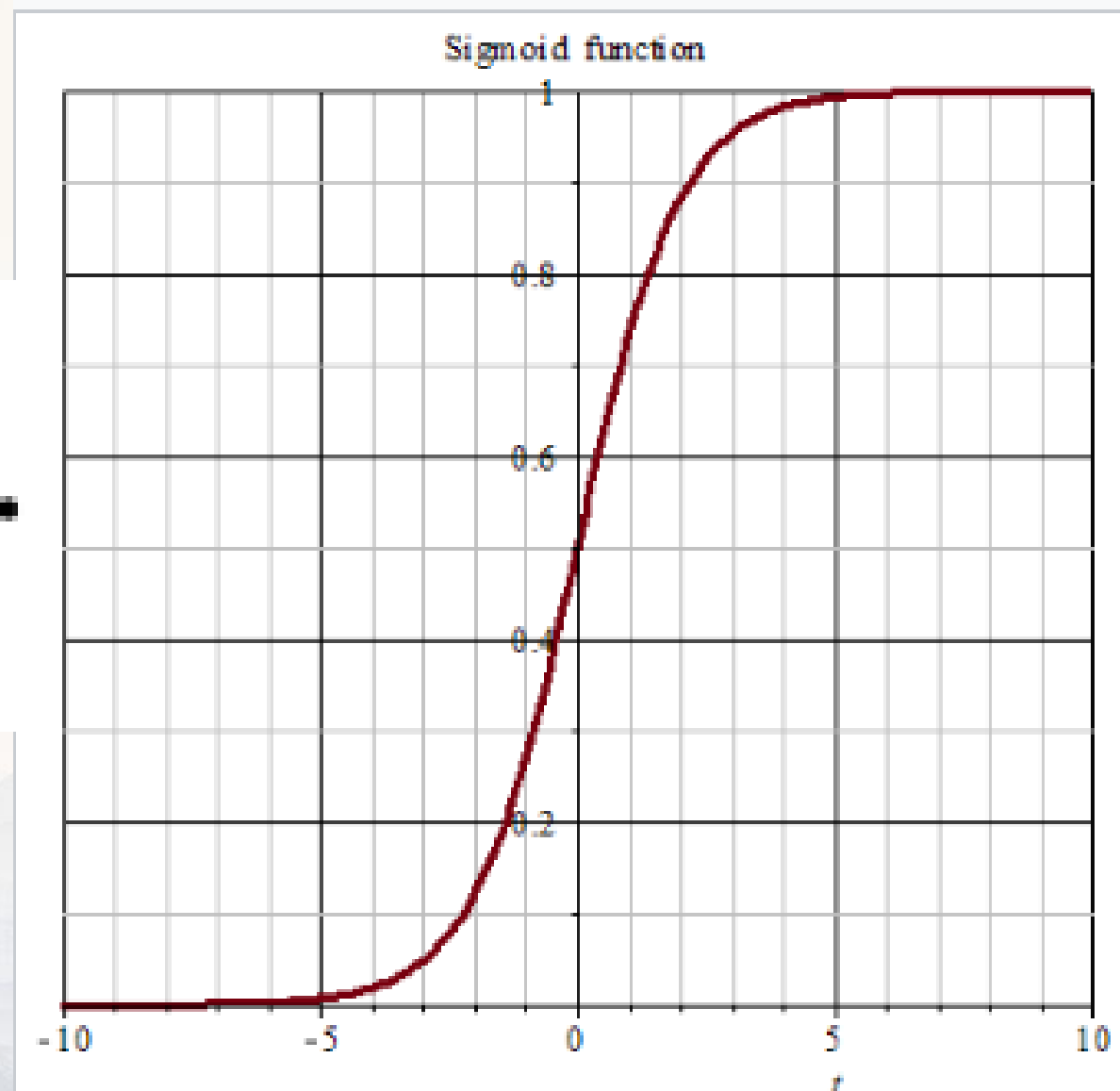


激活函式

- 線性函式 + 非線性函式 + 線性函式
- 非線性函式 = 激活函式 (Activation)
- 最常見的激活函式是 ReLU、Sigmoid、Tanh

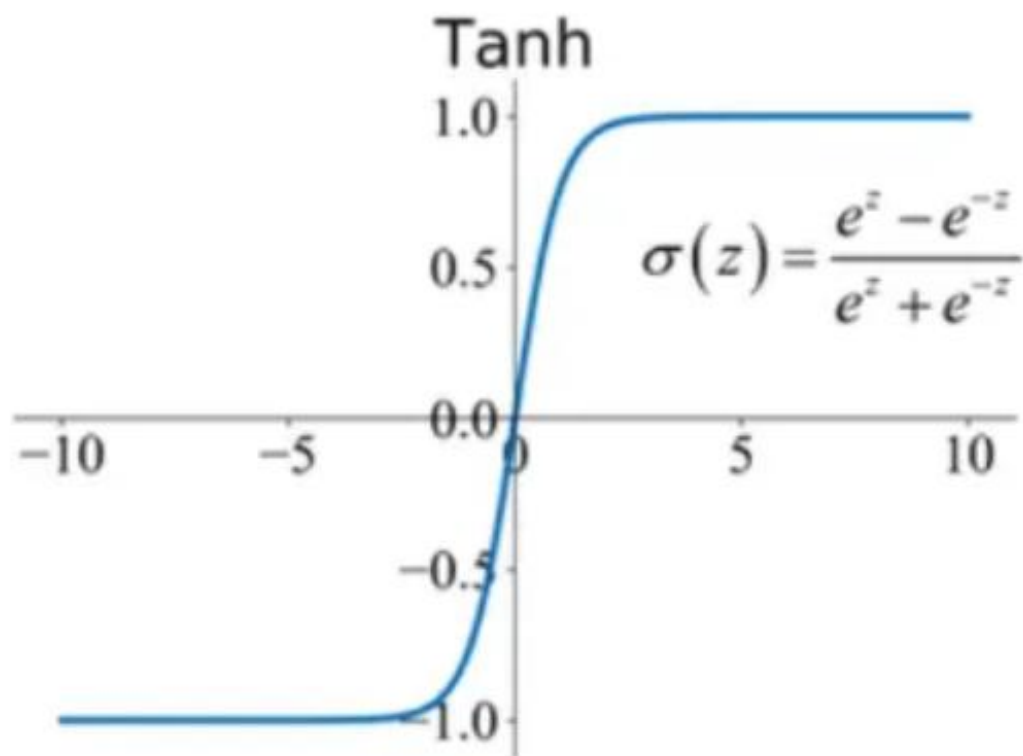
Sigmoid = S型函数

$$S(t) = \frac{1}{1 + e^{-t}}.$$



Tanh = 雙曲函數

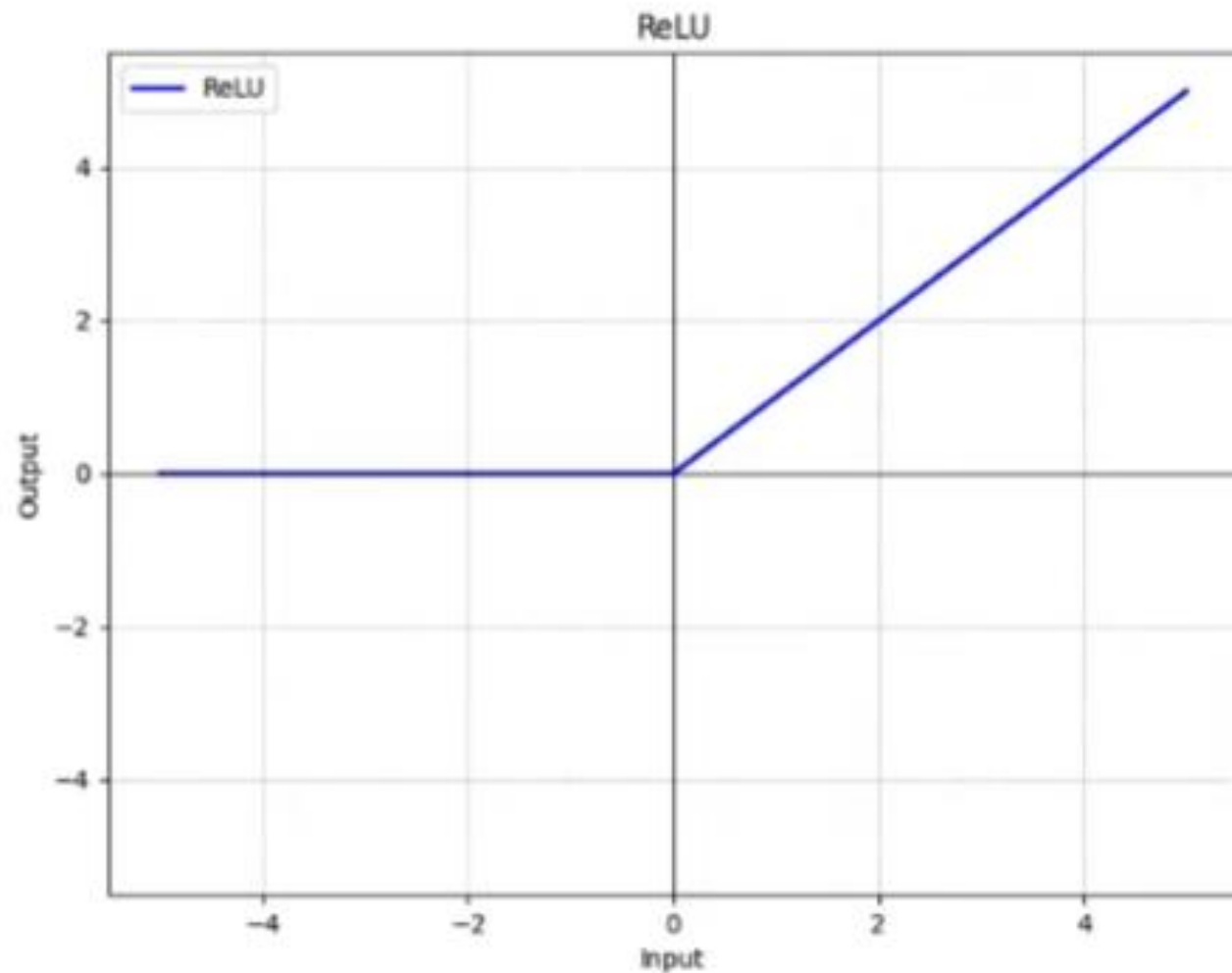
$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



證據·脈絡·可驗證性
AI 賦能史學研究新視野

修正線性單元 ReLU

- Rectified Linear Unit，修正線性單元
- 當輸入是負數時，輸出是0
- 正數時，則輸出輸入本身
- 工程實踐證明，最簡單最容易實現的ReLU，效果與另兩個複雜的函式差不多。
- 如同歷史，往往最簡單易懂的理論，才是最接近歷史事實
- 甚至解決了梯度下降的問題

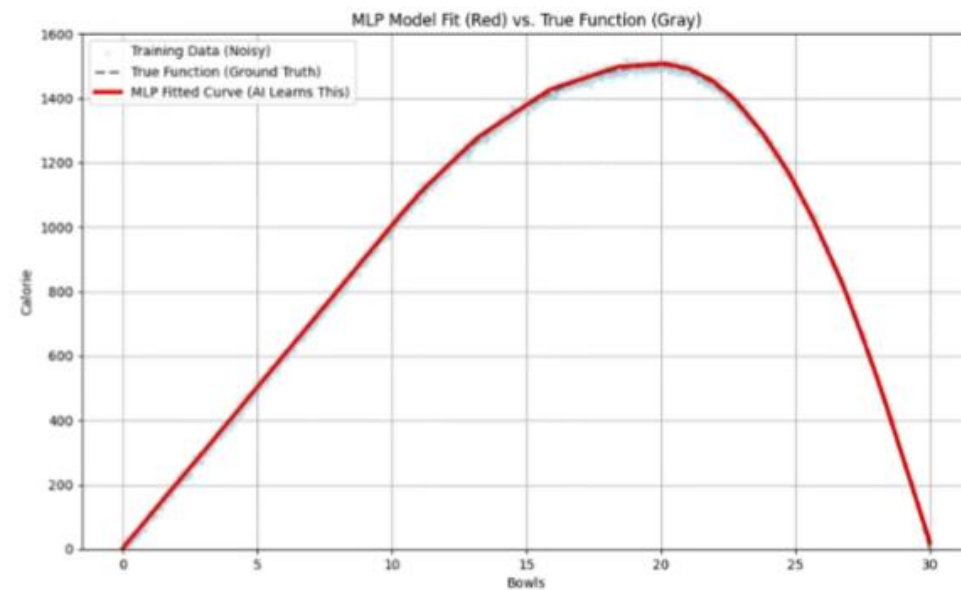
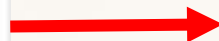
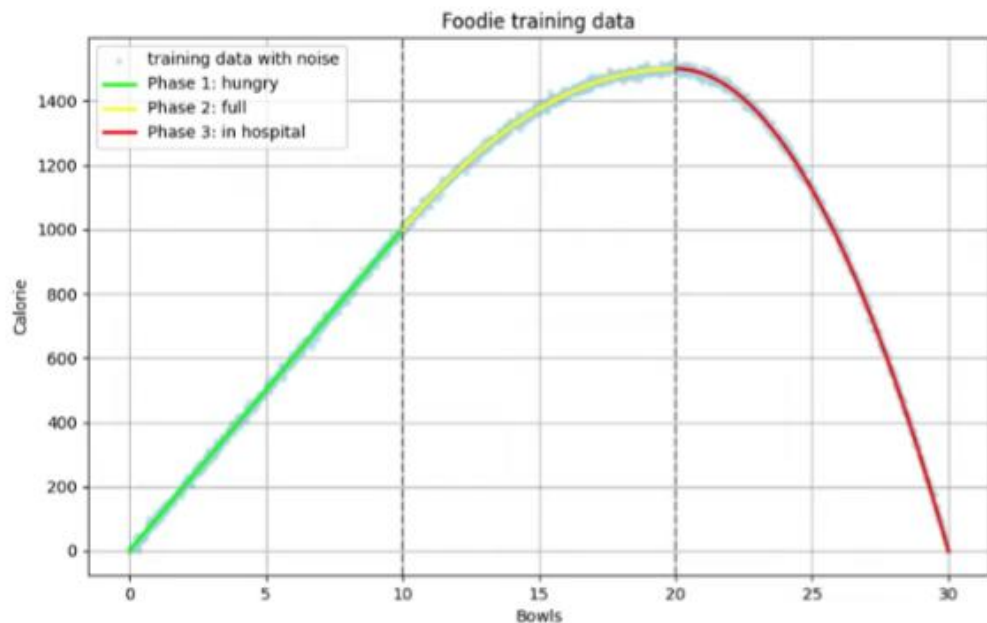


前饋神經網絡

- Feedforward Neural Network
- 兩種函式不斷重複，最終以線性層收尾
- 又稱 多層感知機 MultiLayer Perceptron



透過大量的數據，找出符合這條件的公式



線性層

→ 激活函式

→ 線性層

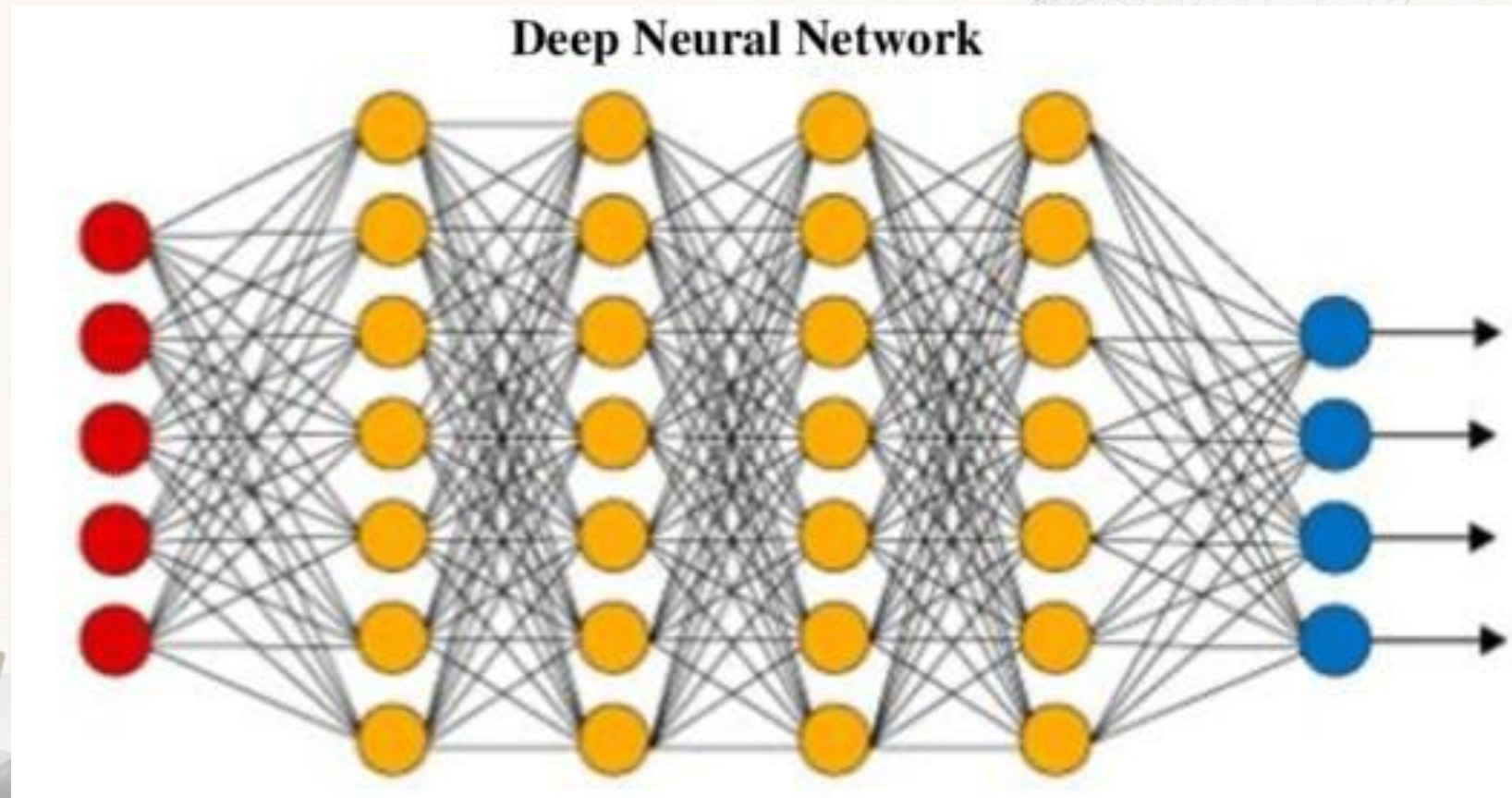
→ 激活函式

→ 線性層

可驗證性
研究新視野

深度神經網路

- a. 前饋神經網路
- b. 卷積神經網路 -> 用於圖片、文字辨識
- c. 遞歸神經網路 -> 用於具有時間次序的資料，如文本。



輸入層

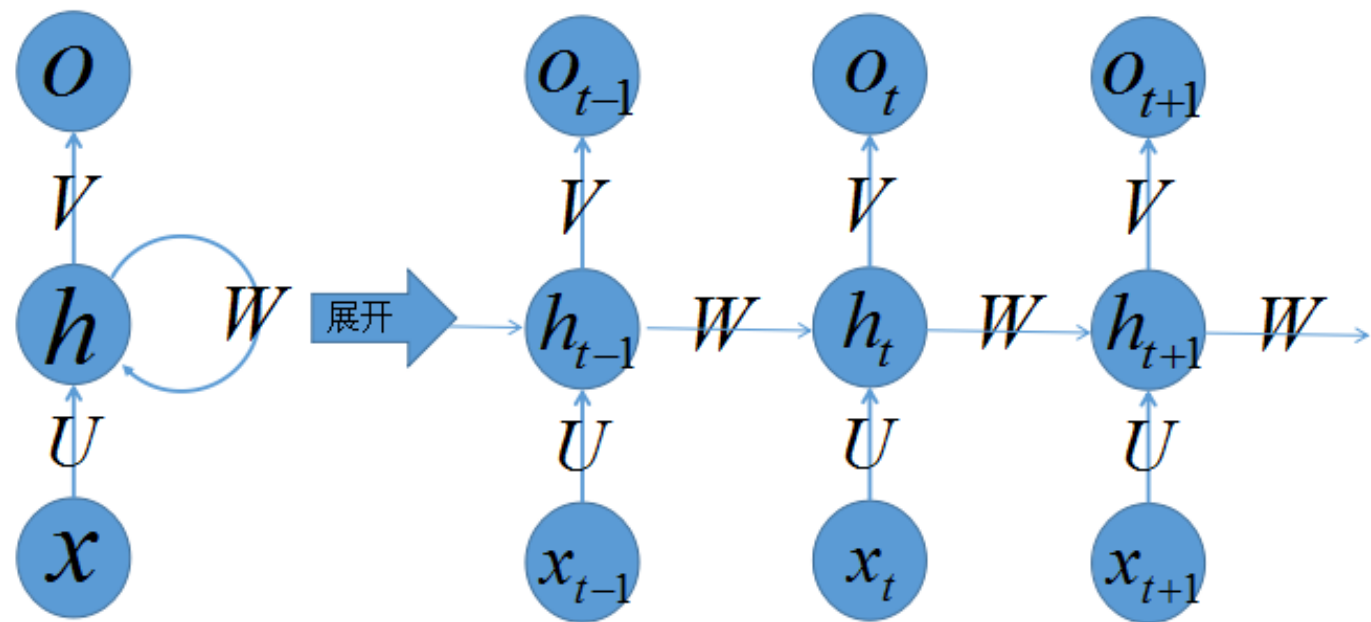
隱藏層

證據
AI 賦能

脈絡：可驗證性
輸出層

遞歸神經網絡

- RNN = Recurrent Neural Network
 - 用於上下文
是一種專門用來處理序列資料（例如文字、語音或時間序列）的深度學習模型
 - 有時間次序的資料
 - 如何在神經網絡結構中的數學函數 創造出時間序列的概念
因而加上遞歸的概念
 - 遞歸 意味著 將先前輸入的處理結果，再度輸出到同一個神經元UNIT中。
 - 例如：手機打字時，自動選出現文字讓使用者選取。
 - 此一將結果暫存，重新在輸入輸入層之數據，稱之為狀態層。（存到記憶體中）
 - 再將狀態層的數據，重新匯入神經元中，乘上神經元中的遞歸權重，再進行加總，當超過閾值時，則激發此神經元。
 - 學習=訓練
- 如何訓練RNN
- 如同所有函數一樣，當 $Y=aX+b$ 。（ a 、 b = 參數、 x 、 y = 變數）
我們已知的問題 x ，及其答案 y
透過大量的問題 x 和答案 y ，推估出參數 a 、 b 。
- 在訓練RNN時，就需要大量的具有時間次序的問題與答案，例如句子。
爾朱榮總士馬於此寺
OO榮總士馬於此寺
爾朱O總士馬於此寺
爾朱榮總OO於此寺
爾朱榮總士馬於此O
 - 電腦會將訓練結果與實際之間的誤差值，降到最低。



檔案

↑ ↺ 📁 📄

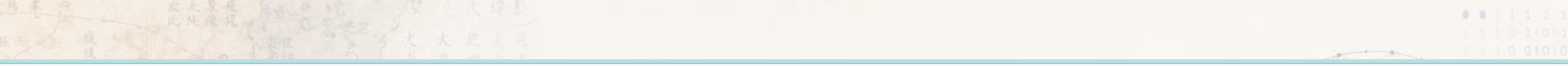
..
AI原理整合教學輸出
sample_data
水經注.md
洛陽伽藍記.md
繁體中文文本_AI原理14階段整合...

9. RNN：調整隱藏向量維度、觀察誤差並優化

RNN 依序讀入 TOKEN，隱藏狀態可寫成：
$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b)$$
隱藏維度愈大，容量與參數量通常愈大，但不保證驗證誤差下降。應同時比較訓練與驗證 Loss：訓練下降、驗證上升通常是過度擬合。

[12]
✓ 42 秒

```
1 rnn_hidden_dim=ask_int("RNN 隱藏向量維度",64,8,512)
2 rnn_layers=ask_int("RNN 層數",1,1,4)
3 rnn_epochs=ask_int("RNN 訓練輪數",15,1,200)
4 rnn_lr=ask_float("RNN 學習率",.001,.00001,.1)
5 rnn_dropout=ask_float("RNN Dropout (單層時自動為0)",.1,0,.8)
6
7 class RNNLanguageModel(nn.Module):
8     def __init__(self):
9         super().__init__(); self.embedding=nn.Embedding(model_vocab_size,embedding_dim,padding_idx=pad_id)
10        with torch.no_grad(): self.embedding.weight.copy_(torch.from_numpy(initial_embeddings))
11        self.rnn=nn.RNN(embedding_dim,rnn_hidden_dim,num_layers=rnn_layers,batch_first=True,nonlinearity="tanh",dropout=rnn_dropout if rnn_layers>1 else 0)
12        self.output=nn.Linear(rnn_hidden_dim,bpe_vocab_size)
13    def forward(self,ids,lengths):
14        x=self.embedding(ids); packed=nn.utils.rnn.pack_padded_sequence(x,lengths.cpu(),batch_first=True,enforce_sorted=False)
15        _,hidden=self.rnn(packed); return self.output(hidden[-1])
16
17 rnn_model=RNNLanguageModel().to(DEVICE); optimizer=torch.optim.AdamW(rnn_model.parameters(),lr=rnn_lr)
18 rnn_rows=[]
19 for epoch in range(1,rnn_epochs+1):
20     rnn_model.train(); total=correct=items=0
21     for ids,lengths,target in train_loader:
22         ids,lengths,target=ids.to(DEVICE),lengths.to(DEVICE),target.to(DEVICE); optimizer.zero_grad()
```



檔案



..

▶ AI原理整合教學輸出

▶ sample_data

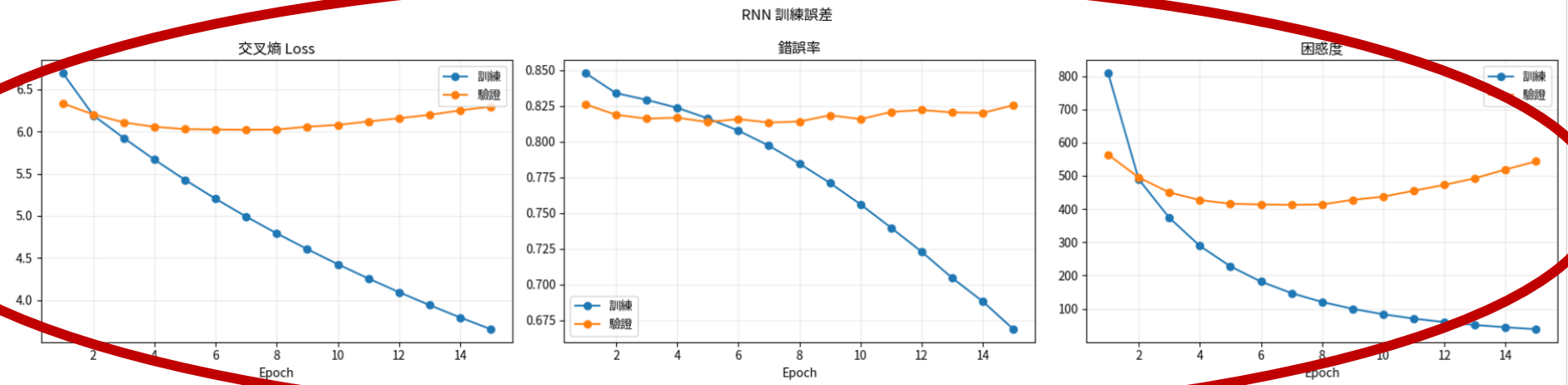
水經注.md

洛陽伽藍記.md

繁體中文文本_AI原理14階段整合...

RNN Dropout (單層時自動為0) [預設 0.1]:

Epoch 001/15	train loss 6.6959, error 84.80%	valid loss 6.3347, error 82.62%
Epoch 002/15	train loss 6.1936, error 83.39%	valid loss 6.2048, error 81.88%
Epoch 003/15	train loss 5.9237, error 82.93%	valid loss 6.1095, error 81.61%
Epoch 004/15	train loss 5.6677, error 82.37%	valid loss 6.0569, error 81.68%
Epoch 005/15	train loss 5.4277, error 81.63%	valid loss 6.0305, error 81.38%
Epoch 006/15	train loss 5.2027, error 80.78%	valid loss 6.0253, error 81.58%
Epoch 007/15	train loss 4.9916, error 79.72%	valid loss 6.0224, error 81.34%
Epoch 008/15	train loss 4.7922, error 78.47%	valid loss 6.0256, error 81.41%
Epoch 009/15	train loss 4.6048, error 77.11%	valid loss 6.0583, error 81.85%
Epoch 010/15	train loss 4.4255, error 75.61%	valid loss 6.0804, error 81.58%
Epoch 011/15	train loss 4.2565, error 73.98%	valid loss 6.1208, error 82.08%
Epoch 012/15	train loss 4.0947, error 72.30%	valid loss 6.1586, error 82.21%
Epoch 013/15	train loss 3.9409, error 70.46%	valid loss 6.1997, error 82.05%
Epoch 014/15	train loss 3.7945, error 68.82%	valid loss 6.2522, error 82.01%
Epoch 015/15	train loss 3.6550, error 66.88%	valid loss 6.2977, error 82.55%



已輸出: 09_RNN訓練與誤差報表.xlsx

10. Multi-Head Attention: 調整模型維度並與 RNN 比較

• 1. Loss (損失值)

這是算給電腦看的「扣分機制」。模型每猜錯一個答案，系統就會給予懲罰（分數）。如果正確答案是 100，模型猜 90，Loss 可能就是 10。訓練的目的就是讓這個 Loss 越來越低。

• 2. 錯誤率 (Error Rate)

就是一般考試的「錯題率」。與它相反的是準確率 (Accuracy)。給模型看 100 張貓的圖片，它把其中 5 張猜成狗，錯誤率就是 5%（準確率 95%）。

• 3. 困惑度 (Perplexity, 簡稱 PPL)

這是語言模型專用的指標，代表模型在預測下一個字時，在多少個字之間「猶豫不決」。如果 PPL 是 3，代表模型在選下一個字時，像是在 3 個同樣合理的字中做選擇。數字越小，代表模型越篤定、預測越精準。

- **訓練集 (Training Set) = 課本與練習題**
 - 這是給 AI 學習用的資料。AI 會反覆看這些資料、調整自己的參數，試圖讓訓練集上的 Loss 降到最低。
- **驗證集 (Validation Set) = 模擬考題**
 - 這是 AI 在訓練過程中從未看過的資料。每訓練完一個階段，我們就會用驗證集來測試 AI。目的是檢查 AI 是不是真的「學會了原理」，還是只是「把課本死背下來」。

理解AI本質

- 人們只需要關注於提供大量的輸入數據和預期輸出結果。
- 也就是提供X和Y，電腦透過巨量數據，自動推導出最符合所有數據分布情況的函數

目的導向

- 人文/歷史研究應該關心的是目的，而非AI架構的方法，後者是計算機科學的事。
- 因此，從AI底層來看，清楚自己的目的是使用AI的第一要務
- 要讓AI訓練成自己想要的樣子之前，人們必須知道，自己想要的是什麼

注意力機制

- Attention
- 解決上下文問題
- 概念分解，並且針對相關度打分數，稱之為 Embedding 嵌入
- 四十四隻石獅子，詞彙之間的關係為何？
- 事實上我們不知道標籤=維度的具體涵義，它是模型自動生成的

標籤	數字	量詞	形容詞	名詞
四十四	80	0	20	10
隻	0	65	20	15
石	0	50	30	70
獅子	0	0	35	75

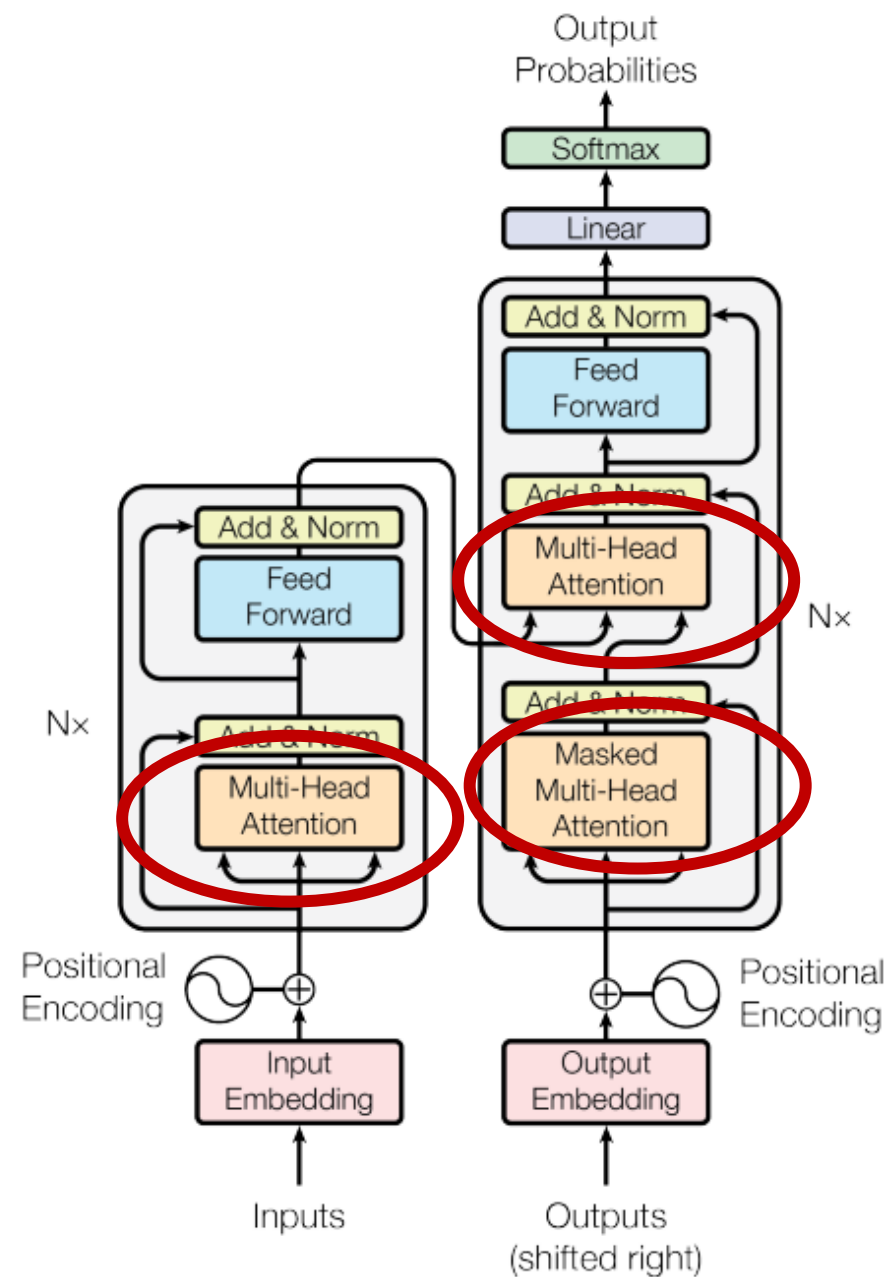


Figure 1: The Transformer - model architecture.

多頭注意

- MultiHeadAttention
- 標籤=維度 混雜了語言、語法、感受...等等
- 注意力機制就是檢測詞與詞之間的關聯程度
- 每一個詞都有上千的向量，因此不會將全部直接丟進去
- 根據標籤類別，進行分類後，再丟入注意力機制，因此稱之為MultiHeadAttention 多頭注意
- 如何提取標籤中的某特定分類，則可以透過線性層。
- 向量總表->線性層->語法表
- 向量總表->線性層->需求表

需求表

標籤	數字	量詞	形容詞	名詞
四十四	...	...	...	...
隻	...	...	...	...
石	...	...	...	...
獅子	10	50	100	0

語法表

標籤	數字	量詞	形容詞	名詞
四十四	80	0	20	10
隻	0	65	20	15
石	0	50	30	70
獅子	0	0	35	75

關聯度

- 關聯度 = 需求X語法 (向量)
- 乘法時，當數值大於1時，會無限膨脹。當數值小於1時，會無限趨近於0。
- 為解決此問題，需要透過Softmax，將數字轉換成百分比。
- Softmax=一種線性函數=數值/所有數字總和*%
- 透過不同類別之間不斷相乘和Softmax後，組合成龐大矩陣，在進行降維。
- 這一過程稱之為Attention
- 如此我們就知道：四十四隻石獅子 這句話每個字詞的詞性。

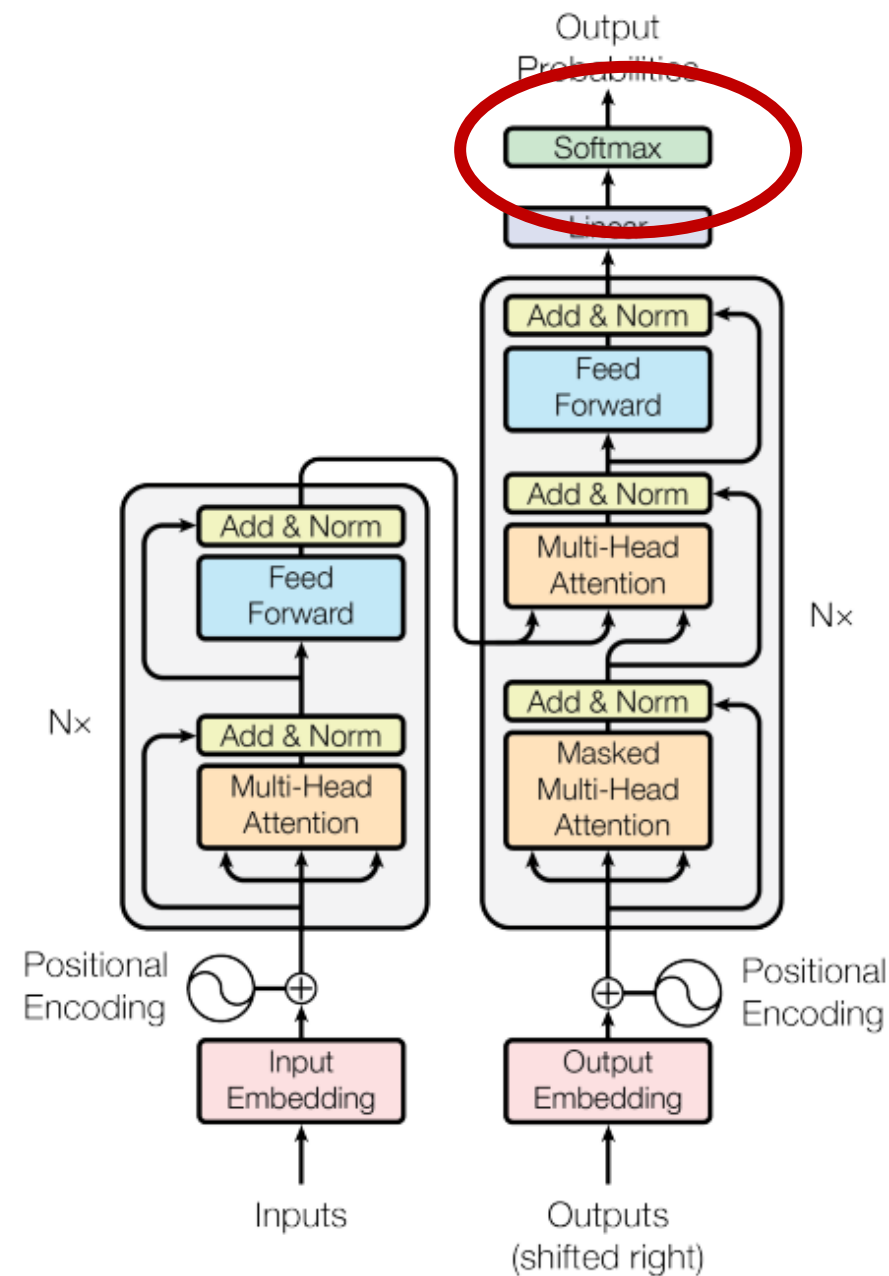


Figure 1: The Transformer - model architecture.

計算量與記憶體

- 在Transformer中的 Masked Multi-Head Attention 就使此一過程重複數次
- $\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k}) \times V$
- 因為是大量的矩陣相乘，所以 GPU適合AI運算，因為GPU本質上就是矩陣運算。
- 當然這也將會是侷限，矩陣運算可以以張量的方式呈現。所以各大晶片設計商，都在設計更適合AI運算的張量處理器，例如google的 TPU、AMD的NPU...等等
- 這過程中，有大量數據需要暫存，因此記憶體的需求量大增
- RNN和Multi-Head Self-Attention的區別：
- RNN 因為具備時間依賴性，無法在硬體上並行運算；Multi-Head Self-Attention 支援高度並行，極度適合善於平行加速的 GPU。
- RNN 容易發生梯度消失或爆炸，難以有效保留遠距離的歷史資訊；Self-Attention 任意兩個位置的路徑長度皆為 1，能直接跨越距離限制捕捉關聯。
- RNN 每個時間點的複雜度為線性 $O(n)$ ，但總時間隨序列長度直線累積；Self-Attention 的時間與空間複雜度皆為平方級 $O(n^2)$ ，在處理超長文本時代價較高。

檔案

- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

磁碟 64.96 GB 可用

10. Multi-Head Attention: 調整模型維度並與 RNN 比較

Attention 可讓每個位置直接比較前文其他位置; RNN 則透過逐步隱藏狀態傳遞。Attention 較適合平行運算, 但運算量會隨序列長度增加; 小資料不保證優於 RNN。

```
[13] 57 秒
1 attention_dim=ask_int("Attention 模型維度",64,8,512)
2 while True:
3     attention_heads=ask_int("Attention Heads",4,1,16)
4     if attention_dim%attention_heads==0: break
5     print("模型維度必須能被 Heads 整除。")
6 attention_blocks=ask_int("Attention Blocks",2,1,6)
7 attention_ff_dim=ask_int("Attention FFN 隱藏維度",256,16,2048)
8 attention_epochs=ask_int("Attention 訓練輪數",15,1,200)
9 attention_lr=ask_float("Attention 學習率",.001,.00001,.1)
10 attention_dropout=ask_float("Attention Dropout",.1,0,.8)
11
12 class DenseAttentionBlock(nn.Module):
13     def __init__(self):
14         super().__init__(); self.attn=nn.MultiheadAttention(attention_dim,attention_heads,dropout=attention_dropout,batch_first=True)
15         self.norm1=nn.LayerNorm(attention_dim); self.norm2=nn.LayerNorm(attention_dim)
16         self.ff=nn.Sequential(nn.Linear(attention_dim,attention_ff_dim),nn.GELU(),nn.Dropout(attention_dropout),nn.Linear(attention_ff_dim,attention_dim))
17     def forward(self,x,pad_mask,causal):
18         a,_=self.attn(x,x,x,key_padding_mask=pad_mask,attn_mask=causal,need_weights=False)
19         x=self.norm1(x+a); return self.norm2(x+self.ff(x))
20
21 class AttentionLanguageModel(nn.Module):
22     def __init__(self):
23         super().__init__(); self.embedding=nn.Embedding(model_vocab_size,embedding_dim,padding_idx=pad_id)
24         with torch.no_grad(): self.embedding.weight.copy_(torch.from_numpy(initial_embeddings))
25         self.project=nn.Linear(embedding_dim,attention_dim); self.position=nn.Embedding(context_length,attention_dim)
26         self.blocks=nn.ModuleList([DenseAttentionBlock() for _ in range(attention_blocks)]); self.output=nn.Linear(attention_dim,bpe_vocab_size)
```



繁體中文文本_AI原理14階段整合教學_COLAB.ipynb ☆ ☁

檔案 編輯 檢視畫面 插入 執行階段 工具 說明



分享

哲維

Q 指令 + 程式碼 + 文字 | ▶ 全部執行

RAM 磁碟

檔案



..

AI原理整合教學輸出

sample_data

水經注.md

洛陽伽藍記.md

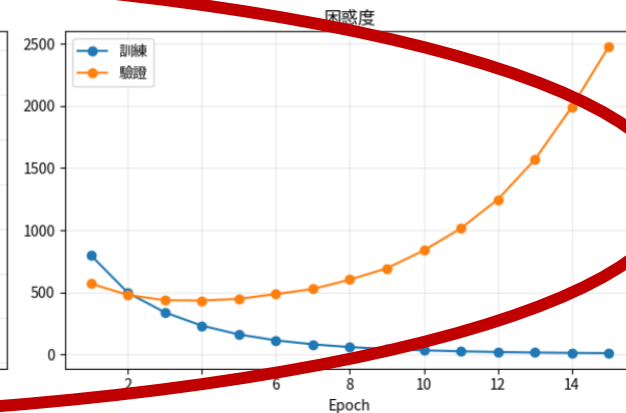
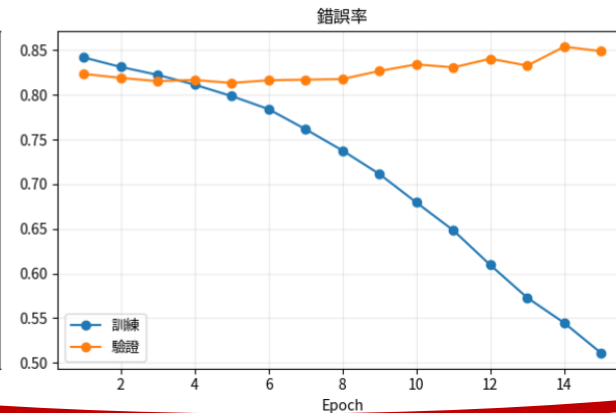
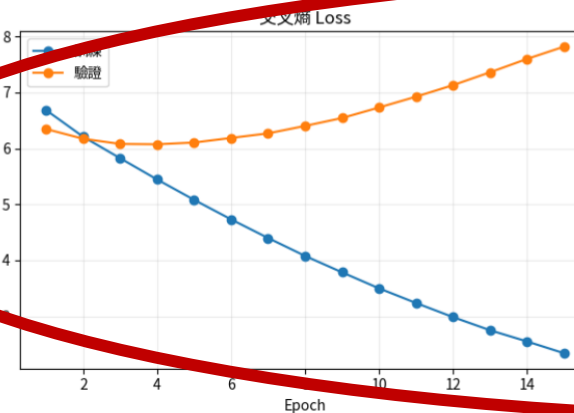
繁體中文文本_AI原理14階段整合...

Attention 訓練輪數 [預設 15]:

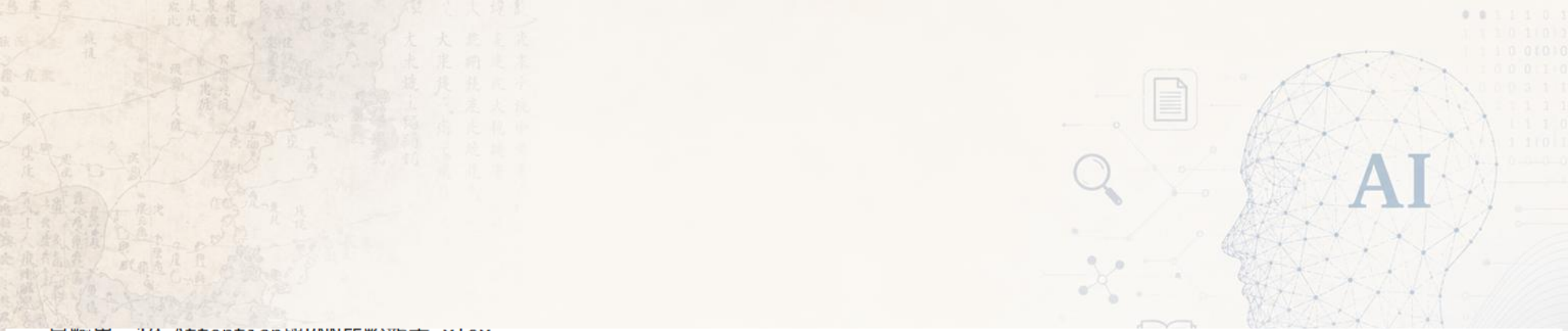
Attention 學習率 [預設 0.001]:

Attention Dropout [預設 0.1]:

Epoch 001/15	train loss 6.6807, error 84.20%	valid loss 6.3440, error 82.35%
Epoch 002/15	train loss 6.2064, error 83.14%	valid loss 6.1711, error 81.91%
Epoch 003/15	train loss 5.8230, error 82.25%	valid loss 6.0786, error 81.54%
Epoch 004/15	train loss 5.4434, error 81.15%	valid loss 6.0724, error 81.68%
Epoch 005/15	train loss 5.0799, error 79.88%	valid loss 6.1039, error 81.34%
Epoch 006/15	train loss 4.7313, error 78.41%	valid loss 6.1859, error 81.64%
Epoch 007/15	train loss 4.3987, error 76.16%	valid loss 6.2672, error 81.71%
Epoch 008/15	train loss 4.0791, error 73.78%	valid loss 6.4000, error 81.78%
Epoch 009/15	train loss 3.7858, error 71.14%	valid loss 6.5413, error 82.68%
Epoch 010/15	train loss 3.4999, error 67.95%	valid loss 6.7295, error 83.42%
Epoch 011/15	train loss 3.2386, error 64.82%	valid loss 6.9216, error 83.09%
Epoch 012/15	train loss 2.9871, error 60.94%	valid loss 7.1283, error 84.06%
Epoch 013/15	train loss 2.7523, error 57.27%	valid loss 7.3568, error 83.29%
Epoch 014/15	train loss 2.5522, error 54.45%	valid loss 7.5962, error 85.40%
Epoch 015/15	train loss 2.3423, error 51.06%	valid loss 7.8144, error 84.90%



證據·脈絡·可驗證性
AI 賦能史學研究新視野



已輸出：10_Attention與KNN比較報告.xlsx

	模型	Embedding維度	上下文表示維度	上下文機制	參數量	驗證loss	驗證錯誤率	延遲ms	每秒樣本
0	RNN	64	64	逐步隱藏狀態	399770	6.297742	0.825503	0.909267	70386.350679
1	Multi-Head Attention	64	64	4 Heads；每Head 16維	497114	7.814388	0.848993	2.332680	27436.250443



證據·脈絡·可驗證性
AI 賦能史學研究新視野

混和專家架構

- Mixture of Experts
- 隨著模型越來越大，線性層越來越龐雜，稱之為Dense模型(稠密模型)
- 這導致運算量大增，過於耗能。
- 因而透過拆解的方式，將模型拆成N個前饋神經網絡
- 每個前饋神經網絡擅長不同的問題，因而稱為混和專家架構
- 人類不知道哪一個模型擅長什麼，這是由AI自己訓練的
- 至於要發給哪個專家，由一個稱之為路由的線性層來計算。
- 路由層的工作是評分，用來確認具體要發給哪些專家來計算
- 在加上這些分數依據比例，總和起來。

- 不斷增加參數 -> 巨量參數模型稱之為 Dense模型 稠密模型
這樣的稠密模型知道許多事，但也導致 每次問問題時，都要全部重算，這導致 算力浪費 速度慢 效率低
- google 的 MoE = Mixture of Experts 技術
- 將FNN拆解成 256個小FNN，每個小FNN僅處理特定問題。每個FNN都是一個專家
怎麼拆 依然是利用線性層來學習的
同樣的 什麼問題要交給哪個小FNN 也是由線性層推論的結果，這個線性層 又被稱為 路由層
- 這個路由層的功能是為每個小FNN打分數，分數越高代表它越適合處理怎麼樣的問題。
將問題發給分數最高的幾個小FNN來處理。選擇多少專家是由模型設定好的。再將分數轉換成權重比例，最終將成果依據權重相加得到結果。這個相加的過程 就是Mixture
- Dense模型會將全部參與計算 但 MoE只會挑選部分重要的參數來計算
- 因此 MoE模型 的速度比較快
但也會有所損失：許多沒有被參考的FNN 也就被丟棄了。
因此以表現來看 Dense模型 > MoE模型
但考量到速度 MoE模型在表現上 其實也已經很不錯了
- DeepSeek 的就是將這技術第一次實踐
- MoE = Mixture of Experts
專業化知識
在生成過程中 加入 混和專家架構

注意力
機制

路由
(線性層)

60%

1號
前饋神經網絡

2號
前饋神經網絡

3號
前饋神經網絡

40%

N號
前饋神經網絡

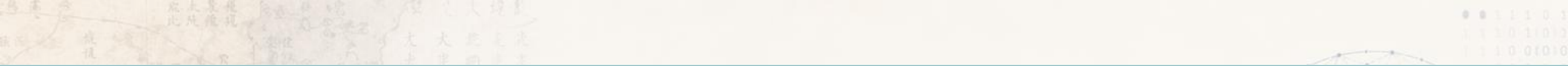
結果

1號X60%+N號X40%...

證據·脈絡·可驗證性
AI 賦能史學研究新視野

Dense架構 -> MoE架構

- 將前饋神經網絡拆分成許多小專家，再由一個選擇專家的路由，來決定要由那些專家來運算，最終將運算結果進行整合
- 因為並非全部的前饋神經網絡參與計算，所以降低了計算量
- 由於許多前饋神經網絡是被拋棄的，會帶來正確率的損耗。
- 但在實際運行中，落差不大。比起消耗算力，是更有效率的
- MoE架構的總參數量>Dense架構，但參與計算的參數只有Dense架構的三分之一，每次計算卻是全部參數一起。



檔案

..
AI原理整合教學輸出
sample_data
水經注.md
洛陽伽藍記.md
繁體中文文本_AI原理14階段整合...

磁碟 64.96 GB 可用

11. Mixture of Experts: 調整 Expert 參數並比較三種模型

MoE 在 Attention 之後以 Router 為每個 TOKEN 選擇 Top-k Experts。總參數可以很大，但每次只啟用少數 Expert。教學版同時加入負載平衡 Loss，避免所有 TOKEN 都擠到同一位 Expert。

[14] 3分鐘

```
1 moe_experts=ask_int("MoE Expert 數", 8, 2, 32)
2 moe_top_k=ask_int("每個 TOKEN 啟用 Experts (Top-k)", 2, 1, moe_experts)
3 moe_hidden_dim=ask_int("每位 Expert 隱藏維度", 128, 16, 2048)
4 moe_blocks=ask_int("MoE Blocks", 2, 1, 6)
5 moe_epochs=ask_int("MoE 訓練輪數", 15, 1, 200)
6 moe_lr=ask_float("MoE 學習率", .001, .00001, .1)
7 moe_balance_weight=ask_float("負載平衡 Loss 權重", .01, 0, 1)
8 moe_z_weight=ask_float("Router z-loss 權重", .001, 0, 1)
9
10 class Expert(nn.Module):
11     def __init__(self):
12         super().__init__(); self.net=nn.Sequential(nn.Linear(attention_dim, moe_hidden_dim), nn.GELU(), nn.Dropout(attention_dropout), nn.Linear(moe_hidden_dim, at
13     def forward(self, x): return self.net(x)
14
15 class SparseMoE(nn.Module):
16     def __init__(self):
17         super().__init__(); self.router=nn.Linear(attention_dim, moe_experts, bias=False); self.experts=nn.ModuleList([Expert() for _ in range(moe_experts)
18     def forward(self, x, valid_mask):
19         shape=x.shape; flat=x.reshape(-1, shape[-1]); valid=valid_mask.reshape(-1); valid_x=flat[valid]; output=torch.zeros_like(flat)
20         if valid_x.numel()==0:
21             zero=x.sum()*0; return output.reshape(shape), zero, zero, torch.zeros(moe_experts, device=x.device)
22         logits=self.router(valid_x); probs=torch.softmax(logits, dim=-1); values, selected=torch.topk(logits, k=moe_top_k, dim=-1); gates=torch.softmax(values, d
23         valid_out=torch.zeros_like(valid_x); assignments=torch.zeros_like(probs, dtype=torch.bool); assignments.scatter_(1, selected, True)
24         for expert_id, expert in enumerate(self.experts):
25             token nos.slot=torch.where(selected==expert_id)
```



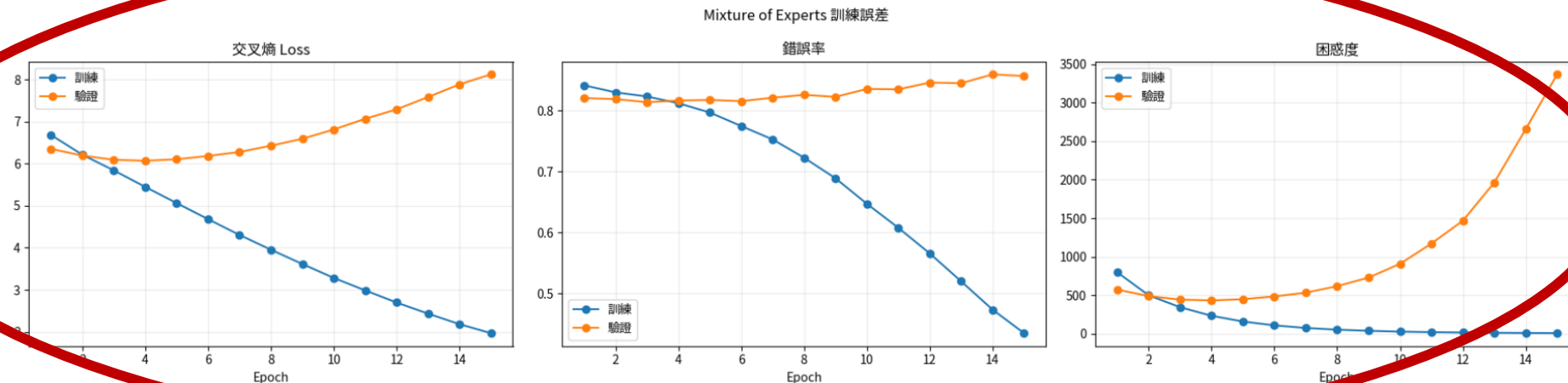


檔案



- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

```
Epoch 004/15 | train CE 5.4470, error 81.21% | valid CE 6.0689, error 81.64%
Epoch 005/15 | train CE 5.0626, error 79.68% | valid CE 6.1043, error 81.74%
Epoch 006/15 | train CE 4.6797, error 77.45% | valid CE 6.1824, error 81.54%
Epoch 007/15 | train CE 4.3047, error 75.26% | valid CE 6.2773, error 82.11%
Epoch 008/15 | train CE 3.9537, error 72.26% | valid CE 6.4264, error 82.58%
Epoch 009/15 | train CE 3.6143, error 68.88% | valid CE 6.5914, error 82.25%
Epoch 010/15 | train CE 3.2834, error 64.70% | valid CE 6.8113, error 83.56%
Epoch 011/15 | train CE 2.9877, error 60.75% | valid CE 7.0657, error 83.49%
Epoch 012/15 | train CE 2.6981, error 56.55% | valid CE 7.2911, error 84.60%
Epoch 013/15 | train CE 2.4346, error 51.97% | valid CE 7.5830, error 84.50%
Epoch 014/15 | train CE 2.1840, error 47.32% | valid CE 7.8853, error 85.94%
Epoch 015/15 | train CE 1.9710, error 43.47% | valid CE 8.1221, error 85.67%
```



已輸出：11_MoE與RNN_Attention比較報表.xlsx

	模型	上下文表示維度	參數量	驗證loss	驗證錯誤率	困惑度	延遲ms	每秒樣本	計算特性
0	RNN	64	399770	6.297742	0.825503	543.343840	0.909267	70386.350679	逐步運算
1	Multi-Head Attention	64	497114	7.814388	0.848993	2475.970627	2.332680	27436.250443	所有 TOKEN 的 Dense FFN
2	Mixture of Experts	64	697178	8.122051	0.856711	3367.922789	6.855052	9336.179772	8位 Expert，每TOKEN啟用2位

	模型	上下文表示維度	參數量	驗證loss	驗證錯誤率	困惑度	延遲ms	每秒樣本	計算特性
0	RNN	64	399770	6.297742	0.825503	543.343840	0.909267	70386.350679	逐步運算
1	Multi-Head Attention	64	497114	7.814388	0.848993	2475.970627	2.332680	27436.250443	所有 TOKEN 的 Dense FFN
2	Mixture of Experts	64	697178	8.122051	0.856711	3367.922789	6.855052	9336.179772	8位 Expert，每TOKEN啟用2位

	Block	Expert	路由使用率
0	1	1	0.121379
1	1	2	0.162546
2	1	3	0.086994
3	1	4	0.141727
4	1	5	0.090091
5	1	6	0.130756
6	1	7	0.131468
7	1	8	0.135040
8	2	1	0.133841
9	2	2	0.125723
10	2	3	0.094458
11	2	4	0.154726
12	2	5	0.123100
13	2	6	0.160201
14	2	7	0.107149
15	2	8	0.100802

文字生成=解碼器

- 將矩陣轉化為人類看得懂的語言
- 解碼器是一個字一個字解碼的
- 逐字輸出可能Token的機率
- <開始> -> 解碼器 -> 我 10%、你2%...等等
- <開始> 我 -> 解碼器 -> 是 85%、水果 0.01%...等等
- <開始> 我是 -> 解碼器 -> 哲維92%、老師 0.01%...等等
- 最後生成<結束>

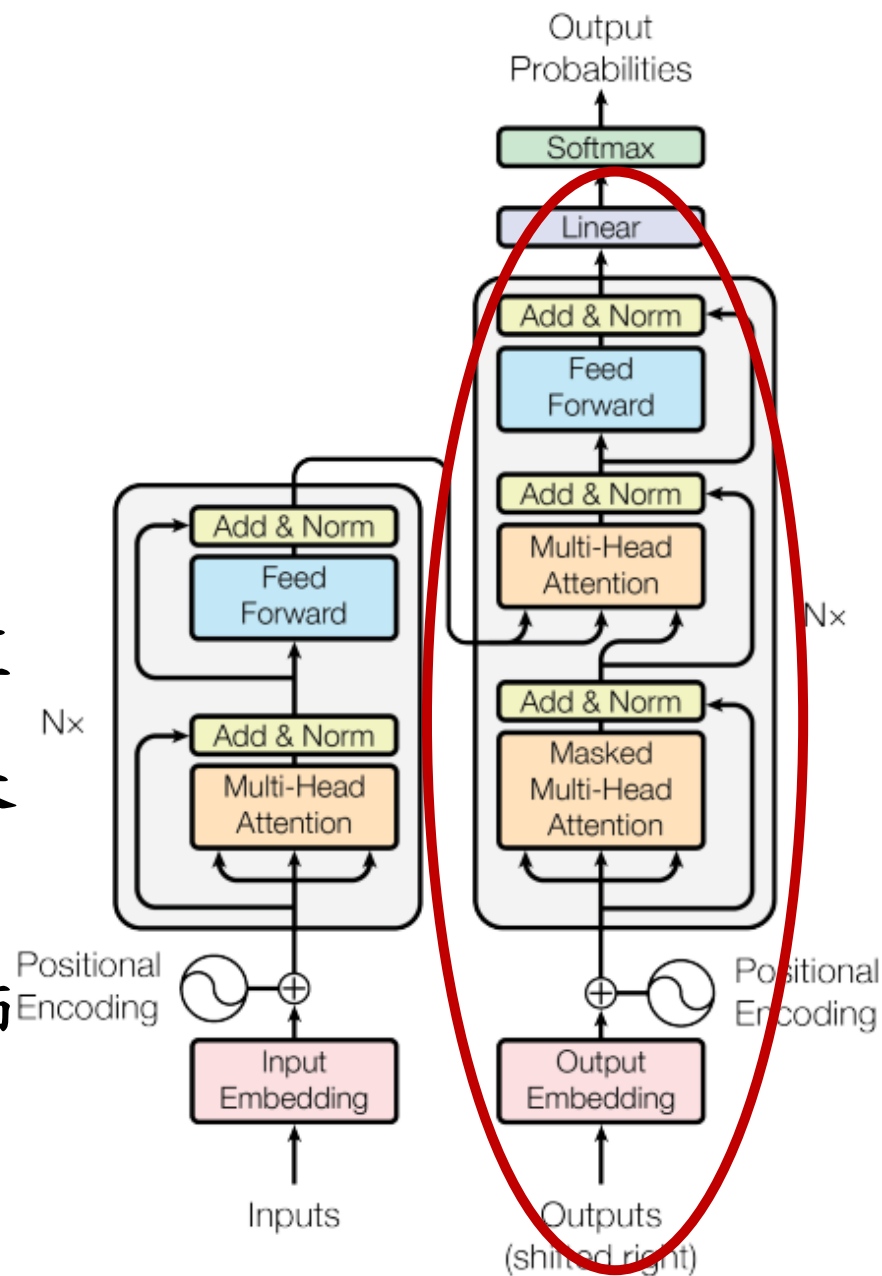
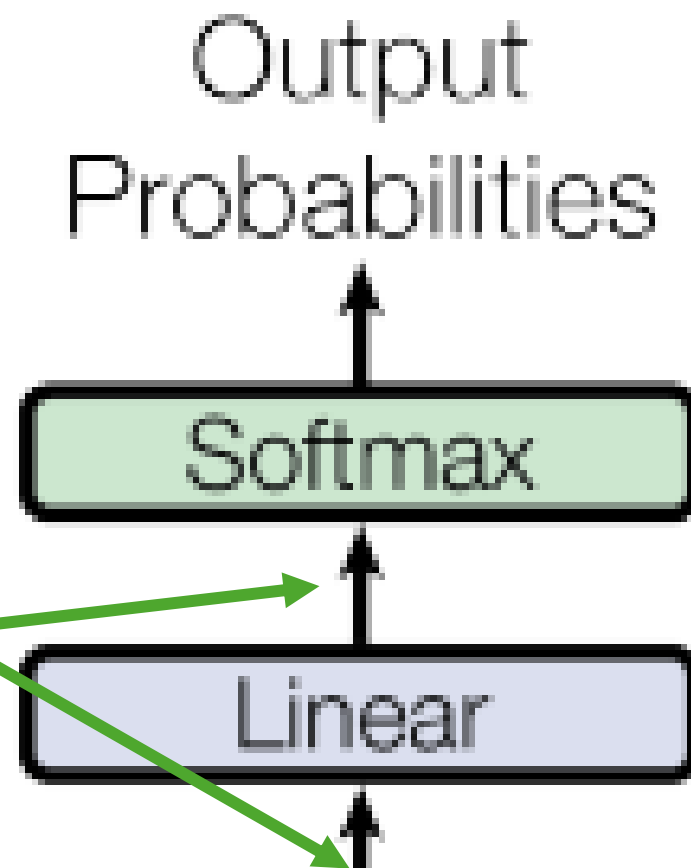


Figure 1: The Transformer - model architecture.

線性層

最基礎的神經網絡結構

- GPT-2 模型的線性層 linear
- 輸入是1600維(向量化)
- 輸入是50257維(Token)
- 在真正的模型中，參數都是算出來的。



編碼(Encoder)<解碼器(decoder)

- 生成文字算力消耗大於輸入文字
- 平均來說 1個token = 1.5~2個中文字、0.75個英文單詞
- GPT-5.4
- 價格
- 輸入：US\$2.50 / 一百萬個 token
- 快取輸入：US\$0.25 / 一百萬個 token
- 輸出：US\$15.00 / 一百萬個 token

溫度Temperature

- 為了讓模型更有創造性，給予概率較低的字詞較高的機會，這稱之為溫度Temperature
- 溫度Temperature越高，隨機性越強
- 溫度Temperature越低，隨機性越弱

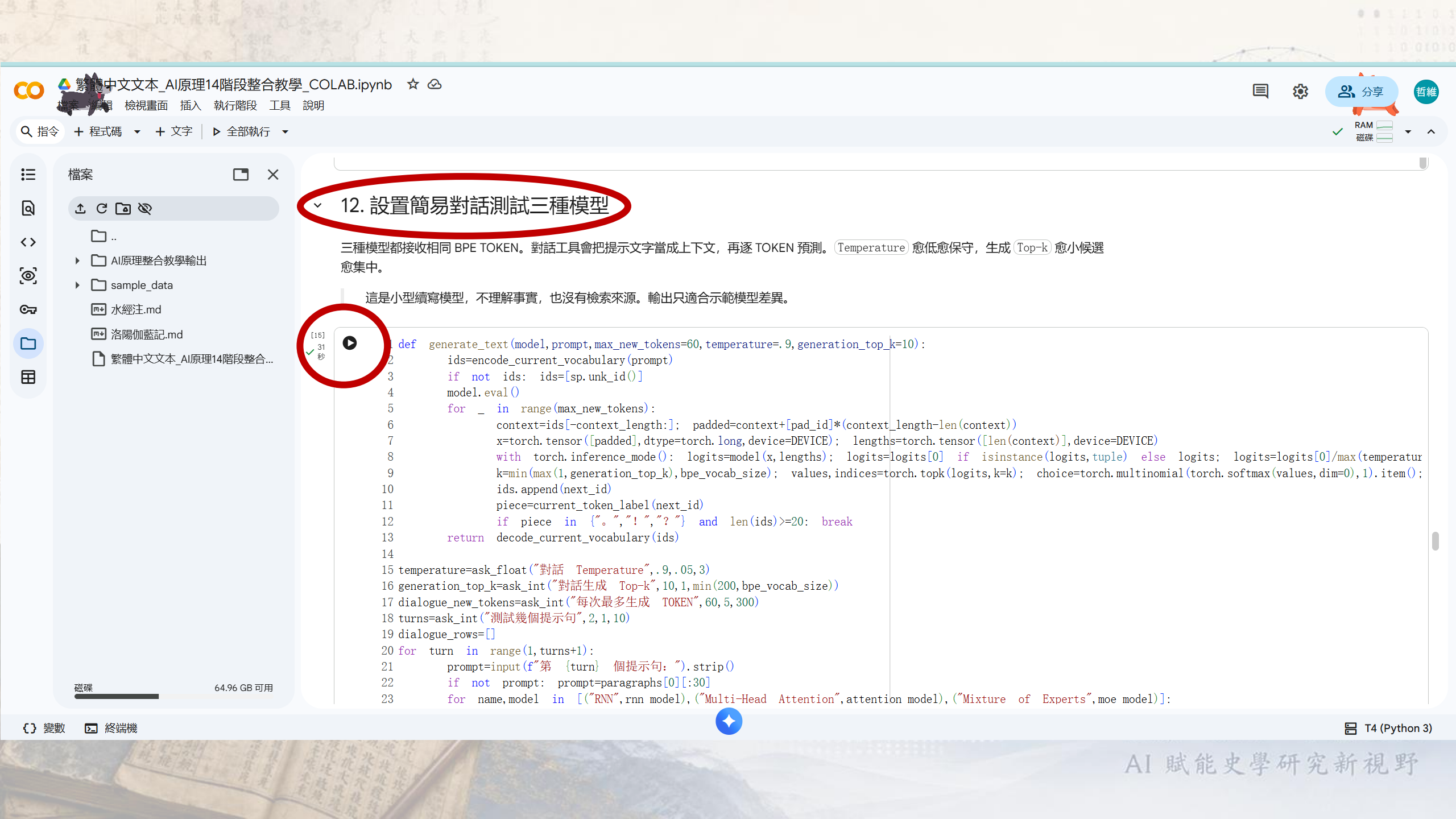
Top-K

從前K個代率最高的Token中進行選擇

Top-K = 1 只選最高的

Top-K = 2 選擇前兩高的

...



12. 設置簡易對話測試三種模型

三種模型都接收相同 BPE TOKEN。對話工具會把提示文字當成上下文，再逐 TOKEN 預測。Temperature 愈低愈保守，生成 Top-k 愈小候選愈集中。

這是小型續寫模型，不理解事實，也沒有檢索來源。輸出只適合示範模型差異。



```
def generate_text(model, prompt, max_new_tokens=60, temperature=.9, generation_top_k=10):
    ids=encode_current_vocabulary(prompt)
    if not ids: ids=[sp.unk_id()]
    model.eval()
    for _ in range(max_new_tokens):
        context=ids[-context_length:]; padded=context+[pad_id]*(context_length-len(context))
        x=torch.tensor([padded], dtype=torch.long, device=DEVICE); lengths=torch.tensor([len(context)], device=DEVICE)
        with torch.inference_mode(): logits=model(x, lengths); logits=logits[0] if isinstance(logits, tuple) else logits; logits=logits[0]/max(temperatur
        k=min(max(1, generation_top_k), bpe_vocab_size); values, indices=torch.topk(logits, k=k); choice=torch.multinomial(torch.softmax(values, dim=0), 1).item();
        ids.append(next_id)
        piece=current_token_label(next_id)
        if piece in {".", "!", ",", "? "}: break
    return decode_current_vocabulary(ids)

temperature=ask_float("對話 Temperature", .9, .05, 3)
generation_top_k=ask_int("對話生成 Top-k", 10, 1, min(200, bpe_vocab_size))
dialogue_new_tokens=ask_int("每次最多生成 TOKEN", 60, 5, 300)
turns=ask_int("測試幾個提示句", 2, 1, 10)
dialogue_rows=[]
for turn in range(1, turns+1):
    prompt=input(f"第 {turn} 個提示句: ").strip()
    if not prompt: prompt=paragraphs[0][:30]
    for name, model in [("RNN", rnn_model), ("Multi-Head Attention", attention_model), ("Mixture of Experts", moe_model)]:
```



檔案

📁 ..

▶ 📁 AI原理整合教學輸出

▶ 📁 sample_data

📄 水經注.md

📄 洛陽伽藍記.md

📄 繁體中文文本_AI原理14階段整合...

```
[15]: 27 dialogue_df=pd.DataFrame(dialogue_rows)
      28 save_excel(OUTPUT_DIR/"12_三模型簡易對話測試.xlsx",{"對話結果":dialogue_df})

✓ 31 秒
```

對話 Temperature [預設 0.9]:
對話生成 Top-k [預設 10]:
每次最多生成 TOKEN [預設 60]:
測試幾個提示句 [預設 2]: 2
第 1 個提示句: 爾朱

【RNN】爾朱王公為爾朱榮所害。卿士刀入人人 ?? 為執表。

【Multi-Head Attention】爾朱尚書令所立也。在陽王所立也。在西。法之北爾朱榮。居北海入洛。

【Mixture of Experts】爾朱陽寺。在僧周餘闍官至七青陽門外三里寺。所謂日。去地丈十年中。
第 2 個提示句: 士馬

【RNN】士馬一千。餘九 ?? 。 ?? ?? 羅 ?? 羅鹿佛。高臺林。

【Multi-Head Attention】士馬之君之還活。因以為名。其是也是也。秦太上公。生嫁。

【Mixture of Experts】士馬老所立也。在慕義里追福。並雕牆峻詩齊據。夜。
已輸出: 12_三模型簡易對話測試.xlsx

13. 加入新文本：微調 Attention 與 MoE，比較前後並測試對話

這一階段先保留原來的 BPE TOKEN ID，再自動找出新文本中會變成 `<unk>` / `??` 的原始字元。程式會把未知字元加入詞彙表，並同步擴充 Attention 與 MoE 的 Embedding、輸出層及 `<PAD>` 位置，因此不必把不同生僻字全部壓成同一個問號。

新增字元的向量先以既有 Embedding 平均值加微小亂數初始化，再由新文本微調。舊 TOKEN 的 ID 與已訓練權重保持不變，所以仍可比較微調前後差異。

微調前後同時比較：

- 新文本 Loss / 錯誤率。
- 原驗證集 Loss / 錯誤率，觀察是否遺忘舊文本。



檔案

- ..
- AI原理整合教學輸出
- sample_data
- 水經注.md
- 洛陽伽藍記.md
- 繁體中文文本_AI原理14階段整合...

【Multi-Head Attention】士馬之君之還活。因以為名。其是也是也。秦太上公。亦生嫁。

【Mixture of Experts】士馬老所立也。在慕義里追福。並雕牆峻詩齊據。雍。

已輸出: 12_三模型簡易對話測試_xlax

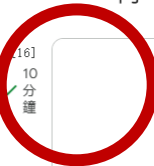
13. 加入新文本：微調 Attention 與 MoE，比較前後並測試對話

這一階段先保留原來的 BPE TOKEN ID，再自動找出新文本中會變成 `<unk>` / `??` 的原始字元。程式會把未知字元加入詞彙表，並同步擴充 Attention 與 MoE 的 Embedding、輸出層及 `<PAD>` 位置，因此不必把不同生僻字全部壓成同一個問號。

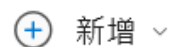
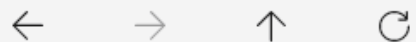
新增字元的向量先以既有 Embedding 平均值加微小亂數初始化，再由新文本微調。舊 TOKEN 的 ID 與已訓練權重保持不變，所以仍可比較微調前後差異。

微調前後同時比較：

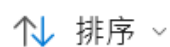
- 新文本 Loss / 錯誤率。
- 原驗證集 Loss / 錯誤率，觀察是否遺忘舊文本。
- Embedding 平均位移。
- `<unk>` 修復前後比例與新增 TOKEN 對照表。
- 同一提示句的生成結果。



```
1 print("請上傳新增文本 (DOCX、PDF、TXT 或 Markdown) : ")
2 new_upload=files.upload()
3 if not new_upload: raise ValueError("尚未上傳新增文本。")
4 new_filename,new_bytes=next(iter(new_upload.items()))
5 new_markdown,new_noise_df,new_mode=extract_and_clean(new_filename,new_bytes)
6 new_plain=re.sub(r"^\s*#{1,6}\s+", "", new_markdown, flags=re.MULTILINE).strip()
7 new_paragraphs=[re.sub(r"^\s*#{1,6}\s+", "", p).strip() for p in re.split(r"\n+", new_markdown) if p.strip()]
8
9 # 先用原 SentencePiece 模型找出真正落入 <unk> 的原始字元。
10 unknown_char_counts=Counter(); raw_piece_count=raw_unk_count=0
11 for paragraph in new_paragraphs:
```



新增 ▾



排序 ▾



檢視 ▾



預覽



常用



圖庫

▼ 哲維 - 個人

> Microsoft Copilot Cha

> 助理用筆記

> 附件

> 待完成工作

> 桌面

> 筆記

> 詹峻軒報帳用

> 德國掃描圖書資料

> 德國檔案館藏台灣史料

▼ OneDrive - 國立成功大學



✓ 1954年台灣醫師名錄



✓ RZ 208_86377 Formosa



✓ 手稿筆記



✓ 法肯豪森將軍手稿



✓ 水經注.md



✓ 洛陽伽藍記.md



✓ 洛陽伽藍記.pdf



✓ 張哲維 當事人視角下的陶德曼調停 新史學 2025 36卷2期.pdf

請上傳新增文本 (DOCX、PDF、TXT 或 Markdown)：

選擇檔案 未選擇任何檔案

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 水經注... 水經注...

已新增 2,713 個字元 TOKEN；詞彙量由 3,034 擴充為 5,747。

新增文本：水經注.md | 擷取：純文字/Markdown | BPE TOKEN：354,129 | 修復前 <unk>：8.38% | 修復後 <unk>：0.00%

已輸出：13_新增文本清理與詞彙擴充報表.xlsx

新增文本微調輪數 [預設 5]：

微調學習率 [預設 0.0003]：

微調前後共同測試提示句：洛陽

Attention 微調 01/5 | loss 5.6965 | error 82.42%

Attention 微調 02/5 | loss 5.0257 | error 77.83%

Attention 微調 03/5 | loss 4.7817 | error 76.16%

Attention 微調 04/5 | loss 4.6125 | error 75.02%

Attention 微調 05/5 | loss 4.4802 | error 74.09%

MoE 微調 01/5 | loss 5.7109 | error 82.30%

MoE 微調 02/5 | loss 5.0015 | error 77.66%

MoE 微調 03/5 | loss 4.7446 | error 75.74%

MoE 微調 04/5 | loss 4.5718 | error 74.57%

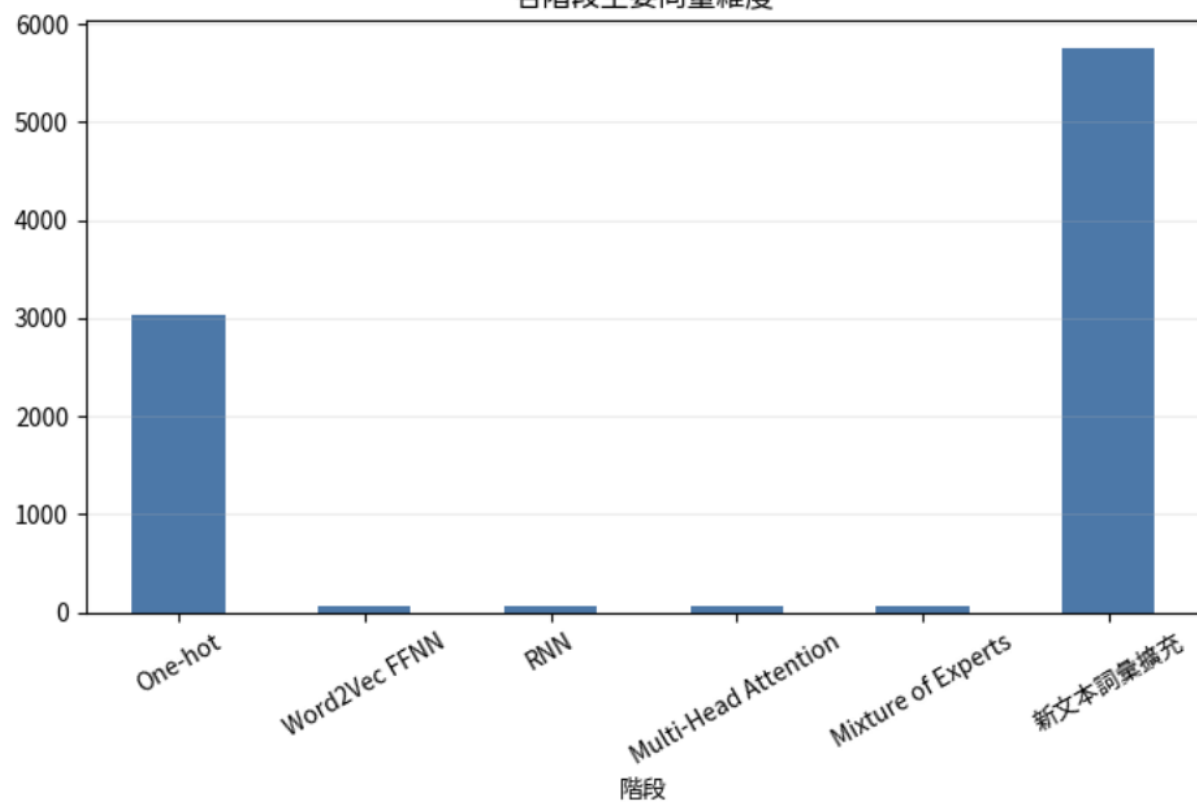
MoE 微調 05/5 | loss 4.4325 | error 73.70%

已輸出：13_新增文本微調前後比較.xlsx

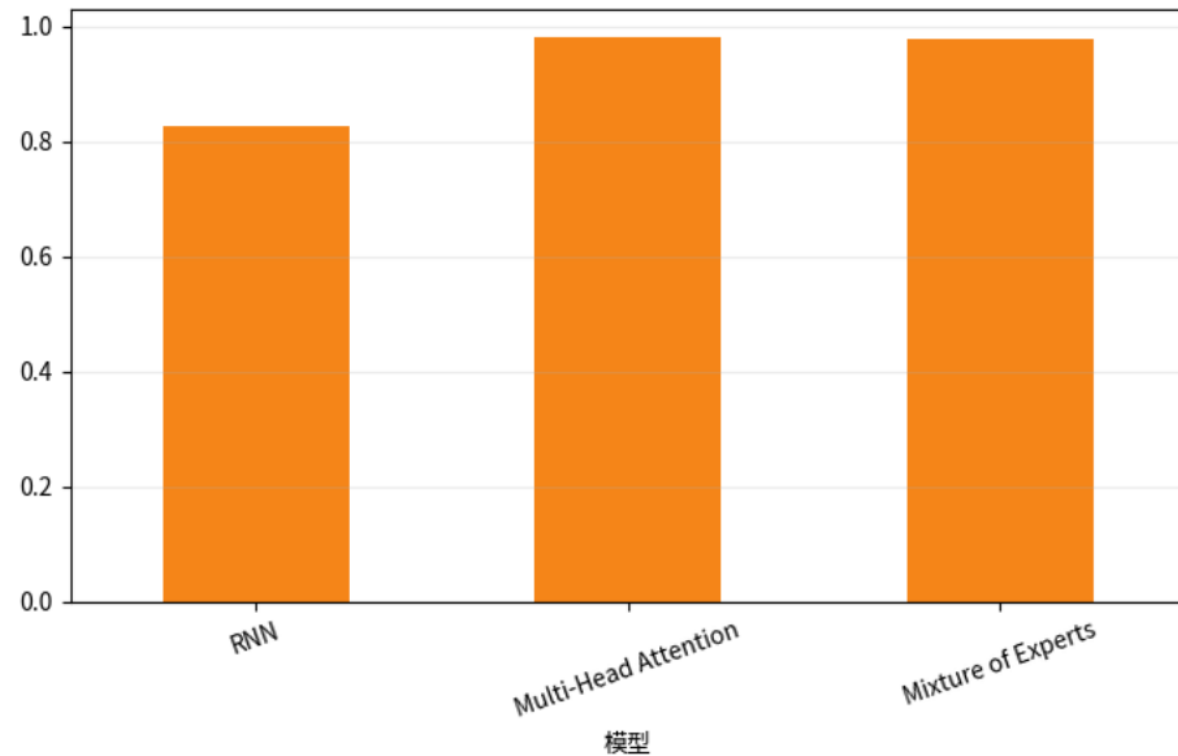
	模型	階段	新文本loss	新文本錯誤率	舊驗證集loss	舊驗證集錯誤率	對話結果	Embedding平均位移L2
0	Multi-Head Attention	微調前	11.382064	0.956615	7.817359	0.848993	洛陽寺寺寺。在不聽里。在道南一里有洛陽。太傅。卷二二尺。	NaN
1	Mixture of Experts	微調前	11.414453	0.961095	8.124506	0.856711	洛陽武宣王魏所立也。在建春門外石塔南。穀水周圍遶。城三建春門外。	NaN
2	Multi-Head Attention	微調後	4.285757	0.725905	7.871397	0.979866	洛陽十 桓 水 水 水 漢水 水 又南流與二水會水,水出西北溪,東出武山,南流注於河。	0.595827
3	Mixture of Experts	微調後	4.233000	0.721810	7.948071	0.978188	洛陽東山,南,南流溪水 谷水出東山在北山,逕五里縣故城南,東流注於河。	0.566358

... 已輸出：14_全流程向量維度與模型比較總表.xlsx

各階段主要向量維度



三模型最終錯誤率



階段					
階段	主要向量維度	維度意義	參數量	補充說明	
0	One-hot	3034	原始詞彙量V	0	每個TOKEN只有一格為1
1	Word2Vec FFNN	64	Embedding維度	391386	One-hot×線性層的稠密結果
2	RNN	64	隱藏狀態維度	399770	輸入Embedding仍為64維
3	Multi-Head Attention	64	模型維度	847091	4 Heads，每Head 16維
4	Mixture of Experts	64	模型輸出維度	1047155	Expert隱藏128維；8位中啟用2位
5	新文本詞彙擴充	5747	擴充後詞彙量V	0	新增2713個字元TOKEN，舊ID不變

	模型	狀態	舊驗證集loss	舊驗證集錯誤率	延遲ms	每秒樣本	相對Word2Vec的Embedding平均位移L2
0	RNN	原本本訓練後	6.297742	0.825503	0.909267	70386.350679	1.624808
1	Multi-Head Attention	新增文本微調後	7.871397	0.979866	1.776719	36021.459785	1.686185
2	Mixture of Experts	新增文本微調後	7.948071	0.978188	7.044413	9085.213497	1.574780

+ 程式碼

+ 文字

1分
鐘

```
56 final_dialogue_difference_df=pd.DataFrame(final_dialogue_difference_rows)
57 final_dialogue_path=OUTPUT_DIR/"15_新增文本前後對話回答比較.xlsx"
58 save_excel(final_dialogue_path, {"前後差異":final_dialogue_difference_df, "逐筆比較":final_dialogue_comparison_df, "左右對照":final_dialogue_wide_df})
59 display(final_dialogue_difference_df)
60 print(f"已輸出: {final_dialogue_path.name}")
61
```

要比較幾個提示句 [預設 3]: 3
每個回答最多生成 TOKEN [預設 60]:
比較用 Temperature [預設 0.8]:
比較用生成 Top-k (建議1) [預設 1]: 2
第 1 個測試提示句 [直接 Enter 使用微調測試句]: 爾朱榮

===== 第 1 組比較 =====

提示句: 爾朱榮

【Multi-Head Attention | 增加新文本前】

爾朱榮寺寺皇魏覺寺。酪歸所立也。形製甚有洛陽。工雕佛殿僧房充溢一里御道。

【Multi-Head Attention | 增加新文本後】

爾朱榮水出西北山在縣縣北山西北逕西,縣北,南逕西北,又東北流注於河。

【Mixture of Experts | 增加新文本前】

爾朱榮統寺內圖積治義里寺。為寺。所謂永一里。南眺洛寺。

【Mixture of Experts | 增加新文本後】

爾朱榮水出縣,地,故縣,山,水,縣,東,東逕於陽,故縣也。

第 2 個測試提示句 [直接 Enter 使用微調測試句]: 洛陽

===== 第 2 組比較 =====

提示句: 洛陽

【Multi-Head Attention | 增加新文本前】

洛陽寺寺寺寺。在融覺寺西一里。在南有租場。南北方三去地。

【Multi-Head Attention | 增加新文本後】

洛陽水 卷 水 渭水 出河水 水 左水 左水 左水 水 左水 左水 左合州水出縣北山,北流逕平陽縣故城南,又東逕平陽縣故城南

... 【Mixture of Experts | 增加新文本後】

爾朱榮水出縣,地,故縣,山,水,縣,東,東逕於陽,故縣也。

第 2 個測試提示句 [直接 Enter 使用微調測試句]: 洛陽

===== 第 2 組比較 =====

提示句: 洛陽

【Multi-Head Attention | 增加新文本前】

洛陽寺寺寺寺。在融覺寺西一里。在南有租場。南北方三去地。

【Multi-Head Attention | 增加新文本後】

洛陽水 卷 水 渭水 出河水 水 左水 左水 左水 水 左水 左水 左合州水出縣北山,北流逕平陽縣故城南,又東逕平陽縣故城南

【Mixture of Experts | 增加新文本前】

洛陽武宣捨宅所立也。在明堂刻社龍南有穀水周圍取四園。

【Mixture of Experts | 增加新文本後】

洛陽水出梁水 "池水 渭水 "北,西南流為,水出南山,北逕其山,東南流注於洛。

第 3 個測試提示句 [直接 Enter 使用微調測試句]: 土馬

===== 第 3 組比較 =====

提示句: 土馬

【Multi-Head Attention | 增加新文本前】

土馬射皆金色。不聽升寺。道比所立也。康敬。唯卷二尺。

【Multi-Head Attention | 增加新文本後】

土馬、武水 水 水 桓水 水 出南流逕長沙國縣北,又東北流,左會於洛,水出西山,北流逕其縣故城北。

【Mixture of Experts | 增加新文本前】

土馬老所立也。為憑王公。在寶光寺西。屋宇六造佛殿一所。

【Mixture of Experts | 增加新文本後】

土馬 淮水 縣水出山,江水出北,東,東,東逕左川縣故城南,又東逕東陽縣,故城也,王莽之屬縣也。

已輸出: 15_新增文本前後對話回答比較.xlsx

測試 提示

模型

增加新文本前回答

「驗證性
研究新視野



士馬 淮水 縣水出山,江水出北,東,東,東逕左川縣故城南,又東逕東陽縣,故城也,王莽之屬縣也。

已輸出：15_新增文本前後對話回答比較.xlsx

測試組	提示句	模型	增加新文本前回答	增加新文本後回答	前後回答是否不同
0	1 爾朱榮	Multi-Head Attention	爾朱榮寺寺皇魏覺寺。酪歸所立也。形製甚有洛陽。工雕佛殿僧房充溢一里御道。	爾朱榮水出西北山在縣縣北山西北逕西,縣北,南逕西北,又東北流注於河。	是
1	1 爾朱榮	Mixture of Experts	爾朱榮統寺內圖積治義里寺。為寺。所謂永一里。南眺洛寺。	爾朱榮水出縣,地,故縣,山,水,縣,東,東逕於陽,故縣也。	是
2	2 洛陽	Multi-Head Attention	洛陽寺寺寺寺。在融覺寺西一里。在南有租場。南北方三去地。	洛陽水 卷 水 渭水 出河水 水 左水 左水 左水 水 左水 左水 左合州水出縣北山,北流逕...	是
3	2 洛陽	Mixture of Experts	洛陽武宣捨宅所立也。在明堂刻社龍南有穀水周圍取四園。	洛陽水出梁水 "池水 渭水 "北,西南流為,水出南山,北逕其山,東南流注於洛。	是
4	3 士馬	Multi-Head Attention	士馬射皆金色。不聽升寺。道比所立也。康敬。唯卷二尺。	士馬、武水 水 水 桓水 水 出南流逕長沙國縣北,又東北流,左會於洛,水出西山,北流逕其縣故城北。	是
5	3 士馬	Mixture of Experts	士馬老所立也。為憑王公。在寶光寺西。屋宇六造佛殿一所。	士馬 淮水 縣水出山,江水出北,東,東,東逕左川縣故城南,又東逕東陽縣,故城也,王莽之屬縣也。	是

已輸出：15_新增文本前後對話回答比較.xlsx



證據·脈絡·可驗證性
AI 賦能史學研究新視野

儲存模型並一次下載所有成果

[20]

✓ 1 秒



```
1 base_tokens=[sp.id_to_piece(i) for i in range(base_bpe_vocab_size)]
2 current_tokens=[current_token_label(i) for i in range(bpe_vocab_size)]
3 vocab_metadata={"base_sentencepiece_vocab_size":base_bpe_vocab_size,"extra_token_to_id":extra_token_to_id,"padding_id":pad_id}
4 torch.save({"state_dict":rnn_model.state_dict(),"embedding_dim":embedding_dim,"hidden_dim":rnn_hidden_dim,"tokens":base_tokens,"padding_id":base_bpe_vocab_size},OUTPUT
5 torch.save({"state_dict":attention_state_before,"embedding_dim":embedding_dim,"model_dim":attention_dim,"heads":attention_heads,"blocks":attention_blocks,"tokens":curr
6 torch.save({"state_dict":moe_state_before,"embedding_dim":embedding_dim,"model_dim":attention_dim,"heads":attention_heads,"experts":moe_experts,"top_k":moe_top_k,"toke
7 torch.save({"state_dict":attention_model.state_dict(),"embedding_dim":embedding_dim,"model_dim":attention_dim,"heads":attention_heads,"blocks":attention_blocks,"tokens
8 torch.save({"state_dict":moe_model.state_dict(),"embedding_dim":embedding_dim,"model_dim":attention_dim,"heads":attention_heads,"experts":moe_experts,"top_k":moe_top_k
9
10 manifest_df=pd.DataFrame([{"檔名":p.name,"大小bytes":p.stat().st_size} for p in sorted(OUTPUT_DIR.iterdir()) if p.is_file()])
11 manifest_df.to_csv(OUTPUT_DIR/"成果檔案清單.csv",index=False,encoding="utf-8-sig")
12 zip_base="/content/繁體中文文本_AI原理14階段整合教學_成果"
13 zip_path=shutil.make_archive(zip_base,"zip",OUTPUT_DIR)
14 print(f"共建立 {len(manifest_df)} 個成果檔。\\nZIP:{zip_path}")
15 display(manifest_df)
16 files.download(zip_path)
17
```

... 共建立 34 個成果檔。

ZIP: /content/繁體中文文本_AI原理14階段整合教學_成果.zip

檔名 大小bytes

0	01_清理後文本.md	104002
1	01_雜訊清理報表.xlsx	5656
2	02_N-gram分析報表.xlsx	1896415
3	03_TEI標註與DocuSky匯入表.xlsx	827349



繁體中文文本_AI原理14階段整

+

← → ↑ ↻

OneDrive > ... 繁體中文文本_AI原理14階段整合教學_成果

搜尋 繁體中文文本_AI原理14階段

+

新增

✂

📄

📁

🔗

🗑

↕ 排序

☰ 檢視

...

🖼 預覽

> 桌面

> 筆記

> 詹峻軒報帳用

> 德國掃描圖書資料

> 德國檔案館藏台灣史料資料庫

> OneDrive - 國立成功大學 Ratio

> Attachments

> NCKU-HIS-智慧人文

> 台灣醫師名鑑數據資料

> 助理工作

> 待整理文章資料

> 博士論文創建表單之史筆記

> 博士論文德翻中

> 應用程式

📁 桌面

↓ 下載

📁 文件

🖼 圖片

🎵 音樂

🎬 影片

💻 H:\

名稱	狀態	修改日期	類型	大小
01_清理後文本.md	✓	2026/9/2 上午 04:32	MD 檔案	102 KB
01_雜訊清理報表.xlsx	✓	2026/9/2 上午 04:32	Microsoft Excel 工作...	6 KB
02_N-gram分析報表.xlsx	✓	2026/9/2 上午 04:33	Microsoft Excel 工作...	1,852 KB
03_TEI標註與DocuSky匯入表.xlsx	✓	2026/9/2 上午 04:34	Microsoft Excel 工作...	808 KB
03_自動標註_TEI_P5.xml	✓	2026/9/2 上午 04:33	Microsoft Edge HT...	745 KB
04_bpe_corpus.txt	✓	2026/9/2 上午 04:34	文字文件	102 KB
04_BPE分析報表.xlsx	✓	2026/9/2 上午 04:34	Microsoft Excel 工作...	232 KB
04_繁體中文BPE.model	✓	2026/9/2 上午 04:34	MODEL 檔案	274 KB
04_繁體中文BPE.vocab	✓	2026/9/2 上午 04:34	VOCAB 檔案	32 KB
05_N-gram與BPE比較.png	✓	2026/9/2 上午 04:34	PNG 檔案	36 KB
05_N-gram與BPE比較.xlsx	✓	2026/9/2 上午 04:34	Microsoft Excel 工作...	10 KB
06_BPE_TOKEN字詞表.xlsx	✓	2026/9/2 上午 04:34	Microsoft Excel 工作...	195 KB
07_One-hot完整稀疏矩陣.npz	✓	2026/9/2 上午 04:35	NPZ 檔案	10 KB
07_One-hot編碼報表.xlsx	✓	2026/9/2 上午 04:35	Microsoft Excel 工作...	75 KB
08_Word2Vec_FFNN向量報表.xlsx	✓	2026/9/2 上午 04:35	Microsoft Excel 工作...	2,505 KB
08_Word2Vec訓練Loss.png	✓	2026/9/2 上午 04:35	PNG 檔案	49 KB
09_RNN訓練與誤差報表.xlsx	✓	2026/9/2 上午 04:36	Microsoft Excel 工作...	7 KB
09_RNN誤差圖.png	✓	2026/9/2 上午 04:36	PNG 檔案	145 KB
09_RNN模型.pt	✓	2026/9/2 上午 04:57	PT 檔案	1,606 KB
10_Attention微調前模型.pt	✓	2026/9/2 上午 04:57	PT 檔案	3,417 KB
10_Attention與RNN比較報表.xlsx	✓	2026/9/2 上午 04:37	Microsoft Excel 工作...	8 KB

36 個項目

☰



· 可驗證性
研究新視野